



BIG DATA

Introduction

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

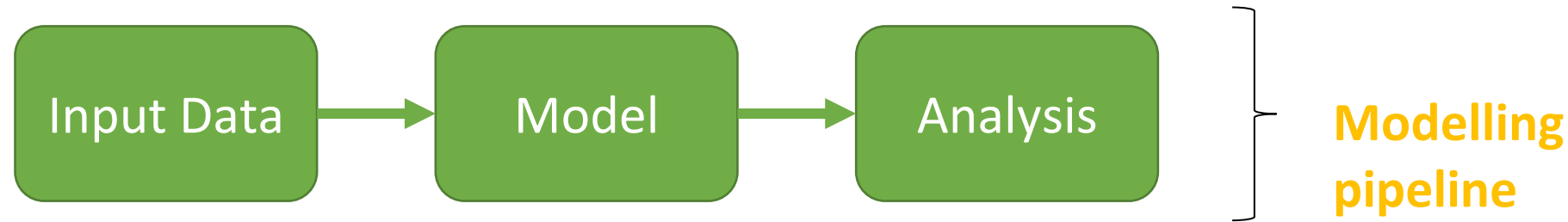
Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

There is no one standard single definition.

Big Data is data whose scale, diversity, and complexity require new forms of processing using new architectures, techniques, algorithms, and analytics to manage and enhance decision making to extract value and hidden knowledge or insights from it...

There is no one standard single definition.

Big Data : Big Data is transforming industries by enabling data-driven decision-making. For example, **in healthcare**, Big Data analytics can **predict disease outbreaks** and **improve patient care** by analysing vast amounts of health records. Customized health care is becoming a buzz word to cater to individual differences in terms of genetics, age, gender etc.



Model

- is a human construct that better helps us understand real-world systems/phenomena.
- Represents the inherent properties of the data
- Can represent a map or schema with groupings of the data elements such as records or tables and their associations
- Does not depend on the software using that data

- A **database (DB)** is a grouping of tables for collecting data. A table provides a systematic way to access, update, and manage data
- A database serves as a repository for querying the information needed for analytics, processes, intelligence, and knowledge discovery. A database can be distributed across a network of servers and data warehouses.
- A **table** is a representation consisting of rows and columns. The values in these fields can be a number, date, hyperlink, image, object, or text.

FLAT FILE:

- A **flat file** is a file where data must be read from the beginning to be interpreted, as it cannot be accessed in the middle.
- A **name-value pair** is a construct where a field has a name and a corresponding value, for example, "data, number."
- A **hash key-value pair** is a similar construct, but it uses a hash function to compute a key for indexing, searching, and distributing the entries across an array of slots (also called buckets).

Processing and Analytics:

- **Batch processing** is the execution of a series of transactions without user interaction.
- **Interactive transaction processing** involves a continuous exchange of information between a computer and a user, such as a UI during e-banking.
- Interactive processing requires low latency, which can be achieved through massively parallel processing (MPP), in-memory databases, and columnar databases.
- **Real-time processing** is the processing of live data (streaming) with low latency to obtain results for decision-making as the data is generated.
- **Stream analytics** is a method of continuous computing where data flows through a system as events occur. This is used for processing log streams and Twitter streams.
- **Stream transaction processing** involves specialized frameworks for real-time streaming computations, such as Storm, S4, Spark Streaming, HStreaming, and Flume.
- **In-memory** refers to operations that use a computer's main memory, like RAM or caches. Data is loaded from a disk or an external source, and these in-memory operations are much faster than disk access.

- The **ETL (Extract, Transform, and Load)** process is a method for retrieving, integrating, transforming, and storing data.
- **Descriptive analytics** involves deriving additional value from visualizations and reports.
- **Predictive analytics** is an advanced method that extracts new facts and knowledge to predict or forecast future outcomes.
- **Prescriptive analytics** derives additional value and helps in making better decisions for new options, such as maximizing profits.
- **Cognitive analytics** is the analysis of sentiments and emotions through processes of learning, understanding, and representing.
- A **service** is a mechanism that provides access to one or more capabilities. An interface provides this access, while a service description specifies the constraints and policies.
 - Examples include web services, cloud services, and BigQuery services.

Service-Oriented Architecture (SoA) is an architectural style that consists of services, messages, operations, and processes.

Not to be confused with [WHTSAPP](#) or [WHTSAP](#).

WhatsApp Messenger, commonly known simply as **WhatsApp**, is an American [social media](#), [instant messaging](#) (IM), and [Voice over IP](#) (VoIP) service accessible via desktop and [mobile app](#).^{[13][14]} Owned by [Meta Platforms](#), the service allows users to send [text messages](#), [voice messages](#), and video messages,^[15] [make voice and video calls](#), and share [images](#), [documents](#), [user locations](#), and other content.^{[16][17]} The service requires a [cellular mobile telephone number](#) to register.^[18] WhatsApp was launched in May 2009.^{[19][20]} In January 2018, WhatsApp released a standalone business app called WhatsApp Business which can communicate with the standard WhatsApp client.^{[21][22]} As of May 2025, the service had [3 billion monthly active users](#),^[23] making it the most popular messenger app.^[24] The name of the app is meant to sound like "[what's up](#)".

WhatsApp Messenger



Screenshot

[\[show\]](#)

Original authors	Brian Acton, Jan Koum
Developer	Meta Platforms ^{[1][2]}
Release	May 3, 2009; 17 years ago
	Stable release(s) ^[±]
iOS	26.10.74 ^[3]  / March 20, 2026; 4 months ago
Android	2.26.10.72 ^[4]  / March 14, 2026; 4 months ago

- **Data** is information derived from a series of observations, measurements, or facts.
- **Webdata** refers to the large-scale integration of data from the web.
- Data can be categorized into three main types:
 - **Structured Data**
 - Structured data is highly organized and follows a predefined schema or data model. It is typically found in a tabular format, like rows and columns in a relational database or a spreadsheet.
 - Structured data is easy for both humans and machines to read, search, and analyze using conventional tools like SQL.
 - **Semi-structured Data**
 - Semi-structured data does not conform to the rigid, tabular structure of a relational database but still contains some organizational properties. It has a flexible schema that can evolve dynamically, and it often uses markers or tags to create a hierarchy and establish relationships. JSON, XML, and CSV files are common examples.
 - **Unstructured Data**
 - Unstructured data is information that does not have a predefined format or schema. It is the most common type of data, making up a significant portion of all data generated.
 - Unstructured data is challenging to store and analyze using traditional database methods, as it lacks a clear organization. Examples include text documents, emails, social media posts, images, audio files, and videos

The trend is for every individual's data footprint to grow, but perhaps more significantly, the amount of **data generated by machines** as a part of the Internet of Things will be even greater than that generated by people.

Machine logs, RFID readers, sensor net works, vehicle GPS traces, retail transactions—all of these contribute to the growing mountain of data.

- **Data Management:**
 - Effective big data management focuses on the infrastructure and processes needed to store, process, and govern massive datasets. This is a foundational theme that makes analytics possible.
- **Data Analytics:**
 - Once the data is properly managed, the next theme is to apply analytical techniques to extract value and knowledge. This is where insights are uncovered to drive decision-making.
- **Google's Search System:**
 - Google's approach to search involves a complex, automated system of web crawlers that find and index billions of pages. When a user queries, the system analyzes many factors—including the words, content, and expertise of sources—to return the most relevant and reliable information. (More details in Unit-2)
- **Amazon's recommendation system:**
 - Amazon's approach is a prime example of big data analytics in action. It uses machine learning algorithms and collaborative filtering to analyze vast amounts of user data, such as purchase history, Browse behavior, and product ratings.

- Machine Translation
- Translating a sentence from English -> Hindi
- What would be the traditional approach?
- How will it differ from the Big Data approach?

Watching movies and shows in different languages: over a time, be able **to learn the language**

English

Can you teach me?

You make mistakes if you do things in a hurry.

Hindi

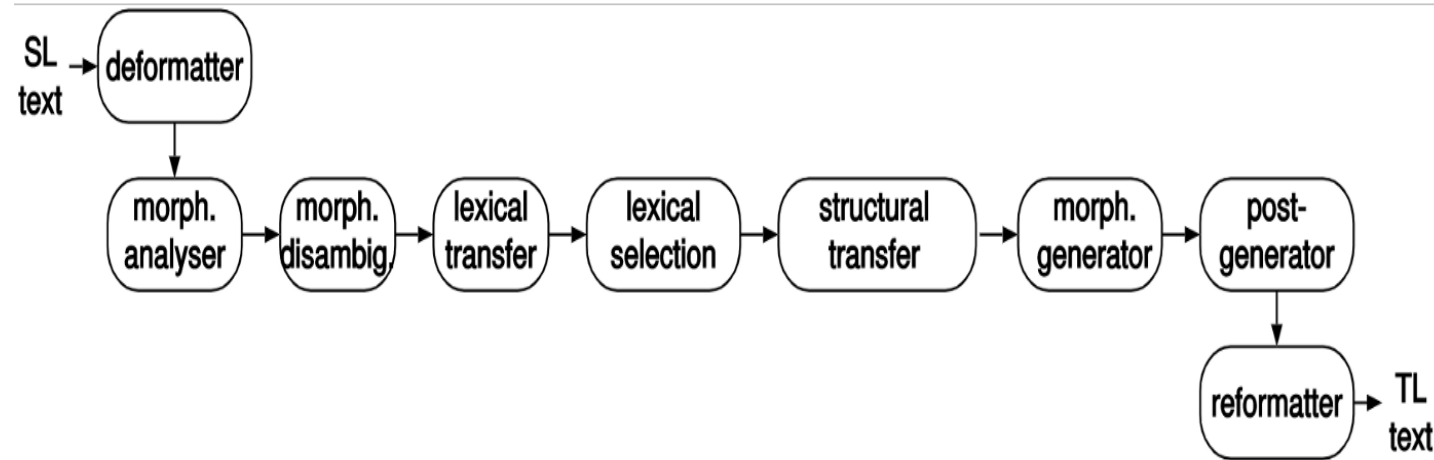
क्या तुम मुझे सिखा सकते हो?

जल्दबाजी में काम करोगे तो गलतियाँ तो होंगी ही।

<https://www.geeksforgeeks.org/artificial-intelligence/statistical-machine-translation-of-languages-in-artificial-intelligence/>

<https://lokalise.com/blog/neural-machine-translation/#:~:text=Instead%20of%20using%20a%20list,%F0%9F%A7%A0%20How%20is%20AI%20trained%3F>

- Understand the system – linguistic approach – rule based
- Requires a linguistic expert to build a model
- Model should include
 - Language structure -> morphology, grammar
 - Meaning of the words
 - Mapping words from one language to another

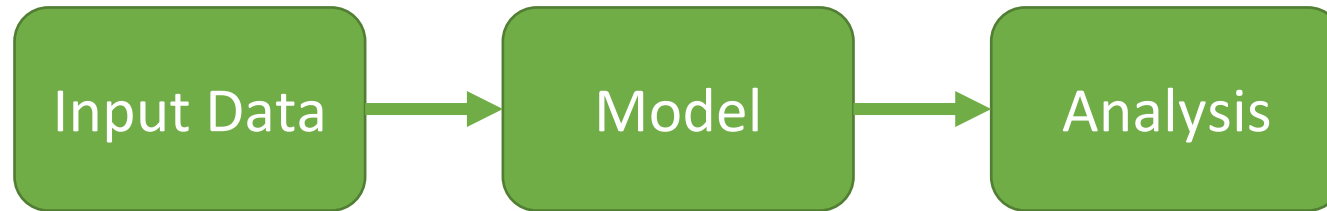


<https://reverieinc.com/blog/how-rule-based-machine-translation-works-a-deep-dive/>

- No attempt to understand language
- Gather data about different sentences and translations
 - Requires a parallel corpus
 - Millions of sentences and their translations
- Build a statistical model
- For example:
 - Every time the word cat appears in the English sentence
 - The hindi equivalent has *billi*
 - So infer that cat can be translated as billi

Corpus of Hindi-English language pair	
1. India is a vast country	1. भारत एक विशाल देश है
2. Delhi is the capital of india	2. दिल्ली भारत की राजधानी है
3. India has 29 states	3. भारत में 29 राज्य हैं

<https://techmediahub.com/machine-translation-complete-useful-guide/>



Traditional Approach

The model is **human** generated.

Big Data Approach

The model is **machine** generated.

What about domain knowledge?

- Correlation is enough?
- Gene sequencing of DNA fragments found in ocean by J. Craig Venter
 - 1000s of new species
 - No idea of what species looks like or any other info
- All models are wrong, and increasingly you can succeed without them
 - Peter Norvig, Google's research director
 - “The unreasonable effectiveness of data”

The End of Theory: The Data Deluge Makes the Scientific Method Obsolete

By Chris Anderson  06.23.08



What about domain knowledge?

- Can we rely only on data alone?
- Does this mean that **domain knowledge** is obsolete?



PES
UNIVERSITY

CELEBRATING 60 YEARS

Big Data: Pitfalls in Analysis

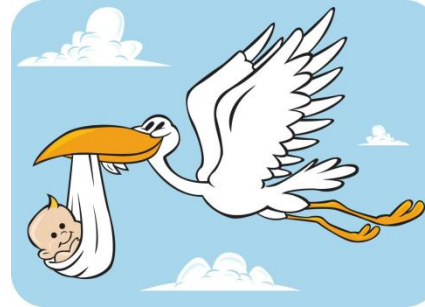


- What about *let the cat out of the bag*?
 - Naïve translation - *billi ko bag ke bahar chhod diya*
 - English meaning: reveal a secret
- To be able to solve this, we need information about the language -> domain knowledge and some experimentation

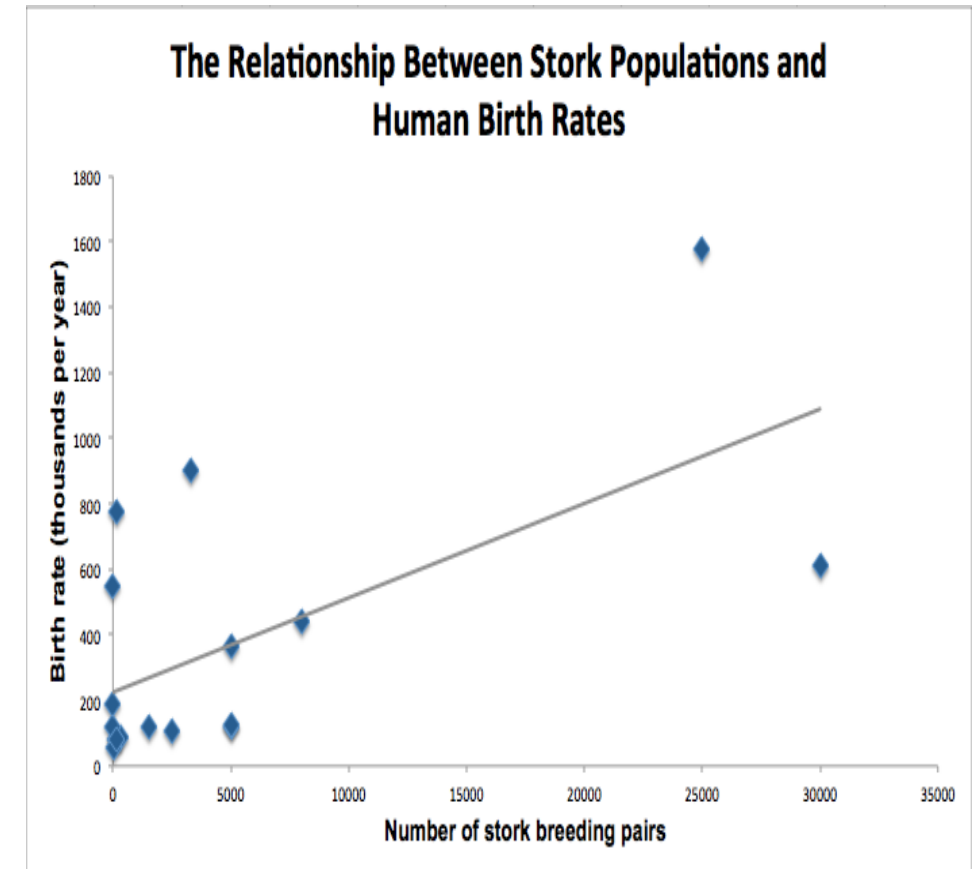


Alon Halevy, Peter Norvig, and Fernando Pereira, Google

- Peter Norvig, Head Google Research, *The Unreasonable Effectiveness of Data*
<https://www.youtube.com/watch?v=yvDCzhbjYWs>



- $C \rightarrow A, C \rightarrow B$
 - *Does $A \rightarrow B$?*
- Example:
 - Do storks deliver babies?
- Chart shows positive correlation between
 - Stork population and human birth rates in European countries
 - What it does not show is a hidden variable
 - Available nesting area?



- http://en.wikipedia.org/wiki/Spurious_relationship
- http://www.cut-the-knot.org/do_you_know/misuse.shtml

Pitfall : Gaps in the data

- **Selection bias** occurs when the data used for a study is not a representative sample of the larger population. This can lead to skewed or incorrect conclusions because the sample systematically excludes certain groups or characteristics.
- **convenience** often contributes to this bias. Researchers may choose to collect data from sources that are easy to access or readily available, which can result in a non-random and unrepresentative sample.
 - The Rutgers University study used tweets to examine emergency decision-making during Hurricane Sandy.
 - This method was convenient, but the resulting data had a significant **selection bias** because most of the tweets came from a single location: **Manhattan**.
 - The sample of tweets was not representative of all areas impacted by the hurricane.
 - Therefore, using this data to assess the overall impact of Sandy would lead to a skewed and **incorrect conclusion that Manhattan was the most affected area.**



- *More Data, More Problems: Is Big Data Always Right?* ARI ZOLDAN <http://www.wired.com/insights/2013/05/more-data-more-problems-is-big-data-always-right/>

- Another example: medicine
- Missing data is always a challenge
 - but we also know that “negative results” are more likely to go missing.
 - This means we have a biased sample, overestimating the benefits of treatments.



- *The Information Architecture of Medicine is Broken* Ben Goldacre
<http://strataconf.com/strata2012/public/schedule/detail/22941>
- https://www.youtube.com/watch?v=AK_EUKJyusg



PES
UNIVERSITY

CELEBRATING 50 YEARS

Big Data: How to address the issues?



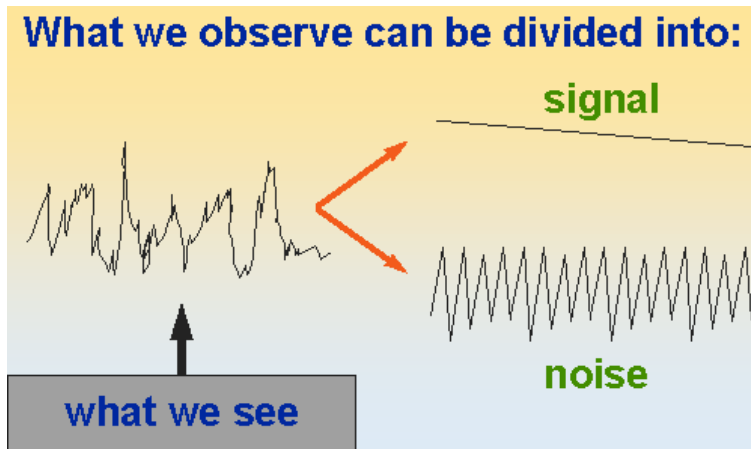
- **Use domain knowledge to check model for validity**
- **Estimate errors**

BIG DATA

Let's look to some experts

- Nate Silver book
 - The Signal and the noise
- On Time Magazine 2009 – 100 most influential people
- Correctly predict US 2008/2012 elections

the signal and the noise and the noise and the noise and the noise why so many predictions fail – but some don't and the noise and the noise and the nate silver noise



https://www.forecastwatch.com/wp-content/uploads/2018/09/Nate_Silver_The_Signal_and_the_Noise.pdf

<https://www.theguardian.com/science/grrlscientist/2012/nov/08/nate-silver-predict-us-election>

Why is weather forecasting very successful?

Chaotic (dynamic, nonlinear system)

- Lorenz: 29.5168 instead of 29.517

Adjustment by humans

- Compute probabilities: how often predict rain, didn't rain?
- On ground reality

The effect of marketing/customer satisfaction in commercial weather forecasting. Embracing probabilistic thinking.

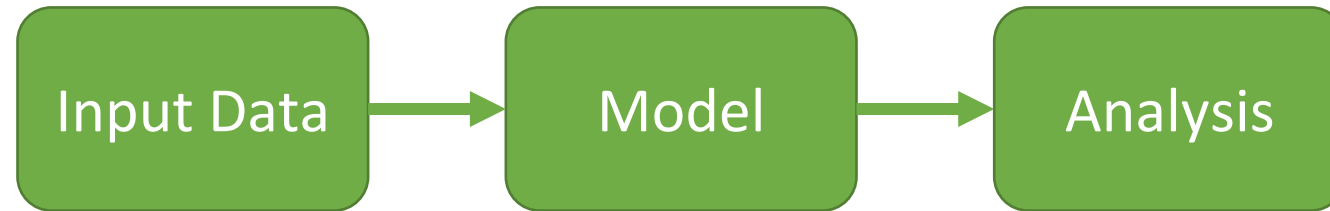
- More sensitive about errors in predicting no rain than rain

Example: Collapse of housing market 2008

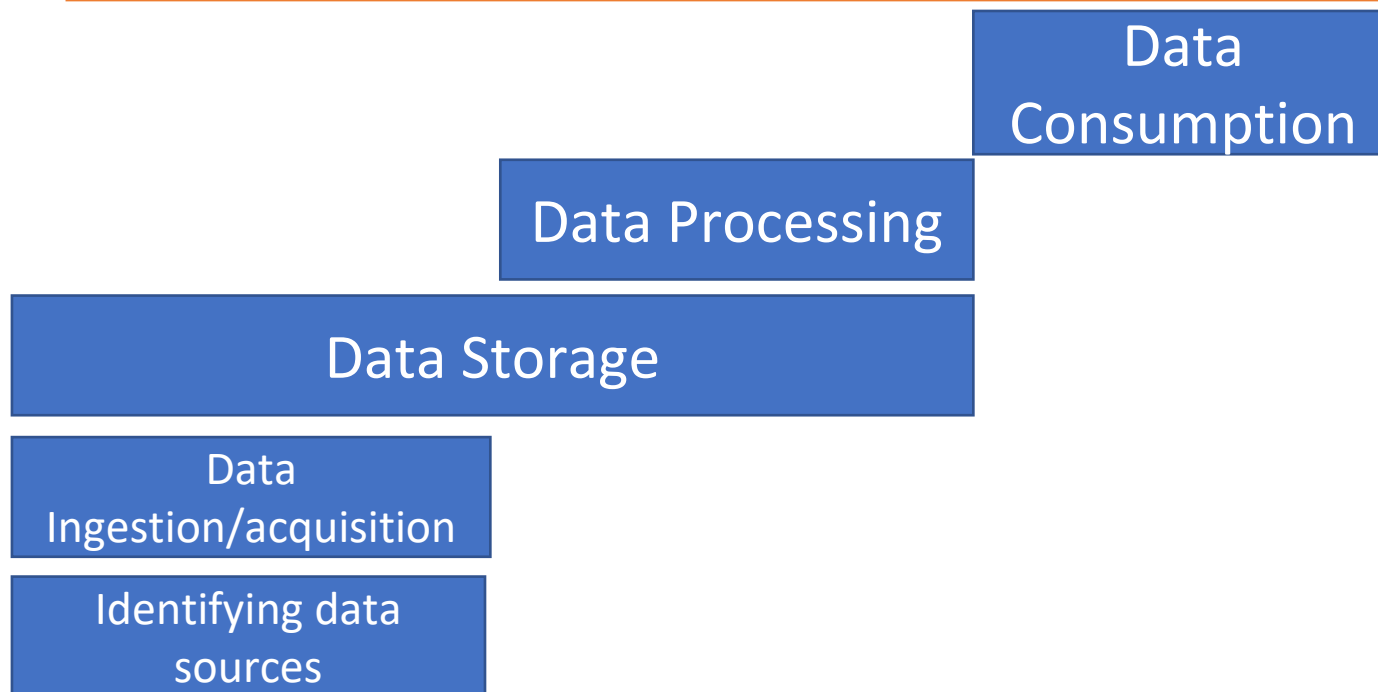
BIG DATA

- 1 The financial companies claimed that they didn't see it coming.
- 2 Were there economists who correctly predicted the collapse of the housing market? Was there a basis in the data for such prediction?
- 3 The problem with risk calculations for combined mortgages: the *independence* assumption.
- 4 Never predict for a situation you've never had based on many situation you've had before, i.e. **when you're *out of sample***.

- **Purely empirical**, meaning it is based on observation and data rather than theoretical analysis.
- To implement this, data is divided into two parts: a **training set** and a **testing set**.
- An algorithm is developed using the **training set**, and its accuracy is then measured by estimating its error on the unseen **testing set**.
- This method is a standard way to compare different analytics algorithms to see which one performs best.
 - Examples of this approach include Nate Silver's weather prediction, which uses human adjustments, and Amazon's recommendation system.
 - For **Amazon recommendations**, a model is derived from historical customer data, and then it is used to make recommendations. Statistics are collected on how many people interact with or purchase these recommendations to evaluate the model's effectiveness.



Big Data Pipeline



Management



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Characteristics and Architecture

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

1. Big Data – History
2. Big Data - Characteristics
3. Data Architecture Design
4. Data Format/Types
5. Big Data Platforms
6. Case Study - Google

- **Big Data's Foundation:** Big Data is a fusion of established statistics and the newer field of computer science, building on existing knowledge from mathematics and data analysis.
- **The Data Deluge:** A significant shift occurred in the 21st century, where the volume and speed of data generation exploded. The total amount of data in the world grew from 4.4 zettabytes in 2013 to a projected 44 zettabytes by 2020. This massive, unmanageable volume of data transformed traditional data analysis into what is now known as "Big Data."

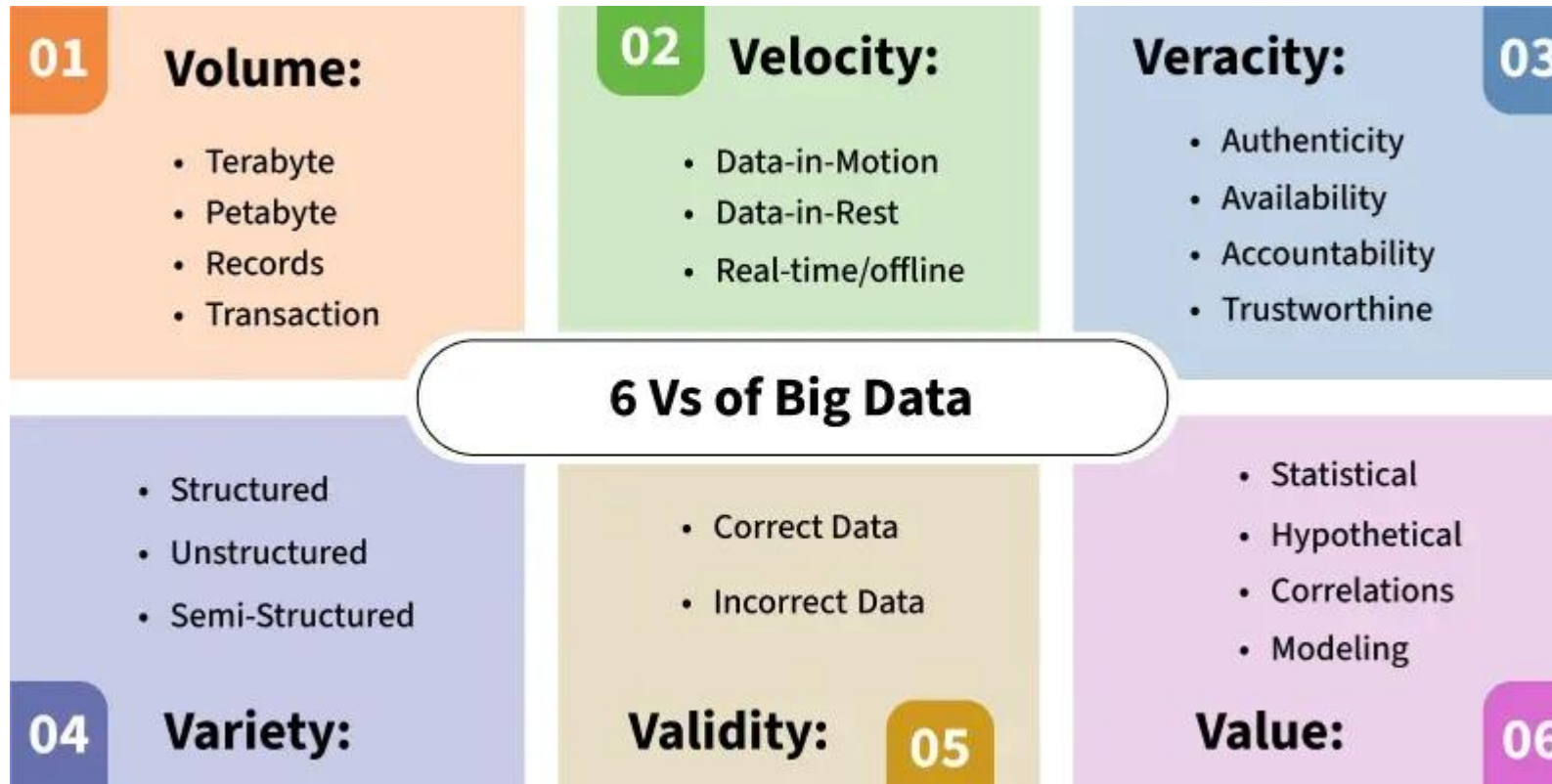
Phases of Evolution: The article outlines three key phases in the history of Big Data:

- **Phase 1 (1970s): Structured Content.** This phase was driven by database management and technologies like SQL.
- **Phase 2 (Early 2000s): Web-Based Unstructured Content.** The rise of the internet and web applications led to the creation of vast amounts of unstructured data from sources like search queries and user interactions.
- **Phase 3 (Current): Mobile and Sensor-Based Content.** The current era is defined by the proliferation of mobile devices and the "Internet of Things" (IoT), which generate high-volume, real-time data every second.

Big Data Phase 1 – Structured Content	Big Data Phase 2 – Web Based Unstructured Content	Big Data Phase 3 – Mobile and Sensor Based Content
Period: 1970-2000	Period: 2000 – 2010	Period: 2010 - Present
• RDBMS & data warehousing • Extract Transfer Load • Online Analytical Processing • Dashboards & scorecards • Data mining & statistical analysis	• Information retrieval and extraction • Opinion mining • Question answering • Web analytics and web intelligence • Social media analytics • Social network analysis • Spatial-temporal analysis	• Location-aware analysis • Person-centred analysis • Context-relevant analysis • Mobile visualization • Human-Computer-Interaction

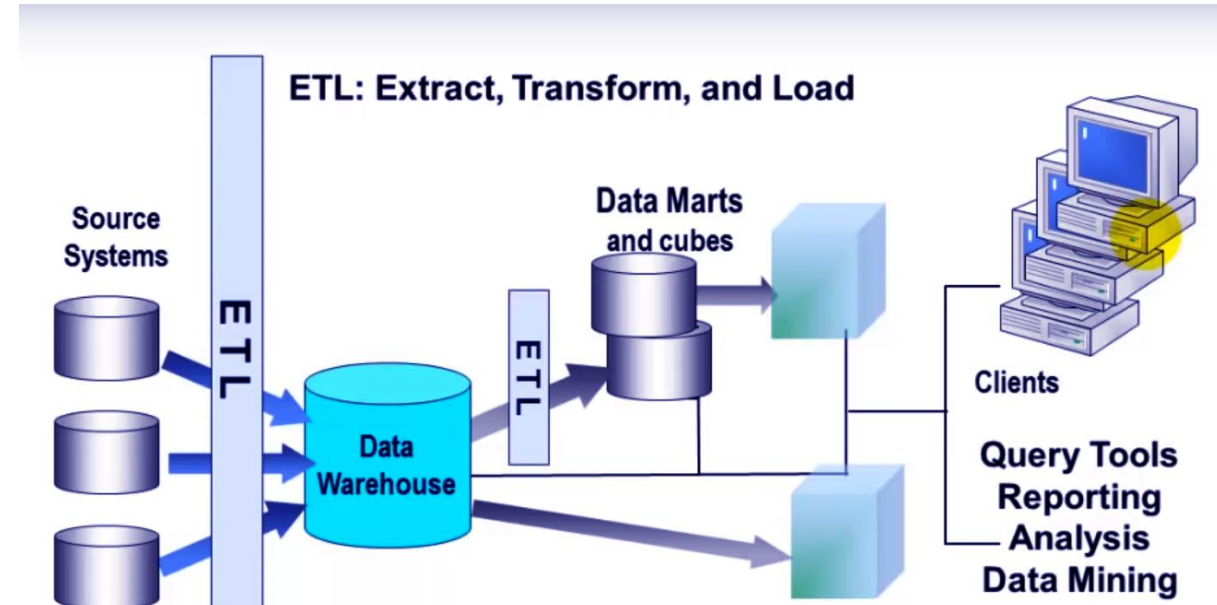
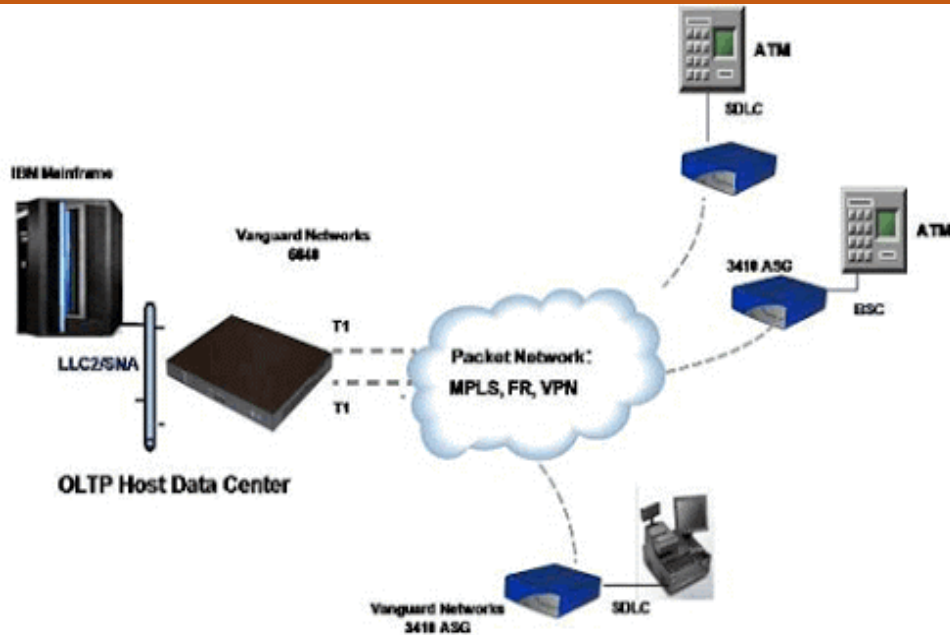
BIG DATA

Characteristics – Quick Summary



Big Data: Characteristics - Volume



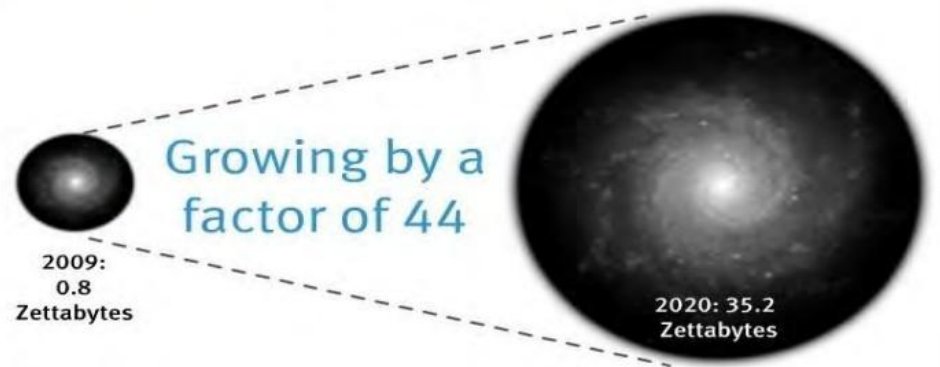


<https://data-flair.training/blogs/business-intelligence-and-data-warehousing/>

- Data is characterized by a **fixed schema and format**, ensuring consistency and structure.
- The data is **clean** and **consistent**, making it reliable for analysis.
- Data rates are **predictable**, which simplifies planning and processing.
- An elaborate **ETL (Extract, Transform, Load) procedure** is used, which is not real-time and is often run in batches, perhaps once a day.
- The output primarily consists of **reports**, which are generated by running queries in batches that can take several hours to complete.

- Old Style Data vs Big Data
1-Scale (Volume)
 - 44x increase from 2009 2020
 - From 0.8 zetta bytes to 35zb
 - Data volume is increasing exponentially

The Digital Universe 2009 to 2020



Equivalent to a stack of DVDs in 2009 reaching to the moon and back, now reaching halfway to Mars by 2020

BIG DATA

Characteristics - Volume

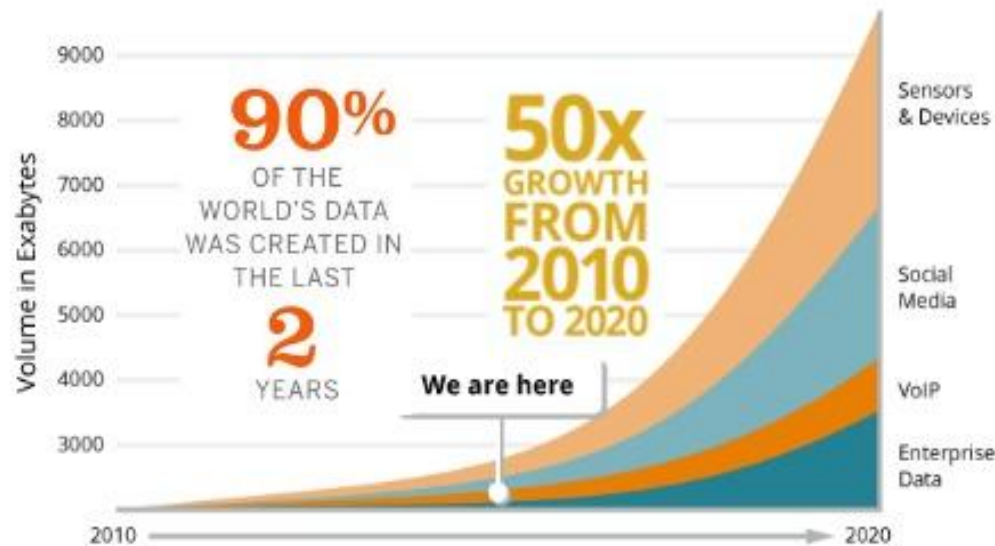
- Why is the volume so high? Who's Generating it?

CONTEXT: WHAT'S BIG DATA?

7

BIG IN GROWTH, TOO.

1 exabyte (EB) = 1,000,000,000,000,000 bytes



Old Style Data

Enterprises
only

Big Data

Everybody

BIG DATA

Characteristics - Volume

- How this VOLUME gets collected?

Old Style Data	Big Data
Manual entry	Automated

- Financial Services

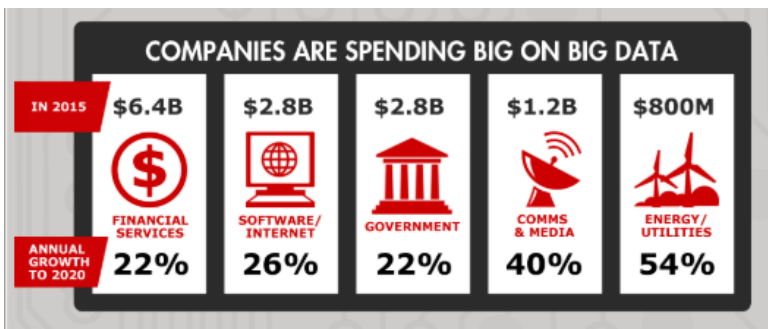
- Stock trading, banking, financial services are all computerized (In India – no paper shares any more)
- Every transaction is recorded

- Energy / Utilities

- Can put sensors in every home
- Smart grid

- Comms & media

- Internet services



<https://www.forbes.com/sites/louiscolumbus/2014/06/24/roundup-of-analytics-big-data-business-intelligence-forecasts-and-market-estimates-2014/#26c06d11388e>

Big Data: Characteristics - Variety



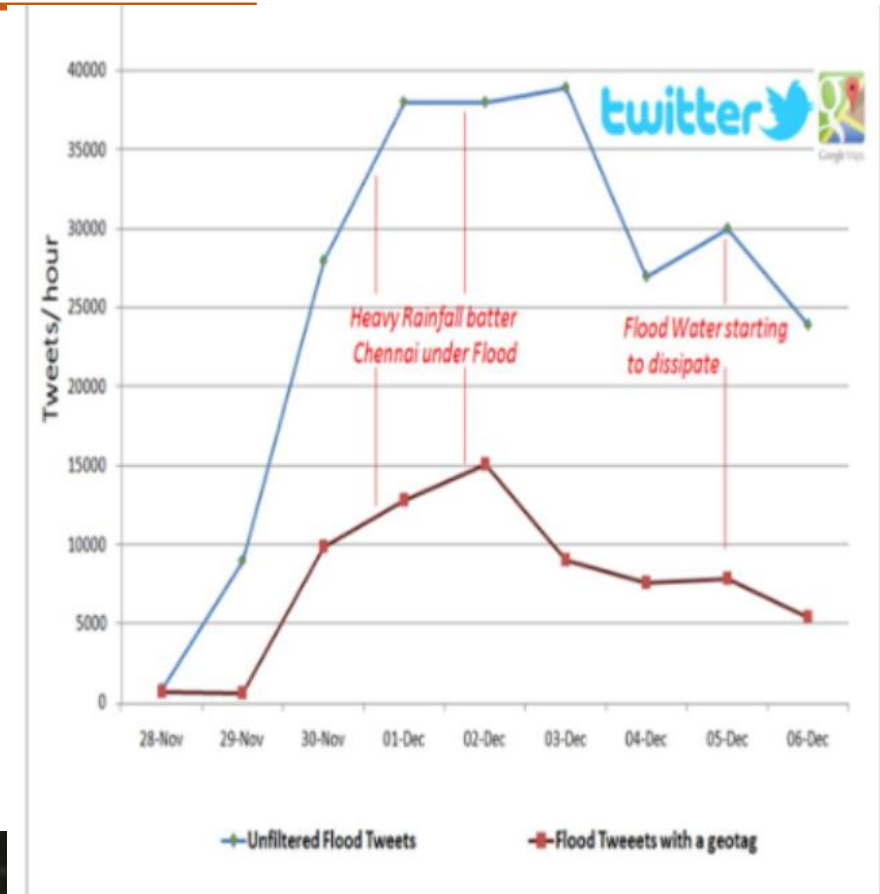
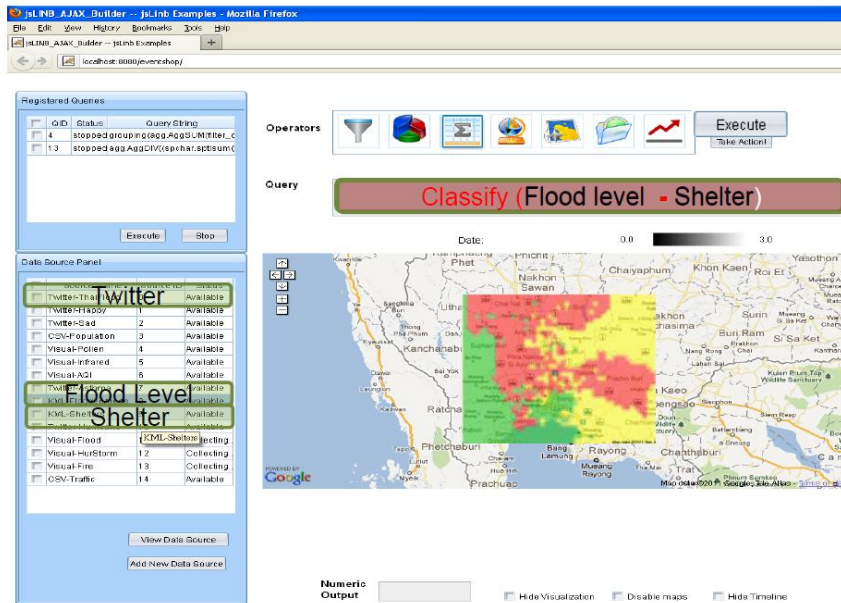
- Why is analyzing Big Data *complex*?
- Various formats, types, and structures
- Text, numerical, images, audio, video, sequences, time series, social media data, multi-dim arrays, etc...
- Static data vs. streaming data
- A single application can be generating/collecting many types of data

To extract knowledge → all these types of data need to be linked together



Old Style Data	Big Data
Fixed format / schema	Integrate Twitter, Maps, Facebook...

EventShop: Thai Floods



SHE SAYS

How Twitter transformed itself during
#ChennaiFloods for me

https://www.researchgate.net/publication/313452567_ChennaiFloods_Leveraging_Human_and_Machine_Learning_for_Crisis_Mapping_during_Disasters_Using_Social_Media

Big Data: Characteristics - Velocity



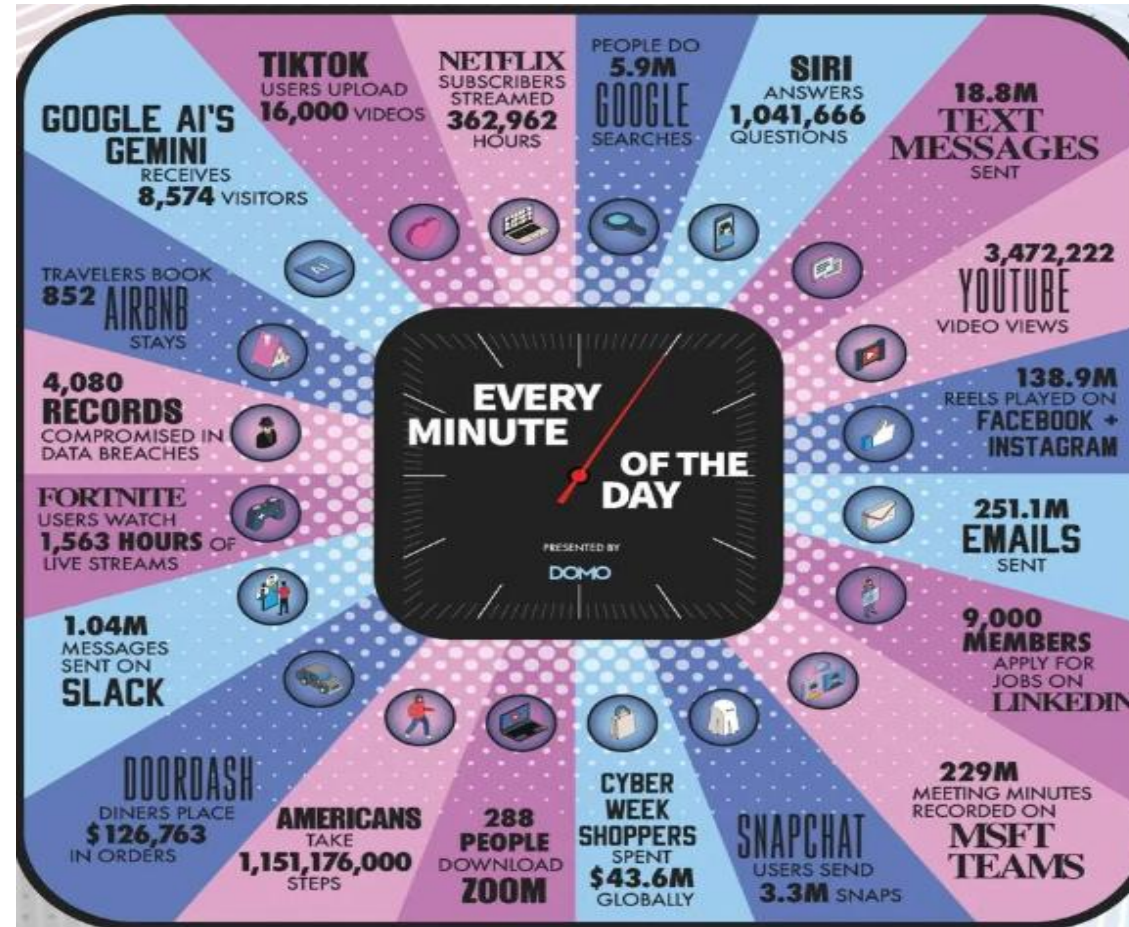
BIG DATA

Characteristics - Velocity

THE INTERNET IN **2023** EVERY MINUTE



Created by: eDiscovery Today & LTMG



2026

https://go.forrester.com/blogs/14-05-27-boost_your_digital_intelligence_with_big_data/

<https://bondhighplus.com/blog/what-happens-in-an-internet-minute/>

<https://ediscoverytoday.com/2023/04/20/2023-internet-minute-infographic-by-ediscovery-today-and-ltmg-ediscovery-trends/>

Nokia launches real-time mobile network analytics platform

Staff writer | July 28, 2016 | Telco Analytics Asia

<https://www.nokia.com/about-us/news/releases/2016/06/30/nokia-real-time-mobile-network-analytics-provides-instant-correlation-of-network-wide-data-and-its-impact-on-customer-experience/>

The analytics link the performance of applications and devices to network issues in real-time, effectively making every mobile device part of a network test bed.

→ This enables operators to pinpoint potential causes of service degradation much more rapidly than they can today since they no longer need to consult a myriad of tools and correlate the data from them manually. It also offers engineering teams a proactive way to understand **over the top (OTT) application** impacts on the network and optimize opportunities.

Old Style Data	Big Data
Input data at night	Real-time input
Output daily report	Real-time response



PES
UNIVERSITY

CELEBRATING 50 YEARS

Big Data: Characteristics - Veracity



- **Veracity** refers to the messiness or trustworthiness of data, which can impact its quality and reliability.
- The **accuracy** of data can be difficult to control due to factors like hashtags, typos, and abbreviations, which are common in unstructured data sources.
- Analysts need to **work with such data** despite its inherent messiness, as it often contains valuable insights.
- In contrast, **traditional data** is typically manually entered into fixed fields, which results in a much lower chance of error and higher reliability.

Old Style Data	Big Data
Clean data	Inconsistent data

- How people get multiple PAN cards
- Aadhar and PAN details not consistent – but belong to the same person
- Voter ID card details – these most important proofs never cease to amaze us.
- Slight changes in name and address
 - PES University, 100 ft Road...
 - PES university, BSK III stage...
 - Pes University, Dwaraka Nagar...
- Names and addresses don't match
- Varying initials in different documents.
- But postman / courier will deliver to correct place!!!

BIG DATA

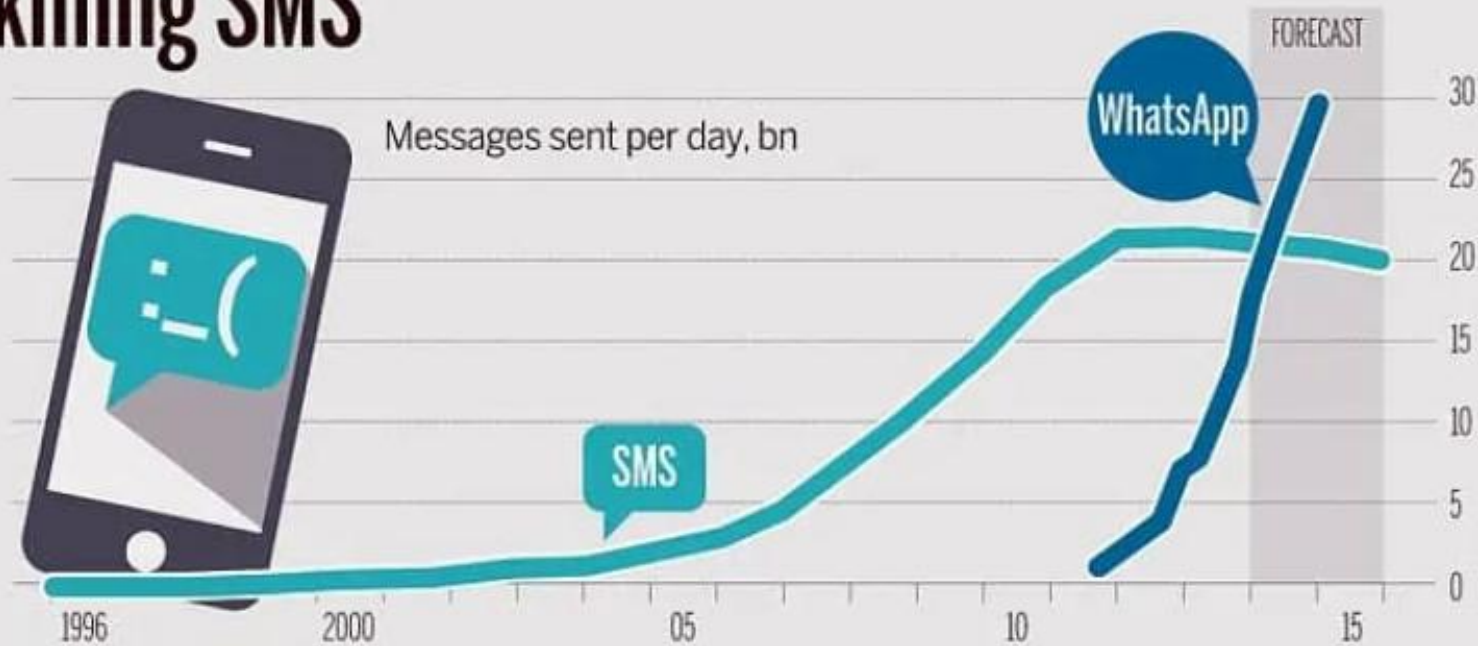
Characteristics – 4 V s

visual EDIT

NF newsflicks To download our visual news app, just sms NF to 52424

WhatsApp is killing SMS

A 4-year-old company is about to triumph over a nearly 20-year one on service—yet another instance of innovation unleashing creative destruction in technology industry



WhatsApp was eventually successful in replacing SMS

Big Data: Data Architecture Design



BIG DATA

Data Formats / Types

Structured Data



0.103	0.176	0.387	0.300	0.379
0.333	0.384	0.564	0.587	0.857
0.421	0.309	0.654	0.729	0.228
0.266	0.750	1.056	0.936	0.911
0.225	0.326	0.643	0.337	0.721
0.187	0.586	0.529	0.340	0.829
0.153	0.485	0.560	0.428	0.628

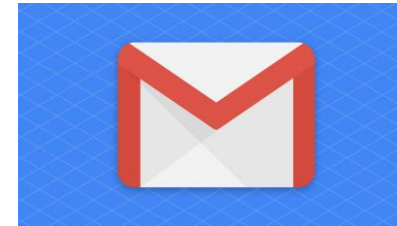
Databases

Unstructured Data

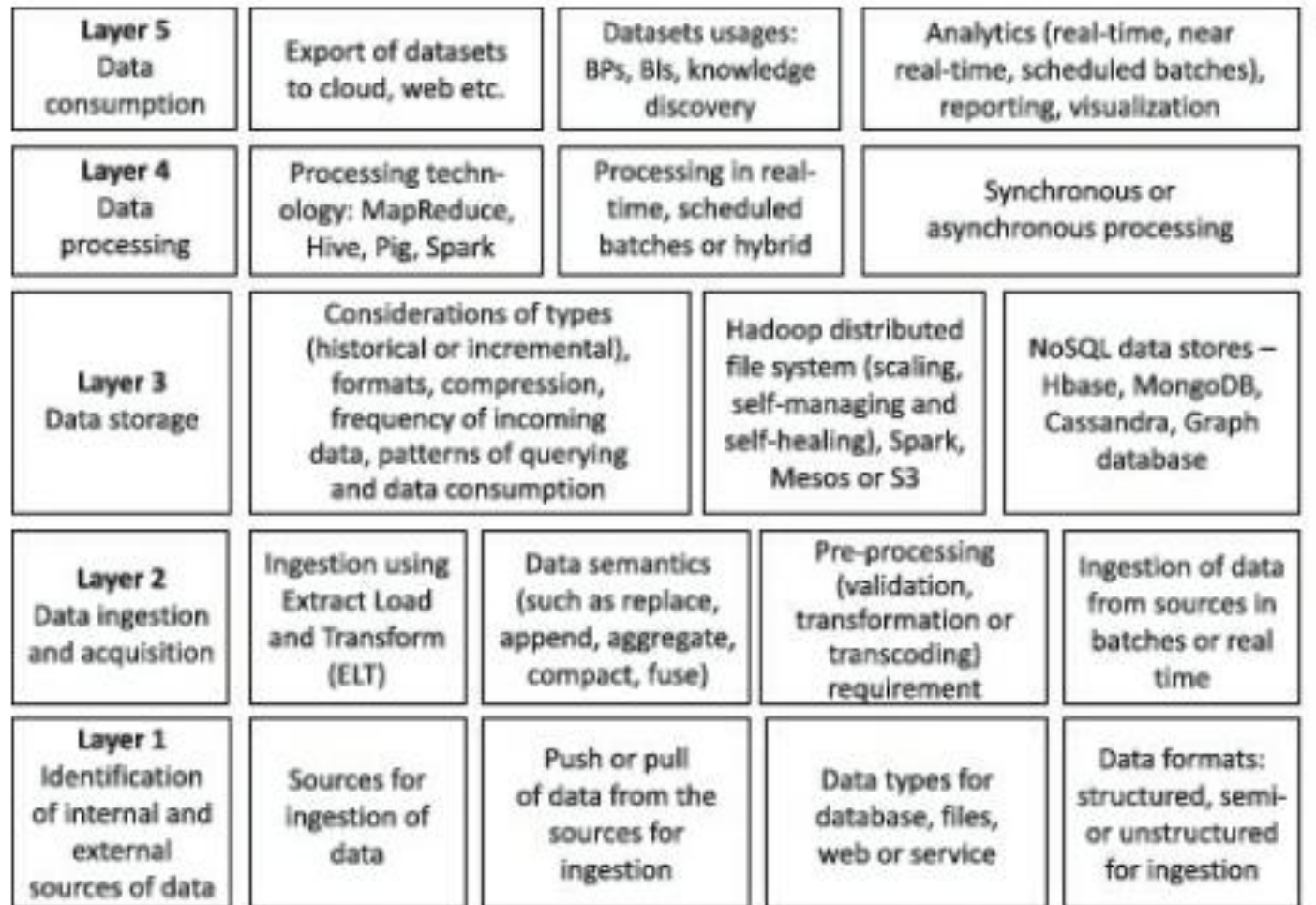


Documents, Tweets,
Videos

Semi Structured Data



Emails, XML

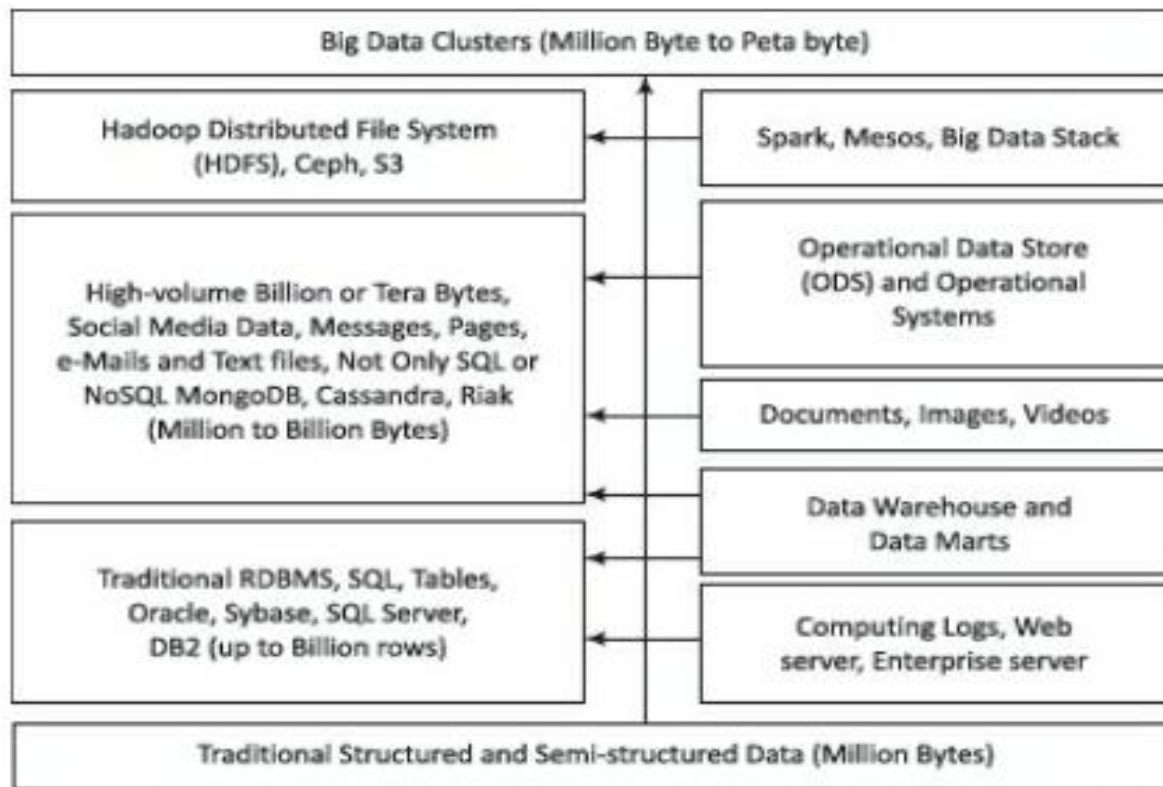


T1: Fig 1.2 – Logical layers in data processing architecture and their functions.

BIG DATA

Data Formats / Types : Big Data Storage

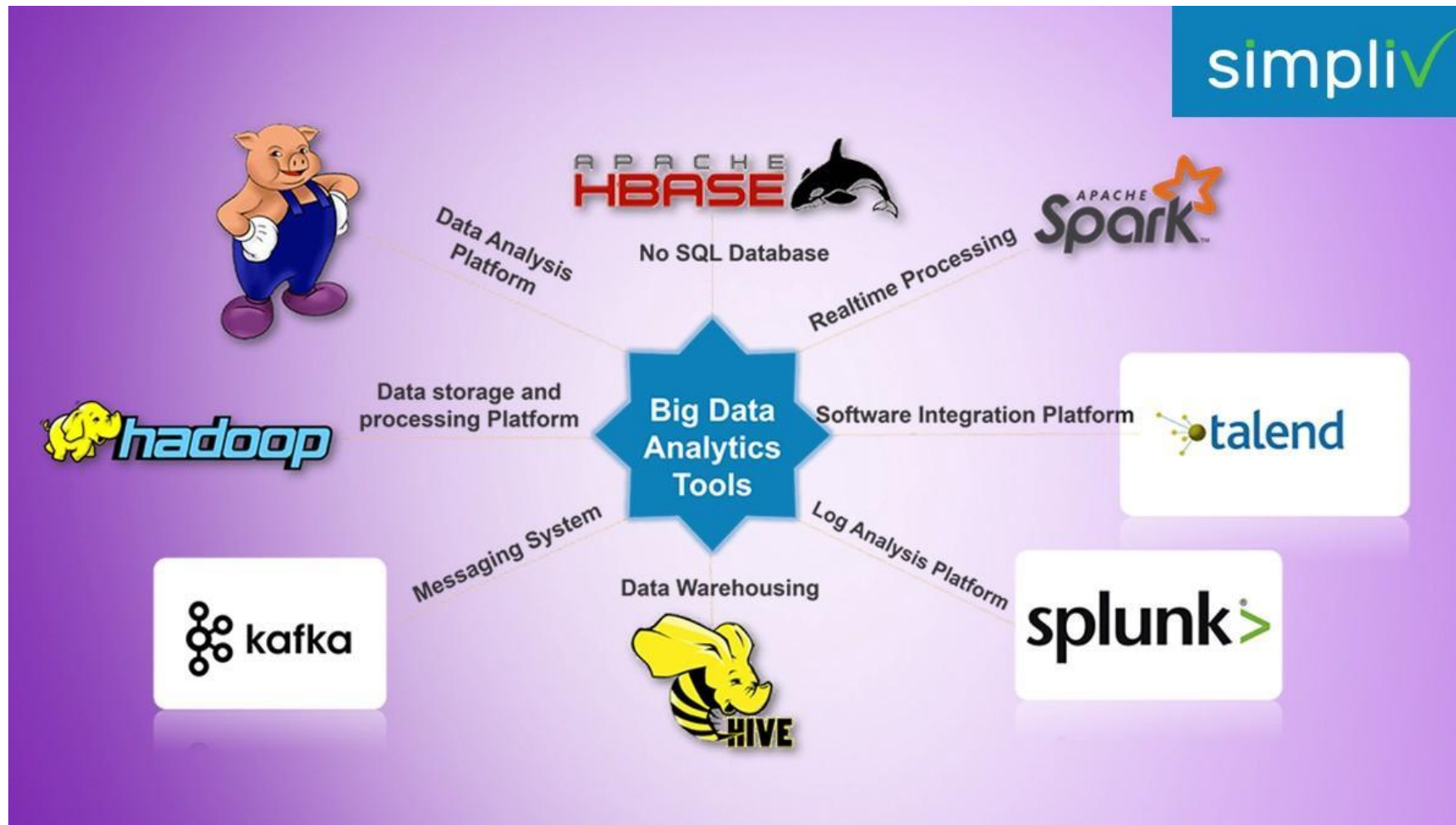
- Data storage for traditional and Big Data



T1: Fig 1.7 – Big data storage plan – RDBMS and NoSQL together

BIG DATA

Big Data Platforms

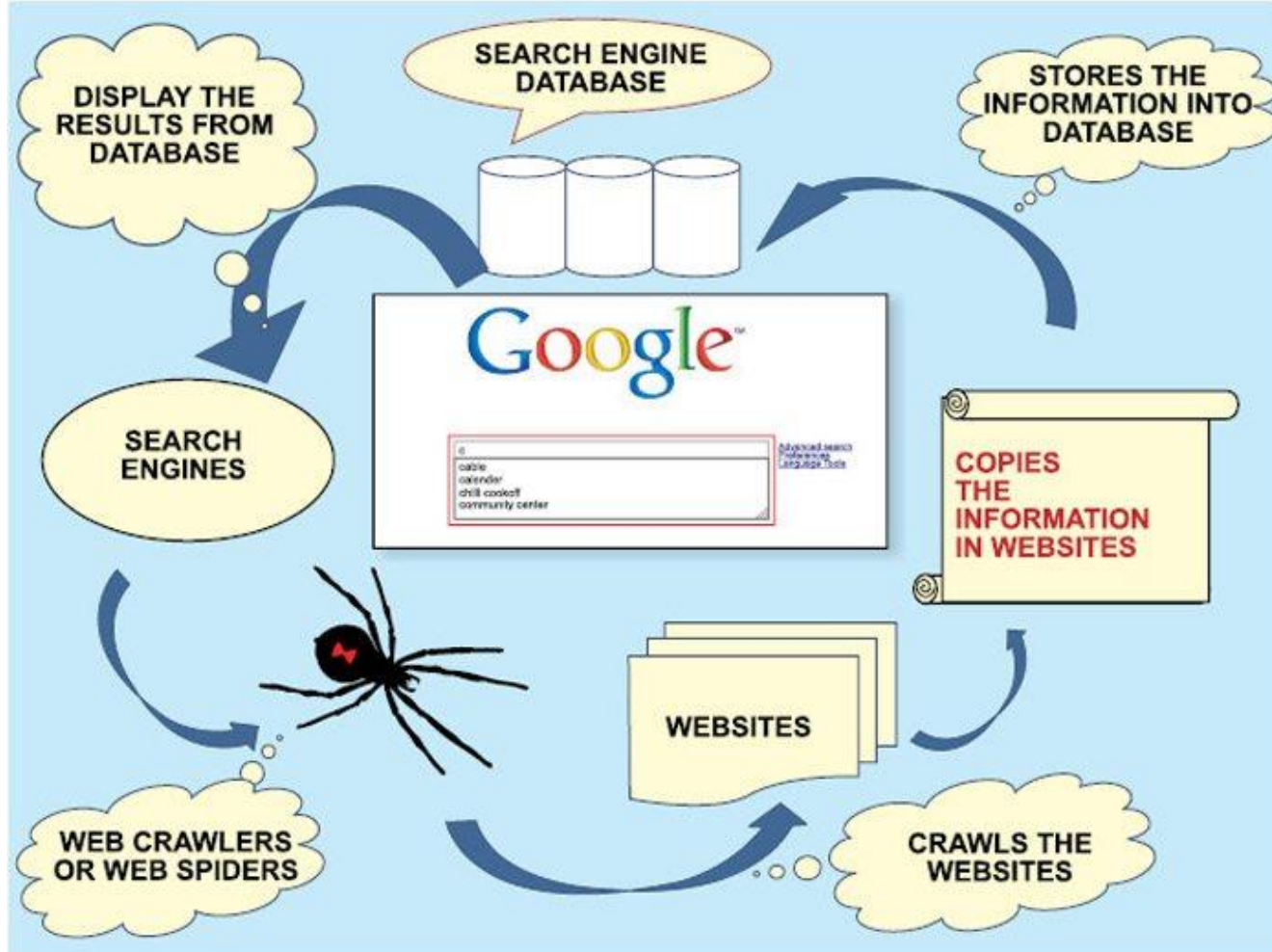


Big Data: Case Study – Google Search



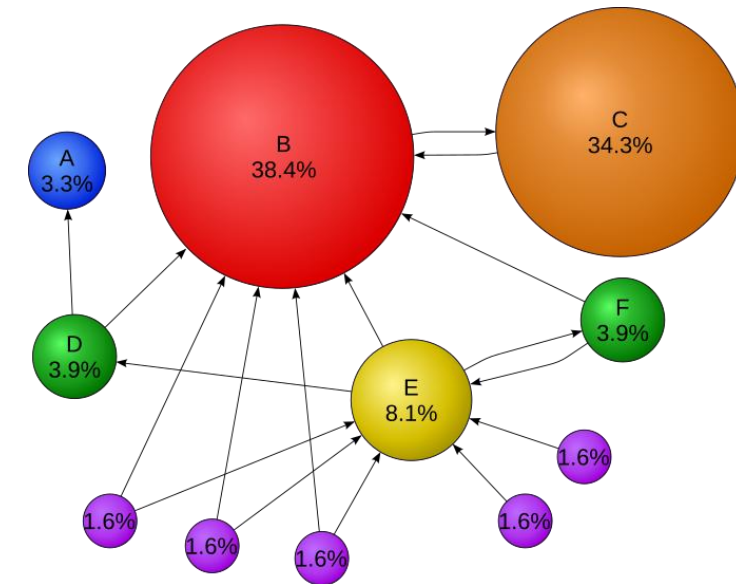
BIG DATA

Case Study: How Google Search works?



- Google initial implementation
 - 24 M web pages, 322 M links
 - 5 days to compute **Page Rank**
 - Page rank is proportional to the popularity of the page
 - If more links point to a page, that page will be more popular
- Page Rank – Treats the web as a graph
 - User -
 - Starts at a random page
 - Takes a random link
 - Page Rank Assumptions –
 - The more popular a page - The better is its quality

<https://en.wikipedia.org/wiki/PageRank>



Page Rank



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Hadoop Distributed File System (HDFS)

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Why do we need HDFS?

Let's take a real world example of financial transactions:

- Credit/debit card transactions generate large data.
- 1.3 billion transactions yield 13 TB data.
- This massive data needs efficient storage and processing.
- HDFS handles such large-scale data challenges.



Introduction to HDFS

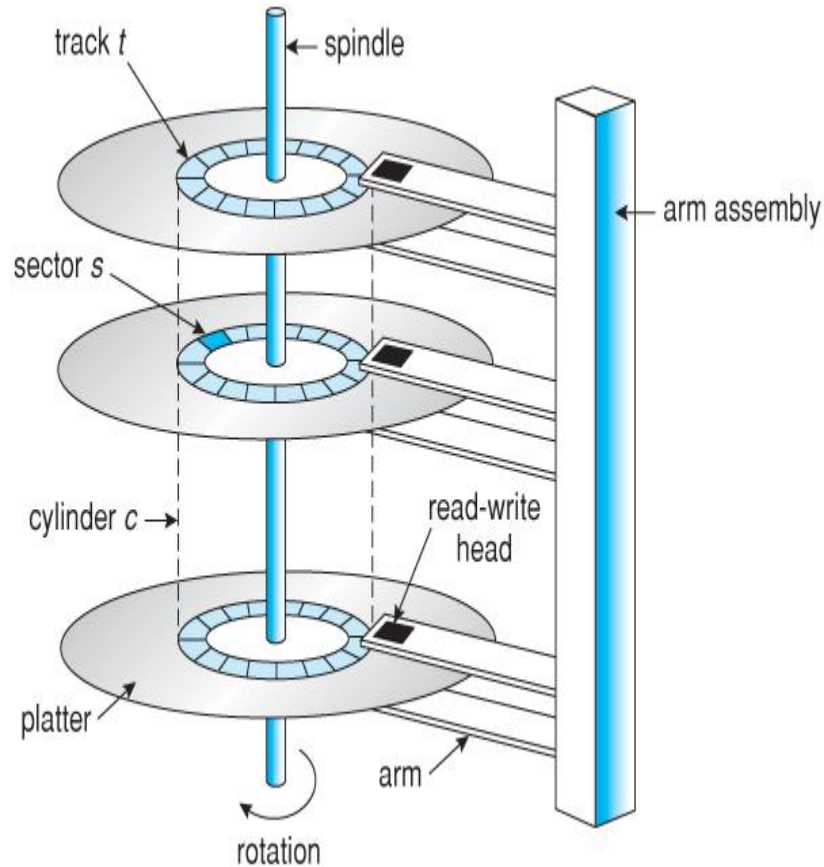
- HDFS stores very large files efficiently.
- It operates on clusters of commodity hardware.
- Designed for **write-once, read-many-times access**.
- Optimized for high throughput data processing on big data.
- HDFS was originally designed to overcome slow mechanical hard drives (HDDs)



File System Fundamentals - managing persistent data

- **Persistent Storage on Disks:**

- Disks are block-oriented devices; data transfer occurs in blocks (e.g., 4KB).
- Directly storing data on raw blocks raises concerns about data location and metadata management.
- File systems address these by providing a structured way to store and retrieve data.



File System Terminologies

- **File system** is used to control how data is stored and retrieved. Without a file system, information placed in a storage medium would be one large body of data with no way to tell where one piece of information stops and the next begins.
- Data is separated and grouped into pieces and given a name called a **file**
- **Files** thus are named collection of related information on disks
- Desirable properties of files
 - Long term existence (stored on disk or other secondary storage and do not disappear when a user logs off)
 - Sharable between processes (have names and can have associated access permissions that permit controlled sharing)
 - Structure (files can be organized into hierarchical or more complex structure to reflect the relationships among files)
- The **structure and logic rules** used *to manage the groups of information (files) and their names* is what forms a file system. It also has a collection of functions that can be performed on files.
- File system also maintains a set of attributes associated with the file
- Typical operations include: Create, Delete, Open, Close, Read, Write
- Operating system can have multiple file systems
- File systems
 - Can be used on numerous different types of storage devices that uses different kinds of media
 - There are also many different kinds of file systems.
 - Each one has different structure and logic, properties of speed, flexibility, security, size and more.

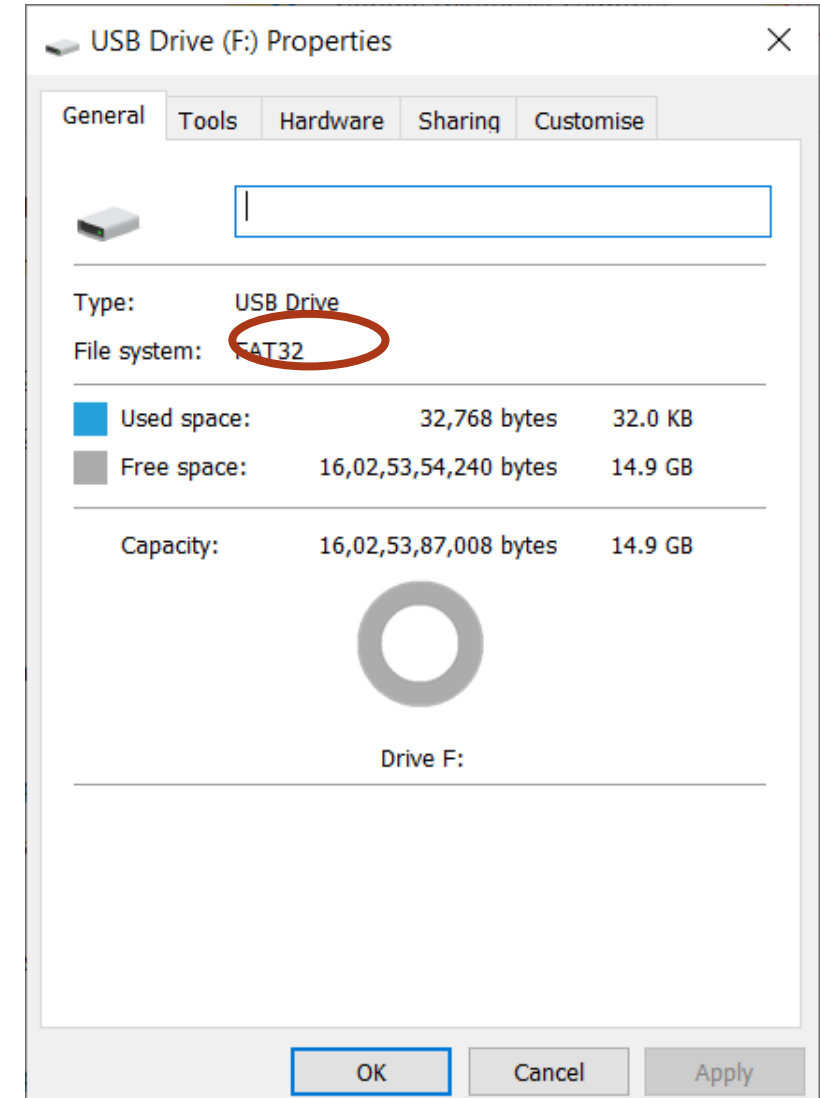
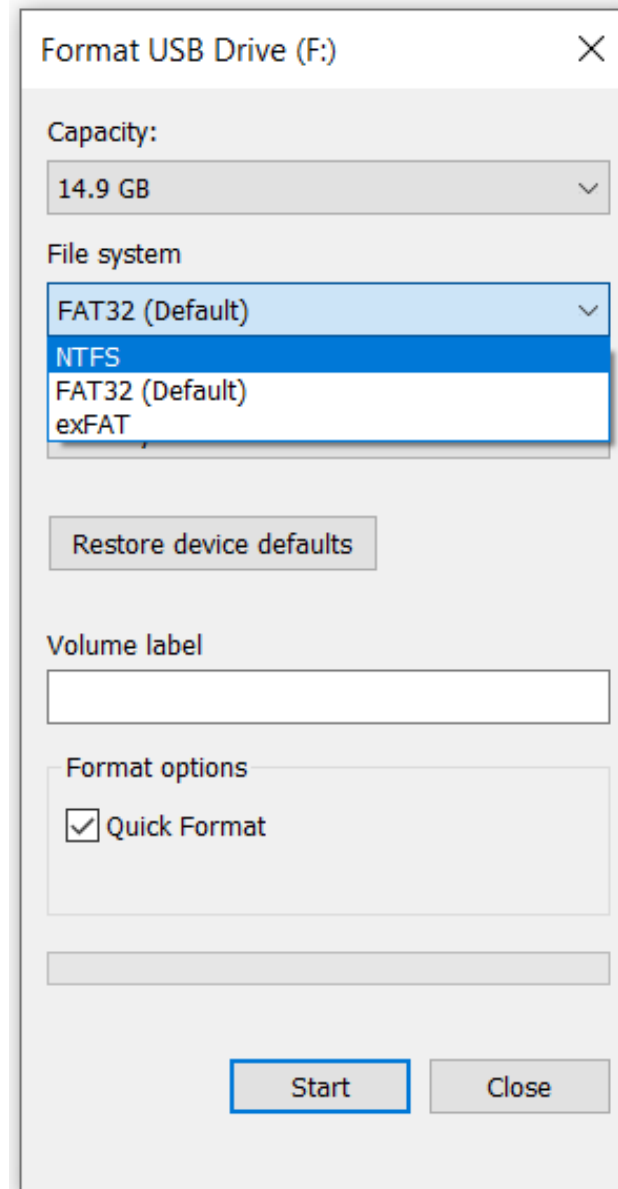
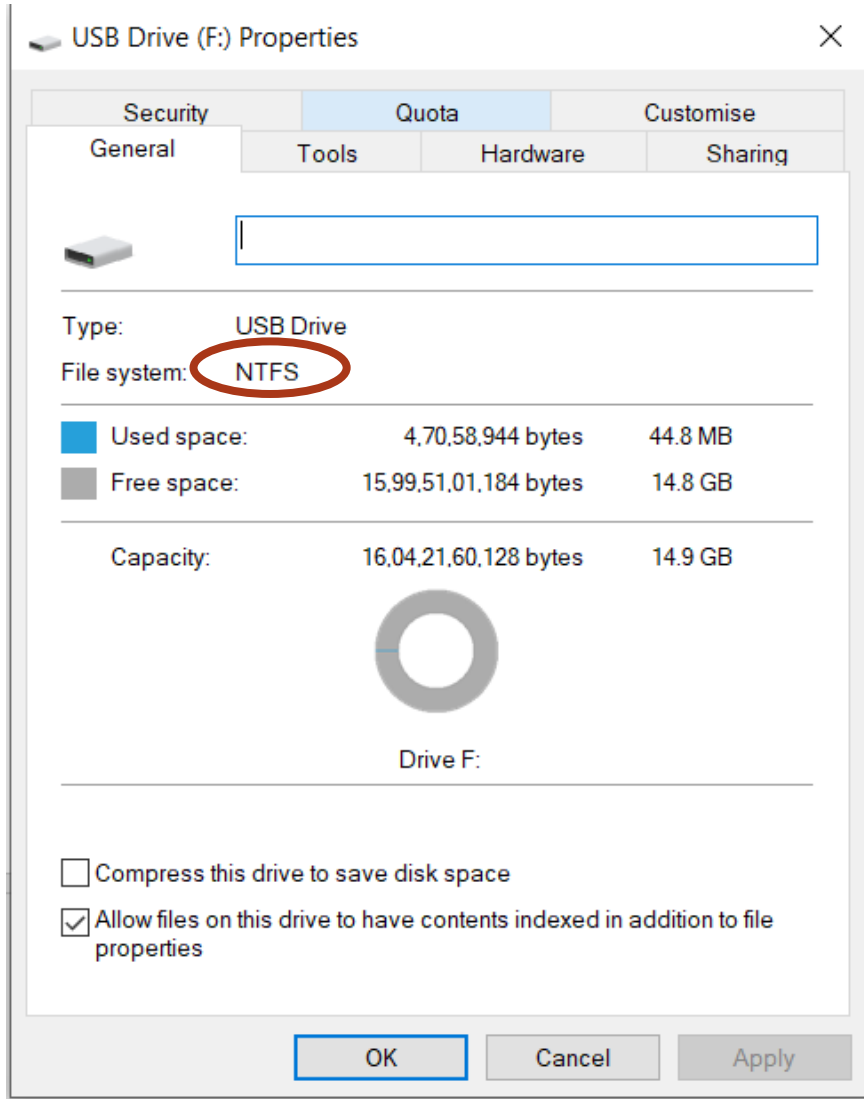
File System – design principles

- **Separation of Data and Metadata:**
 - **Metadata:** Information *about* the file (name, size, permissions, location on disk).
 - **Data:** The actual contents of the file.
- **Block Mapping:** File systems manage the mapping between logical files and the physical blocks on disk.
- **Attributes:** File systems maintain various attributes associated with each file.

Why is this important?:

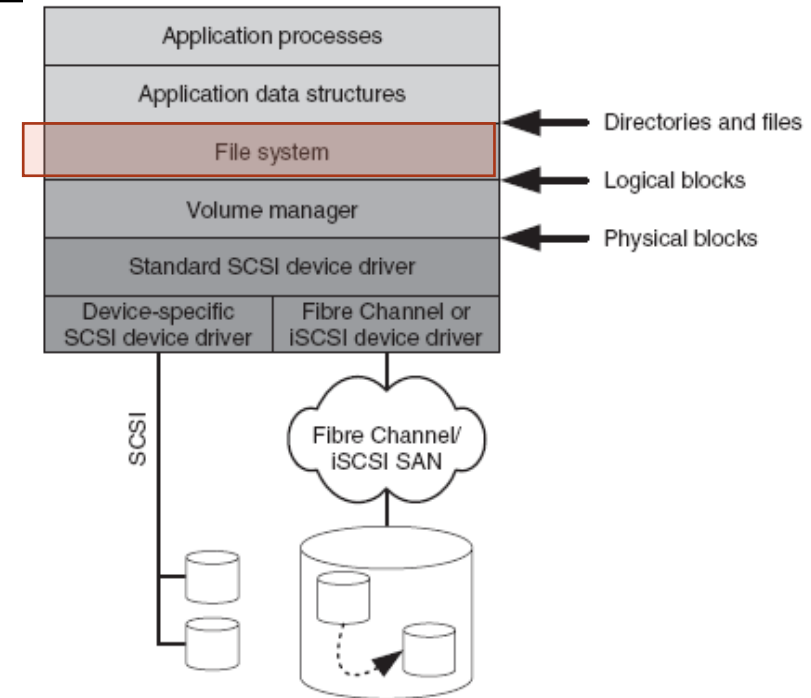
- Different file systems (e.g., NTFS, FAT32) have distinct structures, performance characteristics, and limitations (e.g., FAT32's 4GB file size limit).
- These design choices impact speed, flexibility, security, and maximum storage size

File System – design principles



File Systems : Local File Systems

- For storing data
 - An operating system will engage and schedule a storing level process
 - The filesystem manages where within the address space the data needs to be stored
 - Applications & users, use the storage capacity of the disks via directories and files.
- File systems form an intermediate layer between applications & block oriented disks (with layers of volume management software which includes RAID controllers, virtualization systems) coming into play
- File systems and Volume managers are generic, and support all kinds of applications, and are not tuned for performance of any particular type of Applications



File System – Local File Systems

Disk Management

File Action View Help

Volume	Layout	Type	File System	Status	Capacity	Free Sp...	% Free
(C:)	Simple	Basic	NTFS (BitLo...	Healthy (Boot, Page File, Cr...	476.32 GB	401.34 GB	84 %
(F:)	Simple	Basic	FAT32	Healthy (Primary Partition)	14.92 GB	14.88 GB	100 %
(Disk 0 partition 1)	Simple	Basic		Healthy (EFI System Partition)	200 MB	200 MB	100 %
(Disk 0 partition 5)	Simple	Basic		Healthy (Recovery Partition)	990 MB	990 MB	100 %
(Disk 0 partition 6)	Simple	Basic		Healthy (Recovery Partition)	1.38 GB	1.38 GB	100 %
(Disk 1 partition 1)	Simple	Basic		Healthy (EFI System Partition)	100 MB	100 MB	100 %
(Disk 1 partition 4)	Simple	Basic		Healthy (Recovery Partition)	516 MB	516 MB	100 %
New Volume (D:)	Simple	Basic	NTFS (BitLo...	Healthy (Basic Data Partition)	488.28 GB	488.16 GB	100 %
New Volume (...)	Simple	Basic	NTFS (BitLo...	Healthy (Basic Data Partition)	440.56 GB	417.37 GB	95 %

Disk 0
Basic
931.39 GB
Online

200 MB
Healthy (EFI System Partition)

New Volume (D:)
488.28 GB NTFS (BitLocker Encrypted)
Healthy (Basic Data Partition)

New Volume (E:)
440.56 GB NTFS (BitLocker Encrypted)
Healthy (Basic Data Partition)

990 MB
Healthy (Recovery Partition)

1.38 GB
Healthy (Recovery Partition)

Disk 1
Basic
476.92 GB
Online

100 MB
Healthy (EFI System Partition)

(C:)
476.32 GB NTFS (BitLocker Encrypted)
Healthy (Boot, Page File, Crash Dump, Basic Data Partition)

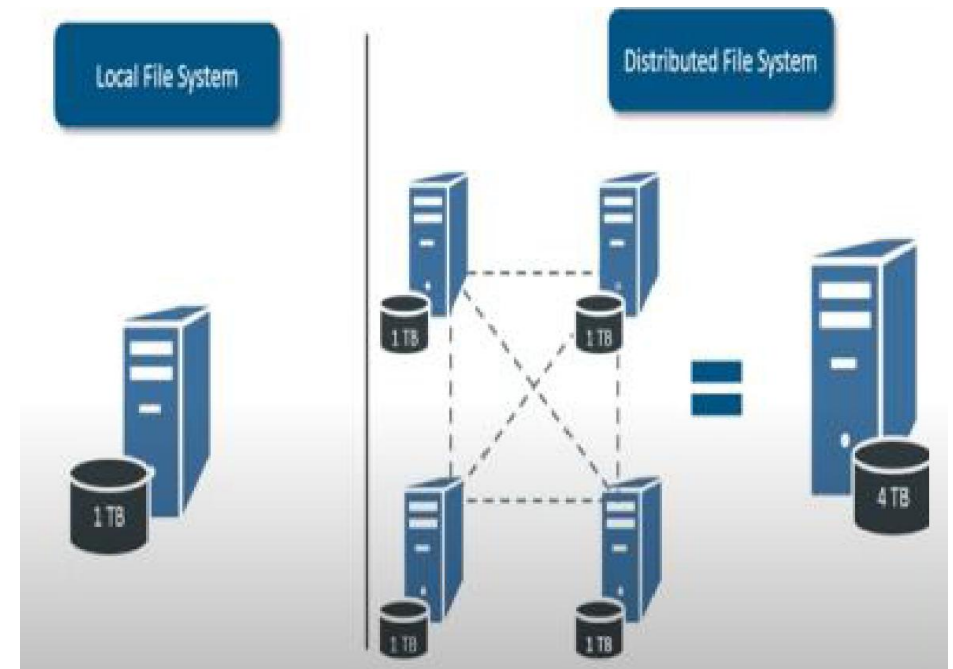
Disk 2
Removable
14.94 GB
Online

(F:)
14.94 GB FAT32
Healthy (Primary Partition)

- Open
- Explore
- Mark Partition as Active
- Change Drive Letter and Paths...
- Format...
- Extend Volume...
- Shrink Volume...
- Add Mirror...
- Delete Volume...
- Properties
- Help

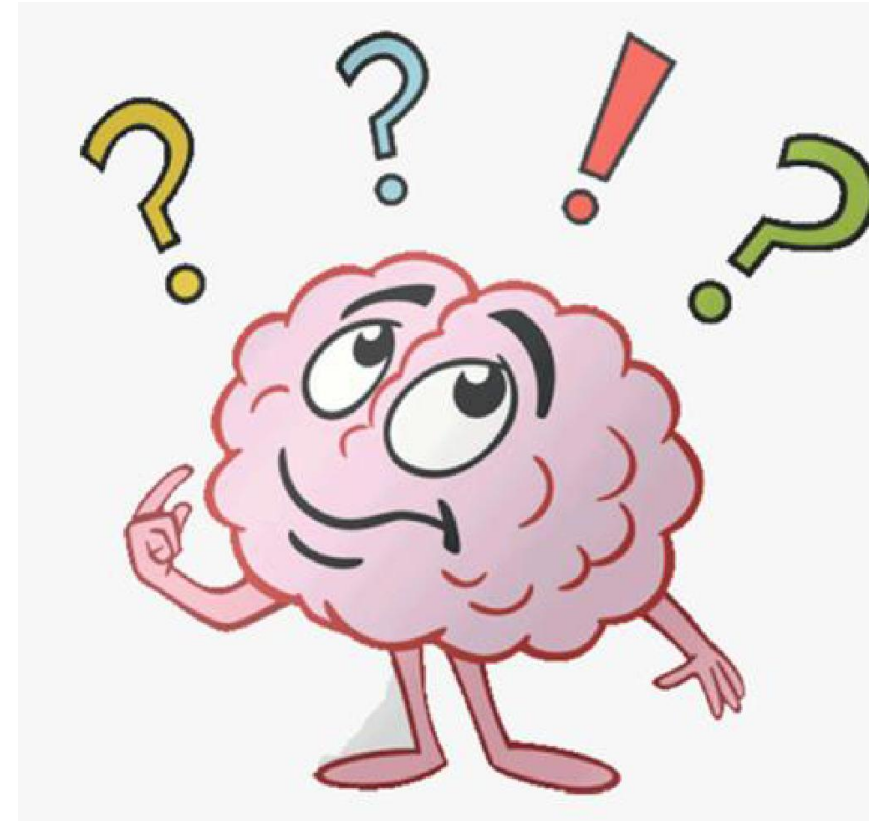
Distributed File System

- Consider the case when data is so large that it cannot fit on a single disk
 - DFS manages files and folders across multiple computers
- DFS can organize and display files as if they are stored on one computer
 - It serves the same purpose as a traditional file system
- Designed to provide file storage and controlled access to files over local and wide area networks



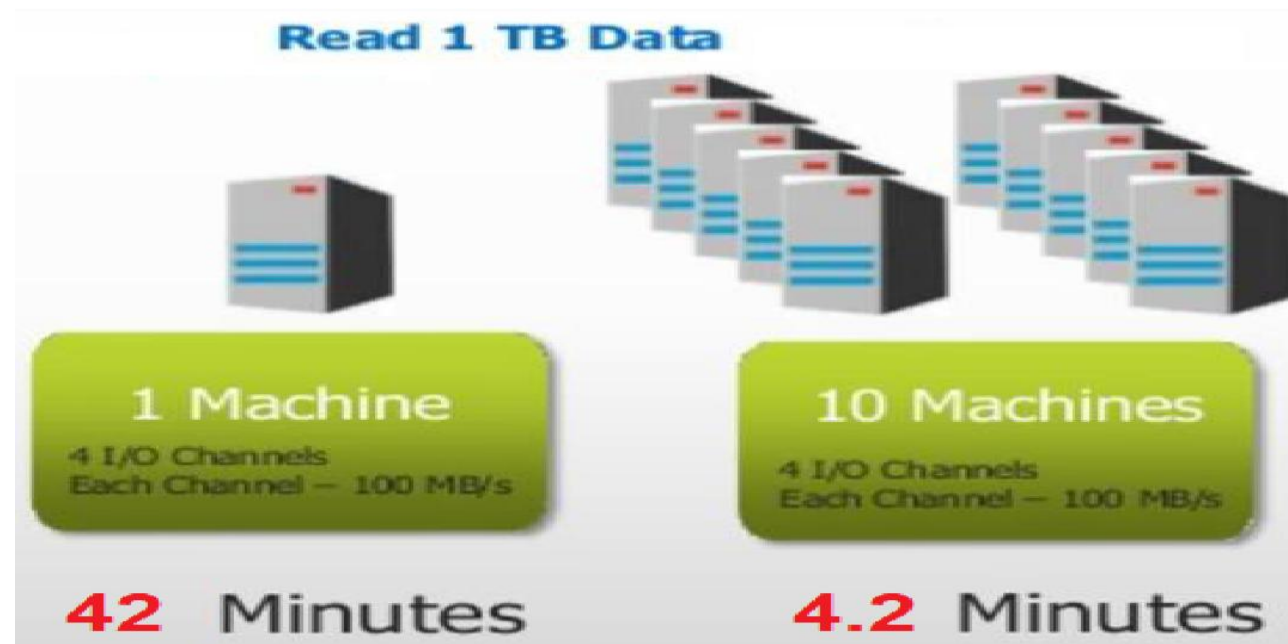
Exercise

- Consider that you have 1 TB of data?
- Compare the time taken to read data in both the cases below
 - Single machine (4 I/O channels each channel 100MB/s)
 - 10 machines (each having 4 I/O channels each channel 100MB/s)



Exercise

- Consider that you have 1 TB of data?
- Compare the time taken to read data in both the cases below
 - Single machine (4 I/O channels each channel 100MB/s)
 - 10 machines (each having 4 I/O channels each channel 100MB/s)



What about existing FS?

- **NFS – Network File System**
 - Small block sizes 8KB-16KB ⇒ large meta-data
 - Designed for large number of small files
- **Handling scaling**
 - Extending size of filesystem requires growing the volume
 - Can mainly handle scaling up.
- **Scaling out – the need**
 - A single machine cannot handle the load. Need a cluster
 - Originally not by design.
 - Was added later on need (Clustered Shared Volume in NTFS).
 - Need to support distributed operations

HDFS is based on the opensource version of GFS

What about existing FS?

HDFS – Inspired by GFS

GFS – Google File System (2003)

Distributed File system on a cluster of machines

Developed by Doug Cutting and Mike Cafarella

Origin - Apache Nutch

- Goal : web search engine on 1 Billion Pages

Open source



Google Distributed File System:

<https://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf>

HDFS – Hadoop Distributed File System

“HDFS is a filesystem designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.”

Very large

- Files can be MB/GB/TB in size
- Hadoop clusters that are PB are currently operational

Read Mostly data

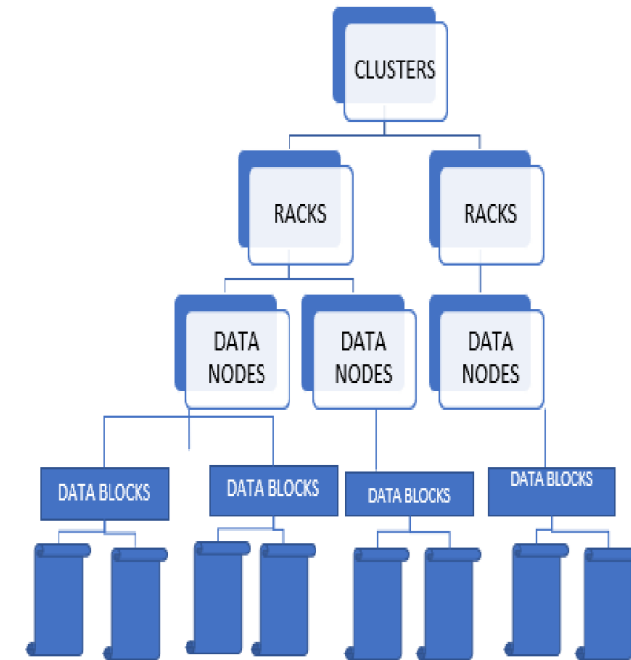
- most efficient data processing pattern is a write-once, read-many-times pattern.
- Each analysis will involve a large proportion of the dataset
- time to read the whole dataset is more important than the latency in reading the first record.

Commodity hardware

- Hadoop doesn't require expensive, highly reliable hardware
- Designed to run on clusters of commodity hardware

Hadoop Distributed File System

To begin with Any data file would be organized as set of data blocks, which are stored on a data node (any server in the cluster), these data nodes can be housed in any of the racks in the data center. Distributed cluster can be configured with servers across racks within a single physical space or can also be configured with servers distributed across geographical space.



Exercise

- If you want to store a file on disk, it constitutes

- Data

Data: much larger in size. Occupies multiple blocks

- Metadata

Metadata: smaller compared to data. Only information on filenames and blocks it occupies.

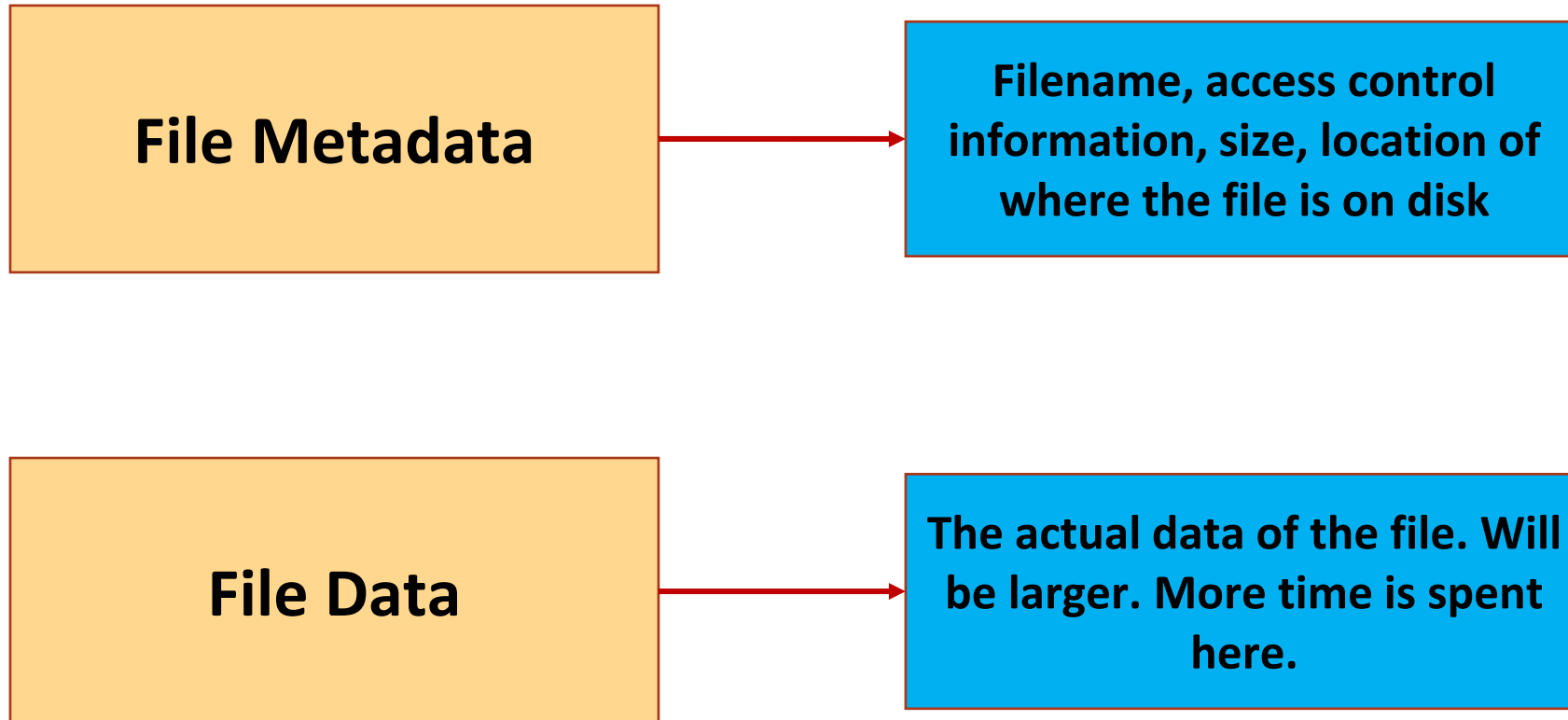
- What are their access patterns

- How often do you think each one will be accessed during a normal file read

Since data is much larger.
Most time spent in fetching data

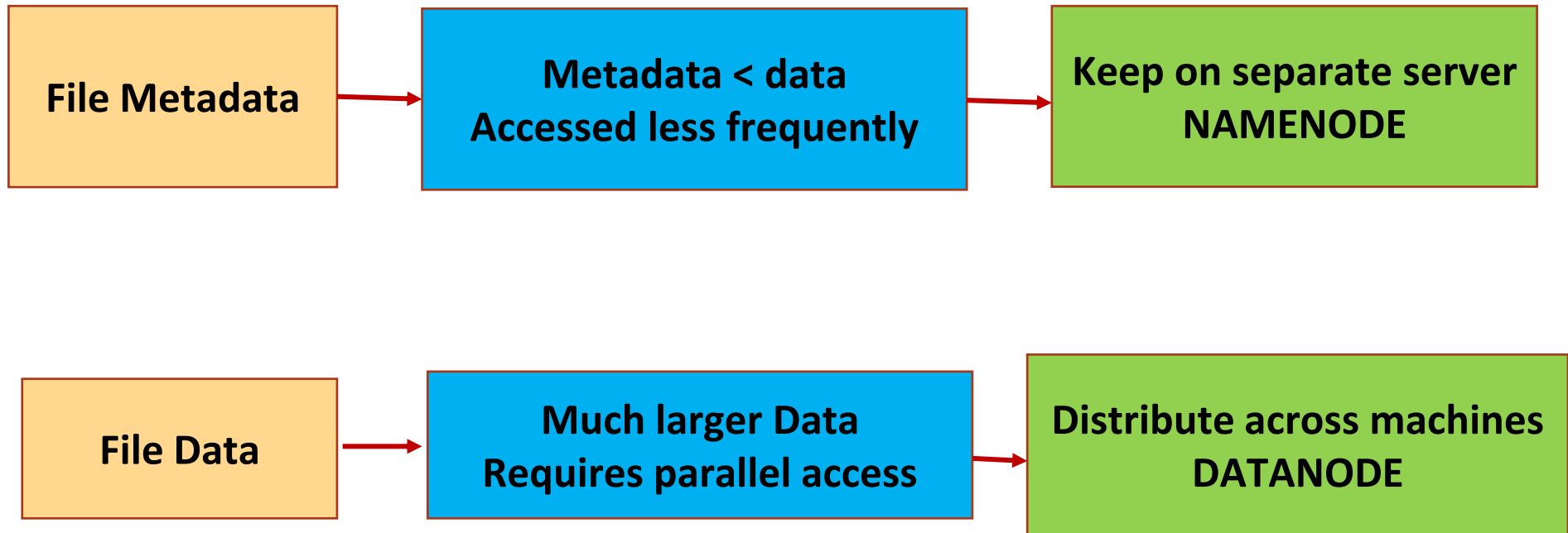
- How large are they (comparatively)? Why is this important?

HDFS Motivation

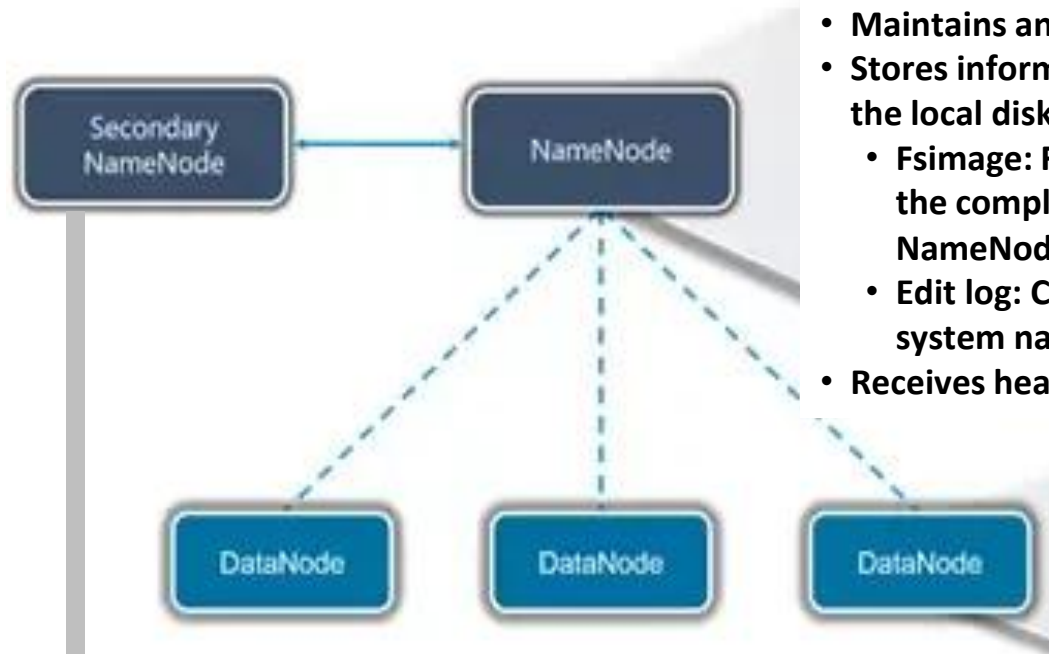


HDFS Motivation

Solution



HDFS - Architecture



Secondary NameNode

- Helper Node to Primary NameNode
- Housekeeping backup **(not hot standby)**
- Supports primary NameNode by downloading and merging the Fsimage file and edit logs file, updates the Fsimage and refreshes the edit logs
- It then sends this updated image to NameNode and also to Stand-by NameNode to enable a faster failover

NameNode

- Master Daemon
- Maintains and Manages DataNodes
- Stores information about blocks locations, permissions, etc. on the local disk in two files:
 - Fsimage: Fsimage stands for File System image. It contains the complete namespace of the Hadoop file system since the NameNode creation.
 - Edit log: Contains all recent changes performed to the file system namespace to the most recent Fsimage
- Receives heartbeat and block report from all the DataNodes

DataNodes

DataNodes are the slave nodes and the workhorses of HDFS

These are inexpensive commodity hardware storing blocks of a file

These are responsible for serving the client read/write requests

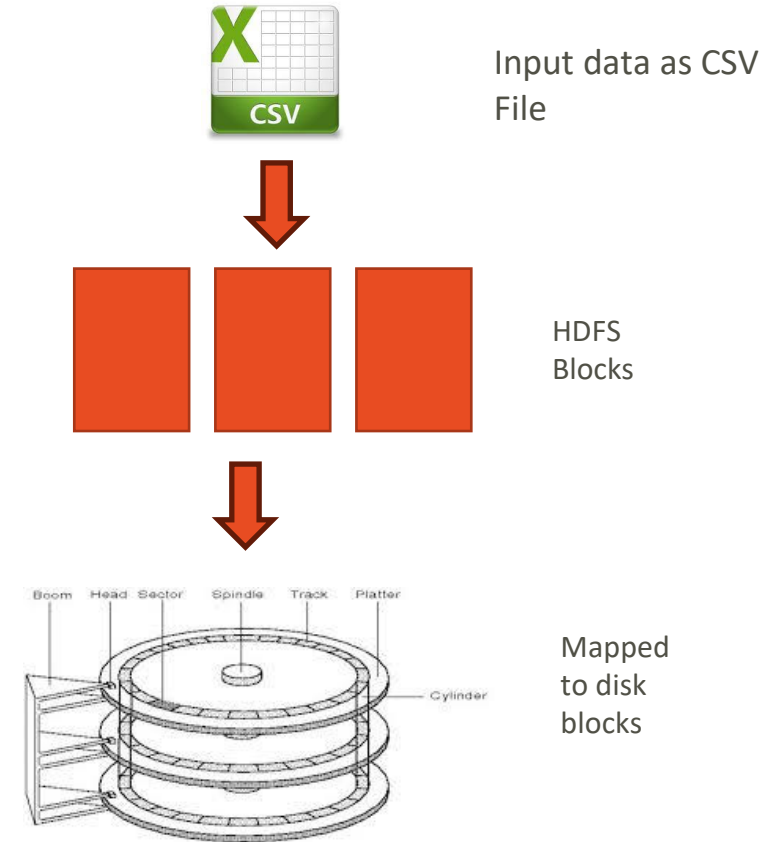
Based on the instruction from the NameNode, DataNodes performs block creation, replication, and deletion.

DataNodes send a heartbeat to NameNode to report the health of HDFS.

DataNodes also sends block reports to NameNode to report the list of blocks it contains.

HDFS Blocks: What are they?

- Hard Disk Blocks:
 - Minimum data that can be read written.
 - Typically 512 bytes.
- HDFS blocks
 - Much larger unit
 - 128MB (in v2 or Hadoop version 2X onwards)
- Files in HDFS are broken into block-sized chunks, which are stored as independent units.
- A file in HDFS that is smaller than a single block does not occupy a full block's worth of underlying storage.
 - Use as many disk blocks as necessary to store any file size.



Sample Hadoop Cluster Overview (Setup at CCBD)

- **Cluster Composition:**
 - **Total Nodes:** 6 (1 Master, 5 Workers)
- **Master Node Configuration:**
 - **RAM:** 64 GB
 - **Processors:** 32 cores
- **Worker Nodes Configuration:**
 - **RAM:** 32 GB - 64 GB per node
 - **Processors:** 32 cores per node
 - **Total Storage:** 6.23 TB
- **Software & Security:**
 - **OS Version:** Ubuntu 22.04 LTS
 - **Hadoop Version:** 3.4
- **Cluster Configuration:**
 - **Replication Factor:** 2

HDFS - Report

Summary

Security is off.

Safemode is off.

824 files and directories, 889 blocks (889 replicated blocks, 0 erasure coded block groups) = 1,713 total filesystem object(s).

Heap Memory used 987.69 MB of 1.77 GB Heap Memory. Max Heap Memory is 15.7 GB.

Non Heap Memory used 102.14 MB of 106.49 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	6.23 TB
Configured Remote Capacity:	0 B
DFS Used:	34.31 GB (0.54%)
Non DFS Used:	120.88 GB
DFS Remaining:	5.76 TB (92.48%)
Block Pool Used:	34.31 GB (0.54%)
DataNodes usages% (Min/Median/Max/stdDev):	0.19% / 0.58% / 0.86% / 0.25%
Live Nodes	5 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Tue Aug 06 12:03:09 +0530 2024
Last Checkpoint Time	Tue Aug 06 12:03:13 +0530 2024
Last HA Transition Time	Never
Enabled Erasure Coding Policies	RS-6-3-1024k

HDFS - Report

NameNode Journal Status

Current transaction ID: 144908

Journal Manager	State
FileJournalManager(root=/home/user/dfsdata/namenode)	EditLogFileOutputStream(/home/user/dfsdata/namenode/current/edits_inprogress_0000000000000144908)

NameNode Storage

Storage Directory	Type	State
/home/user/dfsdata/namenode	IMAGE_AND_EDITS	Active

DFS Storage Types

Storage Type	Configured Capacity	Capacity Used	Capacity Remaining	Block Pool Used	Nodes In Service
DISK	6.23 TB	34.31 GB (0.54%)	5.76 TB (92.48%)	34.31 GB	5

HDFS - Datanodes

In operation

DataNode State

All

Show

25

entries

Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✓ /default-rack/worker-5:9866 (10.10.3.116:9866)	http://worker-5:9864	1s	298m	3.51 GB	22.38 GB	1.79 TB	425	3.51 GB (0.19%)	0%	3.4.0
✓ /default-rack/worker-2:9866 (10.10.3.113:9866)	http://worker-2:9864	2s	291m	7.87 GB	25.75 GB	915.32 GB	328	7.87 GB (0.86%)	0%	3.4.0
✓ /default-rack/worker-1:9866 (10.10.3.112:9866)	http://worker-1:9864	0s	24m	7.88 GB	25.16 GB	915.32 GB	320	7.88 GB (0.86%)	0%	3.4.0
✓ /default-rack/worker-4:9866 (10.10.3.115:9866)	http://worker-4:9864	1s	306m	9.88 GB	22.44 GB	1.79 TB	440	9.88 GB (0.54%)	0%	3.4.0
✓ /default-rack/worker-3:9866 (10.10.3.114:9866)	http://worker-3:9864	1s	145m	5.17 GB	25.15 GB	883.92 GB	265	5.17 GB (0.58%)	0%	3.4.0

Showing 1 to 5 of 5 entries

Previous

1

Next

HDFS - DFS Admin Report

```
user@master:~/testHadoop$ hdfs dfsadmin -report
Configured Capacity: 6849375895552 (6.23 TB)
Present Capacity: 6371172380672 (5.79 TB)
DFS Remaining: 6334435377152 (5.76 TB)
DFS Used: 36737003520 (34.21 GB)
DFS Used%: 0.58%
Replicated Blocks:
    Under replicated blocks: 0
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
-----
Live datanodes (5):

Name: 10.10.3.112:9866 (worker-1)
Hostname: worker-1
Decommission Status : Normal
Configured Capacity: 982819848192 (915.32 GB)
DFS Used: 8463704064 (7.88 GB)
Non DFS Used: 27010301952 (25.16 GB)
DFS Remaining: 897345806336 (835.72 GB)
DFS Used%: 0.86%
DFS Remaining%: 91.30%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Aug 13 11:47:40 IST 2024
Last Block Report: Tue Aug 13 11:17:51 IST 2024
Num of Blocks: 320
```

```
Name: 10.10.3.113:9866 (worker-2)
Hostname: worker-2
Decommission Status : Normal
Configured Capacity: 982818799616 (915.32 GB)
DFS Used: 8448913408 (7.87 GB)
Non DFS Used: 27645718528 (25.75 GB)
DFS Remaining: 896724185088 (835.14 GB)
DFS Used%: 0.86%
DFS Remaining%: 91.24%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Aug 13 11:47:40 IST 2024
Last Block Report: Tue Aug 13 06:50:43 IST 2024
Num of Blocks: 328
```

```
Name: 10.10.3.114:9866 (worker-3)
Hostname: worker-3
Decommission Status : Normal
Configured Capacity: 949102149632 (883.92 GB)
DFS Used: 5551570944 (5.17 GB)
Non DFS Used: 27009441792 (25.15 GB)
DFS Remaining: 868253949952 (808.62 GB)
DFS Used%: 0.58%
DFS Remaining%: 91.48%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Aug 13 11:47:42 IST 2024
Last Block Report: Tue Aug 13 09:17:07 IST 2024
Num of Blocks: 265
```

```
Name: 10.10.3.115:9866 (worker-4)
Hostname: worker-4
Decommission Status : Normal
Configured Capacity: 1967317549056 (1.79 TB)
DFS Used: 10505269248 (9.78 GB)
Non DFS Used: 24197222400 (22.54 GB)
DFS Remaining: 1832605237248 (1.67 TB)
DFS Used%: 0.53%
DFS Remaining%: 93.15%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Aug 13 11:47:42 IST 2024
Last Block Report: Tue Aug 13 06:35:44 IST 2024
Num of Blocks: 440
```

```
Name: 10.10.3.116:9866 (worker-5)
Hostname: worker-5
Decommission Status : Normal
Configured Capacity: 1967317549056 (1.79 TB)
DFS Used: 3767545856 (3.51 GB)
Non DFS Used: 24033984512 (22.38 GB)
DFS Remaining: 1839506198528 (1.67 TB)
DFS Used%: 0.19%
DFS Remaining%: 93.50%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Tue Aug 13 11:47:41 IST 2024
Last Block Report: Tue Aug 13 06:44:34 IST 2024
Num of Blocks: 425
```

HDFS Blocks: What



File

```
shashank@intel2:~$ hdfs dfs -ls test
22/08/03 16:24:30 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 10 items
-rw-r--r--  3 shashank supergroup      0 2021-01-05 19:06 test/_SUCCESS
-rw-r--r--  3 shashank supergroup    803 2021-01-05 19:06 test/_master.heap
-rw-r--r--  3 shashank supergroup 134217697 2021-01-05 19:05 test/part-00000_data_00001
-rw-r--r--  3 shashank supergroup 134217698 2021-01-05 19:05 test/part-00001_data_00001
-rw-r--r--  3 shashank supergroup 134217670 2021-01-05 19:05 test/part-00002_data_00001
-rw-r--r--  3 shashank supergroup 134217699 2021-01-05 19:05 test/part-00003_data_00001
-rw-r--r--  3 shashank supergroup 134217694 2021-01-05 19:06 test/part-00004_data_00001
-rw-r--r--  3 shashank supergroup 134217727 2021-01-05 19:06 test/part-00005_data_00001
-rw-r--r--  3 shashank supergroup 134217719 2021-01-05 19:06 test/part-00006_data_00001
-rw-r--r--  3 shashank supergroup 134217657 2021-01-05 19:06 test/part-00007_data_00001
```

Test is a folder on HDFS.

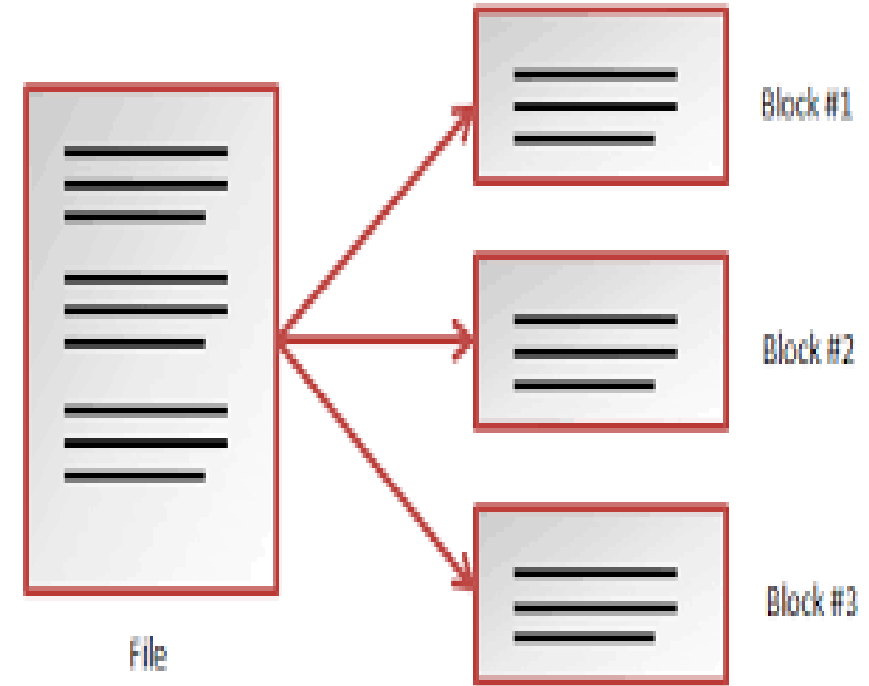
`hdfs dfs -ls`

Above command lists the contents of HDFS.

Above screenshot displays the contents of test folder which has 10 files – each file size is indicated in bytes. (Which is approximately 128MB – HDFS block size)

HDFS Blocks: Why

- Benefits of block abstraction.
 - A file can be larger than any single disk in the network.
 - Files can be distributed across disks
 - Simplifies the storage subsystem
 - Blocks fit well with replication for providing fault tolerance and availability.
- % **hadoop fsck -files -blocks**
 - Above command will list the blocks that make up each file in the filesystem.
 - Try this with any sample file – with size larger than 128MB so that you can observe multiple blocks.



HDFS **fsck** is used to check the health of the file system, to find missing files, over replicated, under replicated and corrupted blocks.

```
user@master:~/testHadoop$ hdfs fsck /wordCount
Connecting to namenode via http://master:9870/fsck?ugi=user&path=%2FwordCount
FSCK started by user (auth:SIMPLE) from /10.10.3.111 for path /wordCount at Tue Aug 13 11:45:11 IST 2024

Status: HEALTHY
Number of data-nodes: 5
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 3414442024 B
Total files: 1
Total blocks (validated): 26 (avg. block size 131324693 B)
Minimally replicated blocks: 26 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 2
Average block replication: 2.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0
FSCK ended at Tue Aug 13 11:45:11 IST 2024 in 3 milliseconds

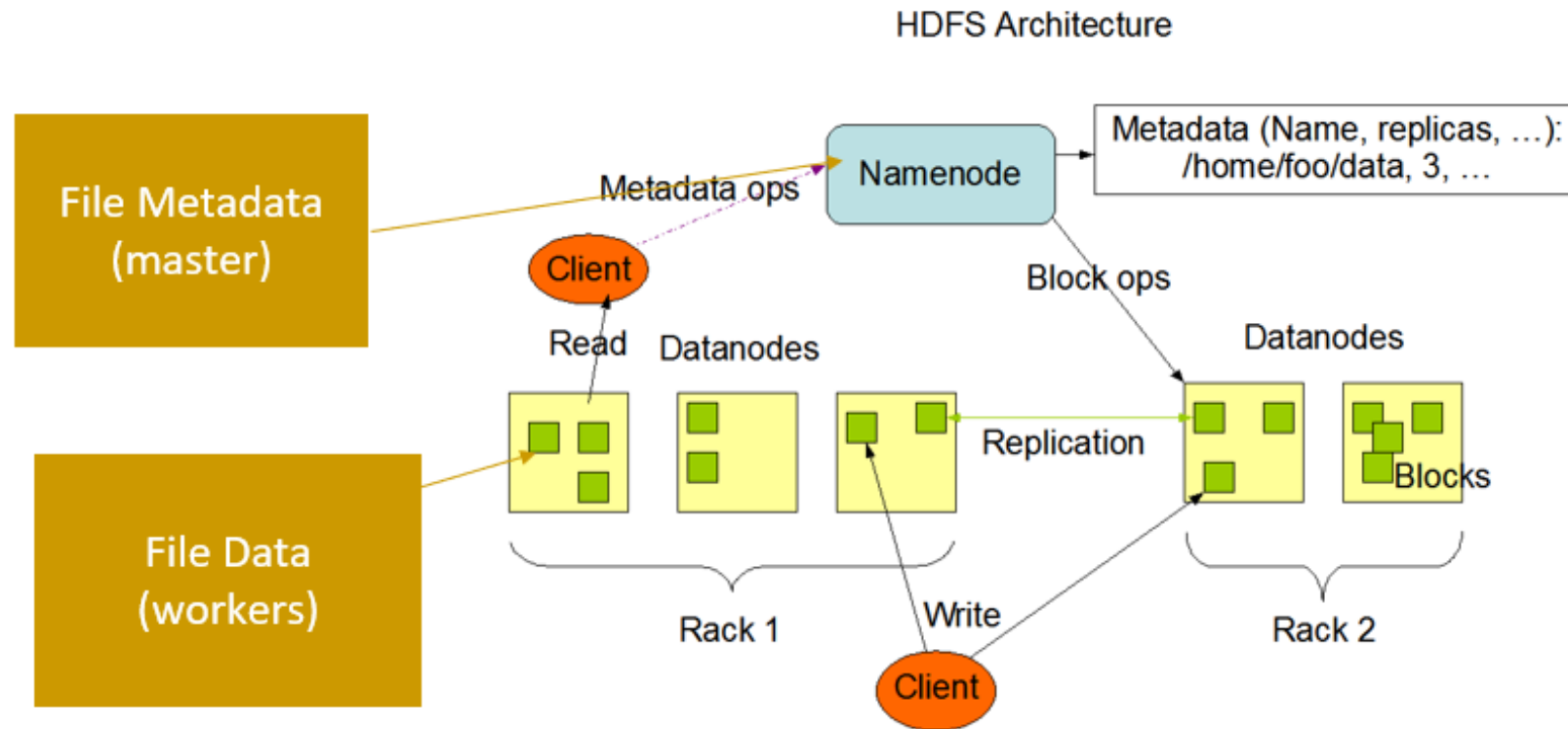
The filesystem under path '/wordCount' is HEALTHY
user@master:~/testHadoop$ |
```

Some Basic Operations in HDFS

HDFS provides a command-line interface that mimics standard Unix file system commands, making it familiar to many users.

- `hdfs dfs -ls /`: List the files and directories in the root directory.
- `hdfs dfs -mkdir /mydir`: Create a new directory named `mydir`.
- `hdfs dfs -put localfile /mydir/`: Copy `localfile` from the local file system to the `/mydir/` directory in HDFS.
- `hdfs dfs -get /mydir/remotefile .`: Copy `remotefile` from HDFS to the current local directory.
- `hdfs dfs -rm /mydir/remotefile`: Delete a file from HDFS.
- `hdfs dfsadmin -report` : to display the cluster status.

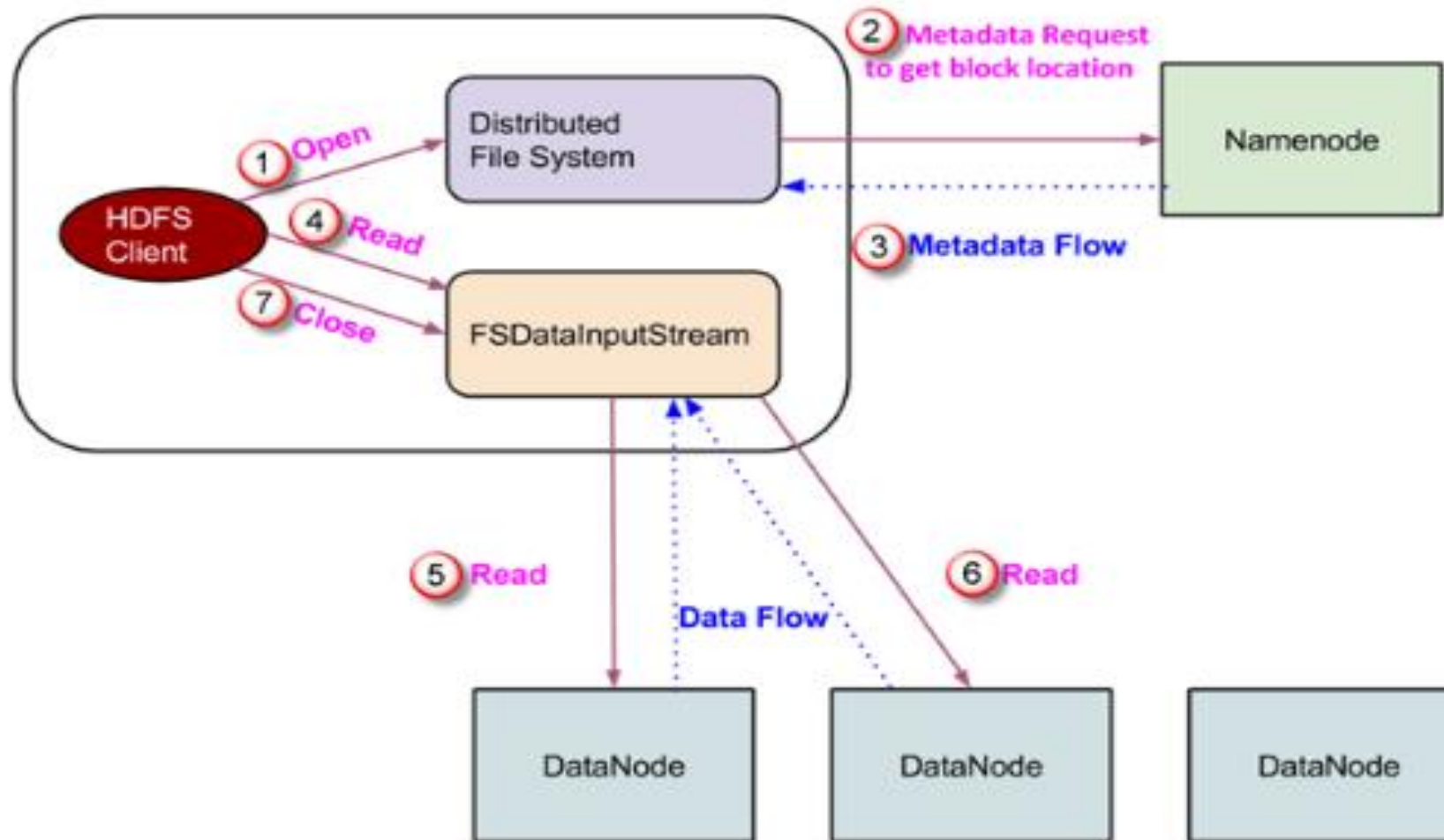
Detailed HDFS Architecture and internal working



what secondary namenode and stand-by namenode would do?

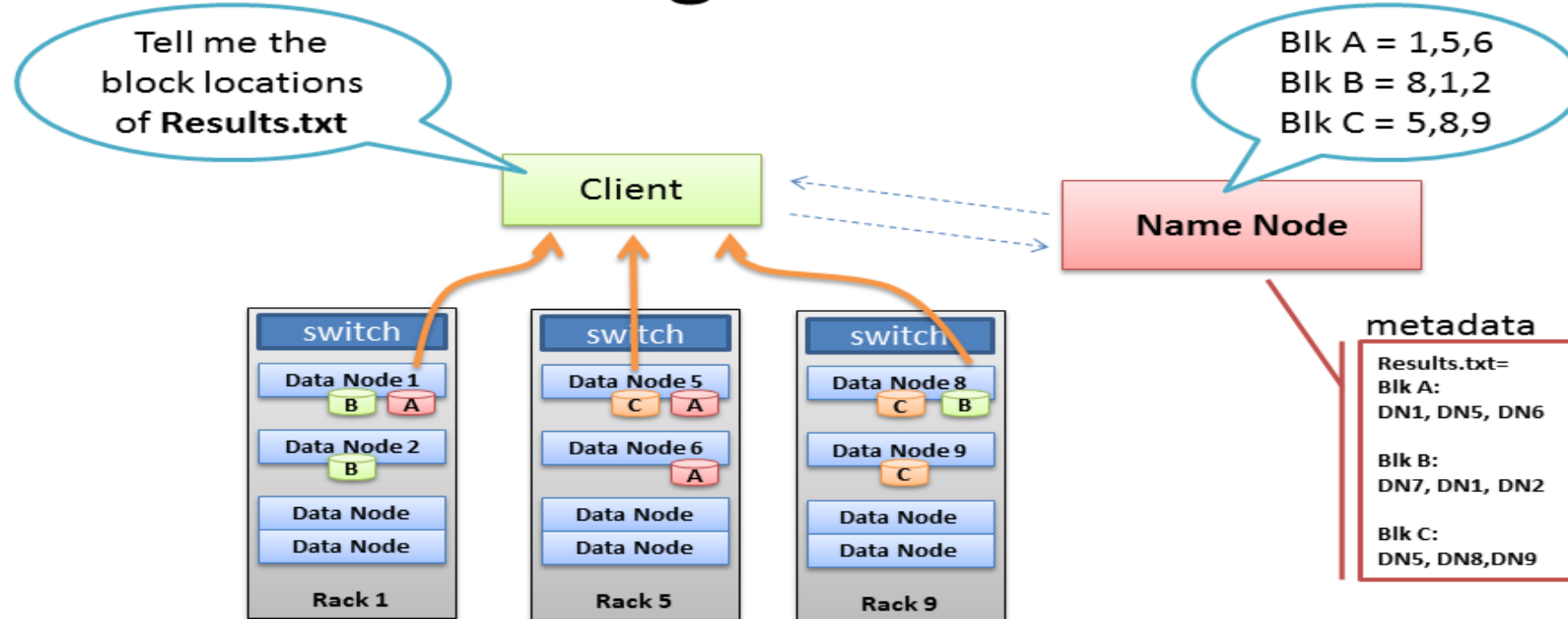


Reading a file from HDFS



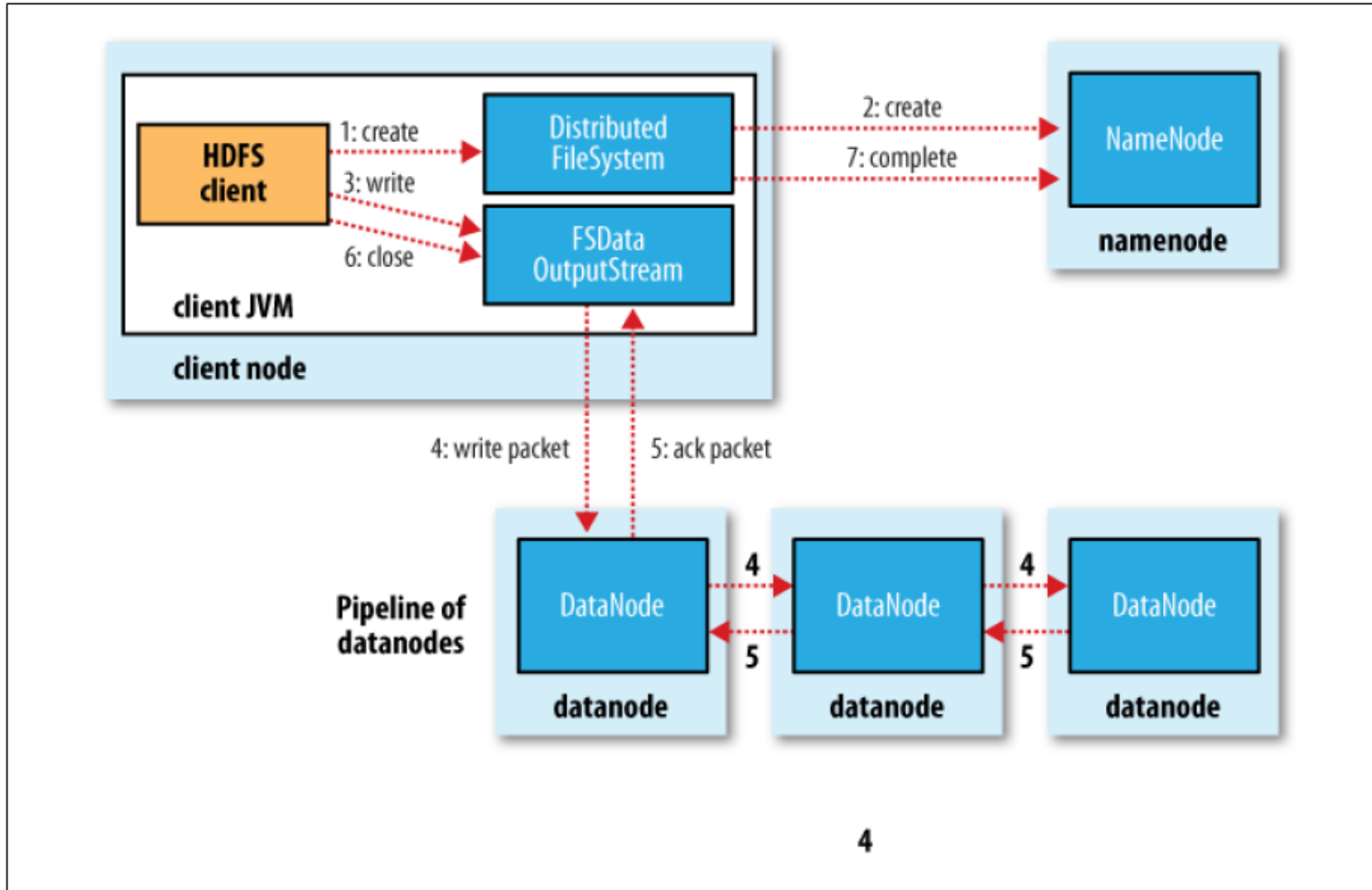
Reading a file from HDFS

Client reading files from HDFS



- Client receives Data Node list for each block
- Client picks first Data Node for each block
- Client reads blocks sequentially

Writing a file to HDFS

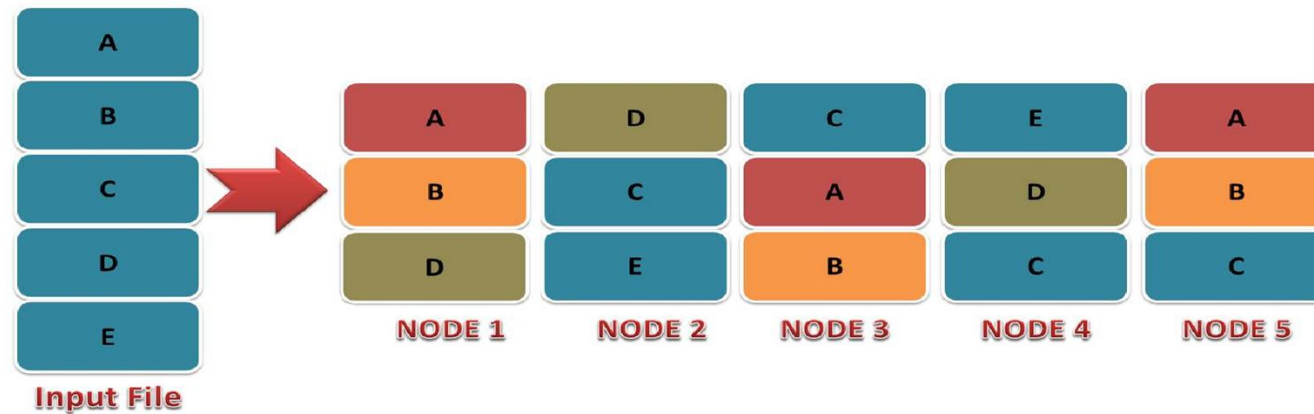


HDFS Fault Tolerance

- Fault tolerance in Hadoop HDFS refers to the working strength of a system in unfavorable conditions and how that system can handle such a situation
- HDFS is highly fault tolerant and handles faults.
 - There are approaches used for handling faults of data nodes or disks holding data blocks.
 - These could be based on replication, to a **replication factor** till Hadoop 3. So whenever if any machine/disk in the cluster goes down, then data is accessible from other machines/disk in which the copy of same data was created
 - Using **Erasure coding** in Hadoop 3
 - There are also approaches to **handle faults of Name Node** and the availability of the metadata pointing to the data blocks

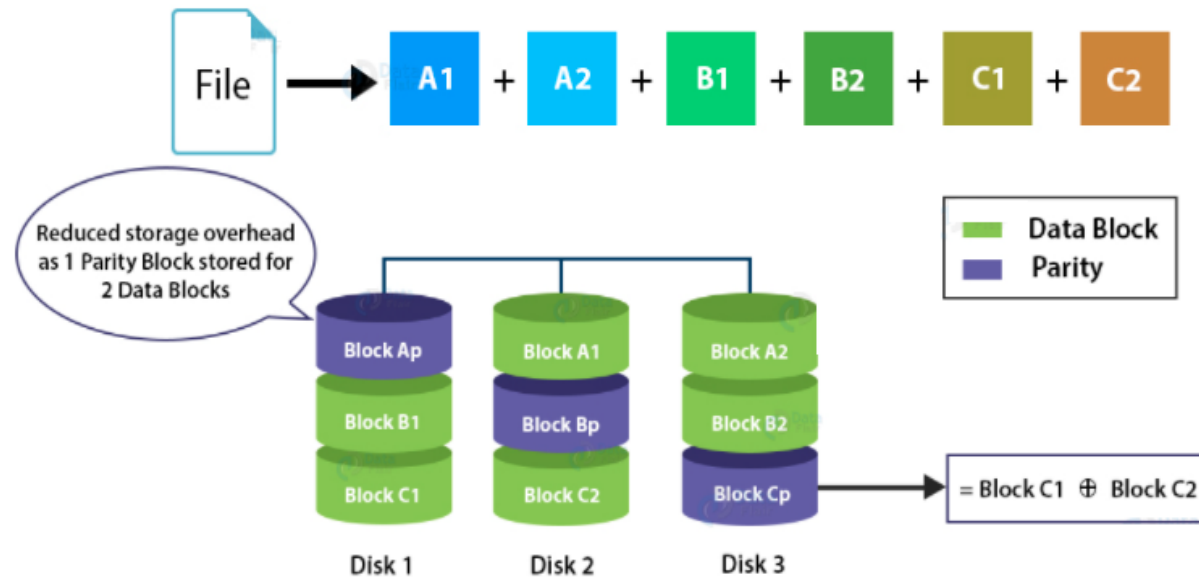
HDFS Fault Tolerance (Replication)

- Fault tolerance of data using Replicas is achieved by creating a replica of users' data based on the replication factor on different machines in the HDFS cluster.
- So if any machine in the cluster goes down, then data is accessible from other machines in which the copy of same data was created



HDFS Fault Tolerance (Erasure Coding)

- Fault tolerance of data could also be based on Erasure coding
- Erasure coding works similar to RAID by striping the file into small units of sequential blocks and storing them consecutively on various disks.
- For each strip of the original dataset, a certain number of **parity** cells are calculated using **erasure coding algorithm** called encoding and stored. If any of the machines fails, the block can be recovered from the parity cell.
- The error in any striping cell can be recovered from the calculation based on the remaining data and parity cells; the process known as decoding.



HDFS High Availability (post 2.x)

Objective

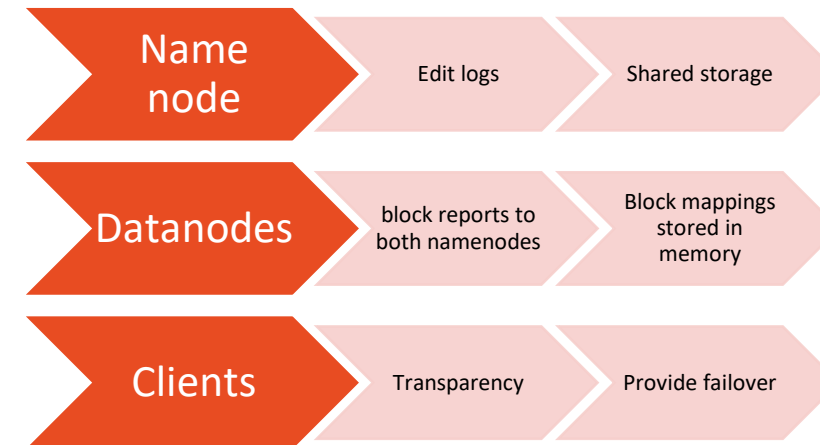
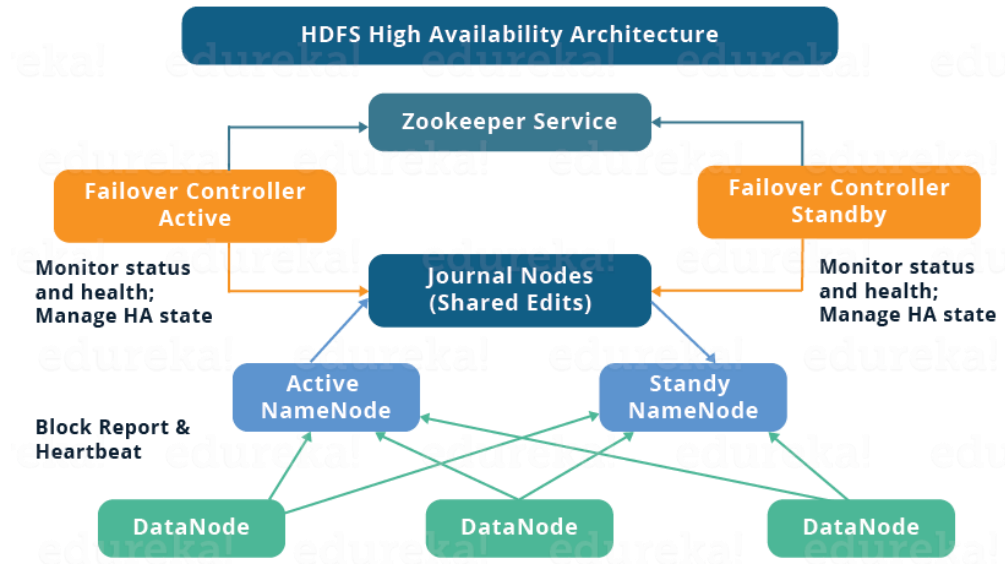
- Make the name node resilient to failure
- Should recover quickly

Configure

- Active – Standby configuration
- Edit logs
 - Store on shared storage
- Block Mappings
 - Stored in memory

On Failure

- Standby takes over using shared edit logs and in-memory block mappings

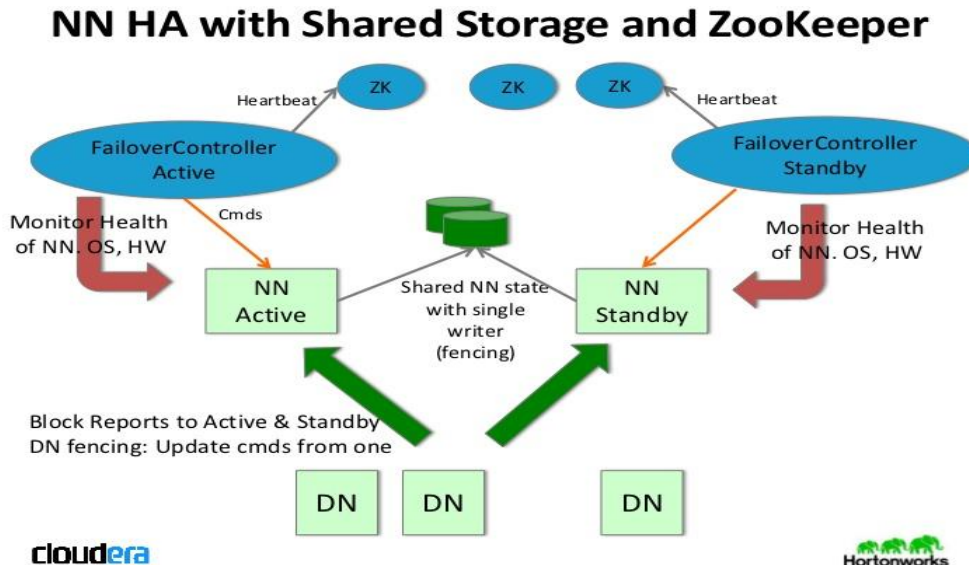


HDFS Failover and High Availability (HA)

- Failover Controller
 - Handles transition from active ➡ standby
 - Failover controllers are pluggable,
 - ZooKeeper to ensure that only one namenode is active.
- Manual Trigger for maintenance
- Ungraceful failure
 - Is it a real failure?
 - Slow network – active treated as failed, but is up and running

HDFS uses Zookeeper for

- Failure Detection
 - Namenode maintains persistent session with ZK
 - If Machine crashes
 - Session would expire
 - Notify the other NameNode of the crash
- Active Namenode Election
 - On crash, other node takes exclusive lock
 - Indicating that it is the leader



Where HDFS doesn't score

- Low latency data access.
- Large number of small files.
- Multiple writers
- Arbitrary file modifications.
- Not POSIX compliant.

(Think why???)

Where HDFS doesn't score

Low Latency Data Access:



Applications that require low-latency access to data, in the tens of milliseconds range. The high throughput offered by HDFS may be at the cost of latency.

Lots of Small Files:



namenode holds filesystem metadata in memory,
Limit to the number of files in a filesystem is governed by the **amount of memory on the namenode**.
As a rule of thumb, each file, directory, and block takes about 150 bytes.

- 1 Million files- metadata would be 300MB of memory. (150+150: Data+Metadata)
- Think of Billion files ?

Multiple writers, arbitrary file modifications:



Files in HDFS may be written to by a single writer.
Writes are always made at the end of the file.
No support for multiple writers, or for modifications at arbitrary offsets in the file.

Not Posix compliant



Can't mount and use standard FS commands
Need a separate client

Summary facts about HDFS:

HDFS was originally designed to overcome slow mechanical hard drives (HDDs).

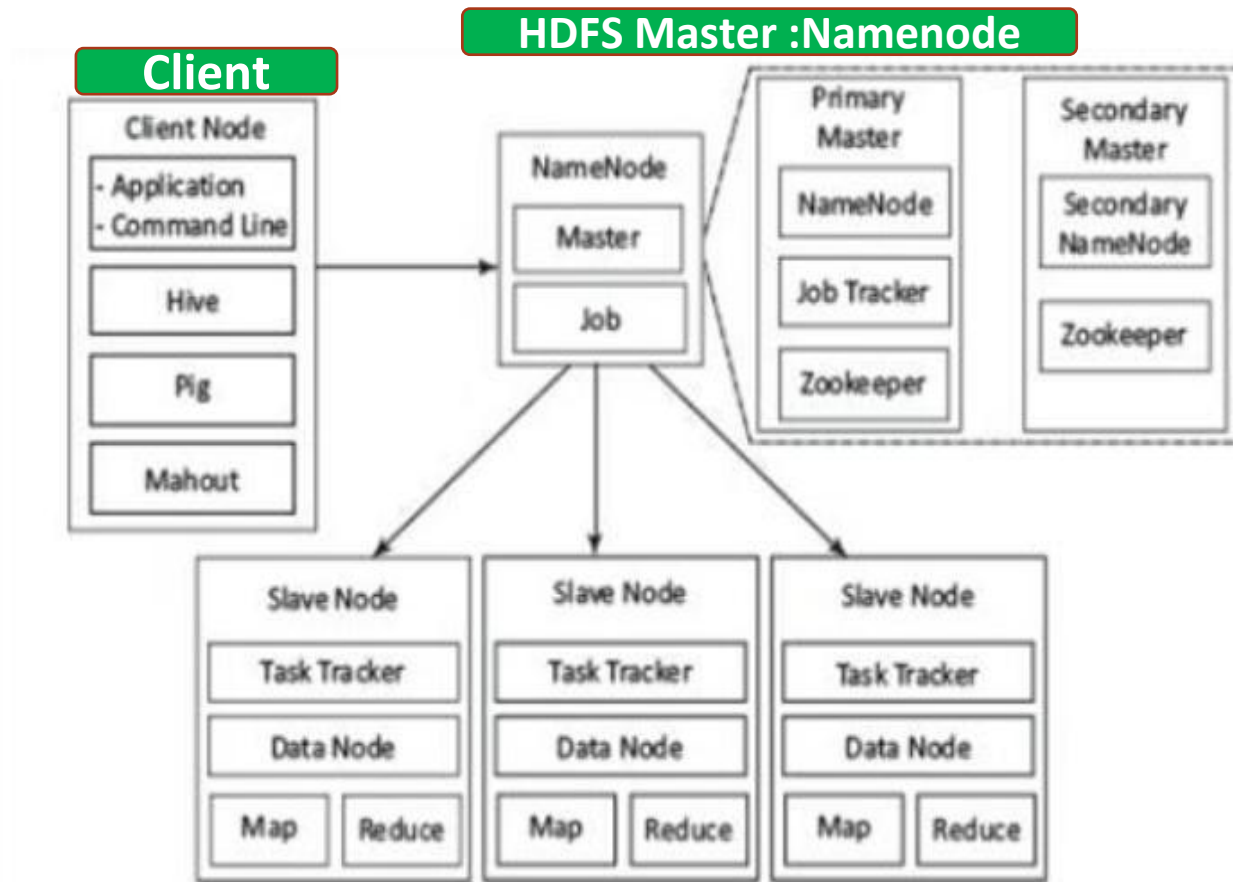
What would happen if HDDs are replaced with SSDs?

- deploying it on all-SSD nodes cluster shifts the system **bottleneck** from the physical disk to the **network and CPU**. (the network infrastructure (requiring 25GbE or 100GbE) and CPU serialization often become the new bottlenecks during parallel MapReduce or Spark jobs)

Block Size Optimization: The traditional HDFS block size of 128MB or 256MB was chosen to minimize HDD seek time overhead. With the zero-seek nature of SSDs, clusters can handle smaller block sizes or highly concurrent random reads (e.g., HBase workloads) far more efficiently.

Replication vs. Erasure Coding: Keeping 3 copies of data (3x replication) on expensive SSDs is economically inefficient. All-SSD HDFS clusters heavily utilize HDFS Erasure Coding (EC), which provides the same fault tolerance as 3x replication but reduces the storage overhead from 200% down to around 50%

HDFS – Master Slave Architecture: A quick peek into Older version



The client, master NameNode, MasterNodes and slave nodes

HDFS : Datanodes



THANK YOU

Prafullata Kiran Auradkar

Department of CSE
prafullatak@pes.edu



BIG DATA

Google File System (GFS)

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

- **Origin:** The Google File System (GFS) was developed by Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung and introduced in a 2003 paper titled "The Google File System" (Ghemawat et al., 2003).
- **Purpose:** It was designed to meet the rapidly growing data processing needs of Google's applications, providing a scalable, fault-tolerant distributed file system built on inexpensive, commodity hardware (Ghemawat et al., 2003).
- Used internally by Google to support applications like the Google search engine, Google Earth, and Gmail.

What about existing FS?

- **Design Characteristics of NFS**

Network File System (NFS) was originally designed for managing a large number of small files, typically using small block sizes (8KB–16KB). This results in relatively high metadata overhead but works well for traditional, centralized file access patterns.

- **Scalability: Scale-Up Focus**

NFS primarily supports **scaling up**, where the size of the filesystem is increased by growing the volume. However, it lacks native support for **scaling out**, which limits its performance in distributed environments.

- **Challenges in Scaling Out**

As workloads grew beyond what a single machine could handle, the need for distributed operations became clear. NFS was not designed with this in mind, and solutions like **Clustered Shared Volumes in NTFS** were introduced later to meet this demand, albeit as retrofitted extensions.

- **Modern Distributed Alternatives**

Modern systems like **HDFS (Hadoop Distributed File System)** were built specifically to scale out across clusters. HDFS is based on the open-source implementation of the **Google File System (GFS)** and is designed from the ground up for distributed, fault-tolerant storage.

What about existing FS?

- The Hadoop Distributed File System (HDFS) is a widely used open-source distributed file system that was directly inspired by Google File System (GFS).
- HDFS shares many of GFS's core design principles, such as a master/slave architecture, a large block size, and a focus on high throughput over low latency, making it a foundational component of the Apache Hadoop ecosystem.

HDFS – Inspired by GFS

GFS – Google File System (2003)

Distributed File system on a cluster of machines

Developed by Doug Cutting and Mike Cafarella

Origin - Apache Nutch

- Goal : web search engine on 1 Billion Pages

Open source

Google Distributed File System:

<https://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf>



Google File System (GFS) - Overview and Key Concepts

- **Architecture:** A GFS cluster has a simple, centralized architecture consisting of a single **Master** node and multiple **Chunk Servers**.
 - **Master Node:** Stores all file system metadata, including the namespace, access control information, and the mapping of files to chunks.
 - **Chunk Servers:** Store the actual data chunks on local disks.
- **Chunk Size:** GFS uses a large, fixed chunk size of 64 MB. This reduces the amount of metadata the master needs to manage and minimizes client-master interaction for sequential reads and writes.
- **Replication:** Each chunk is replicated across multiple chunk servers (default of three) to provide fault tolerance and increase read throughput.

Google File System (GFS) - Overview and Key Concepts

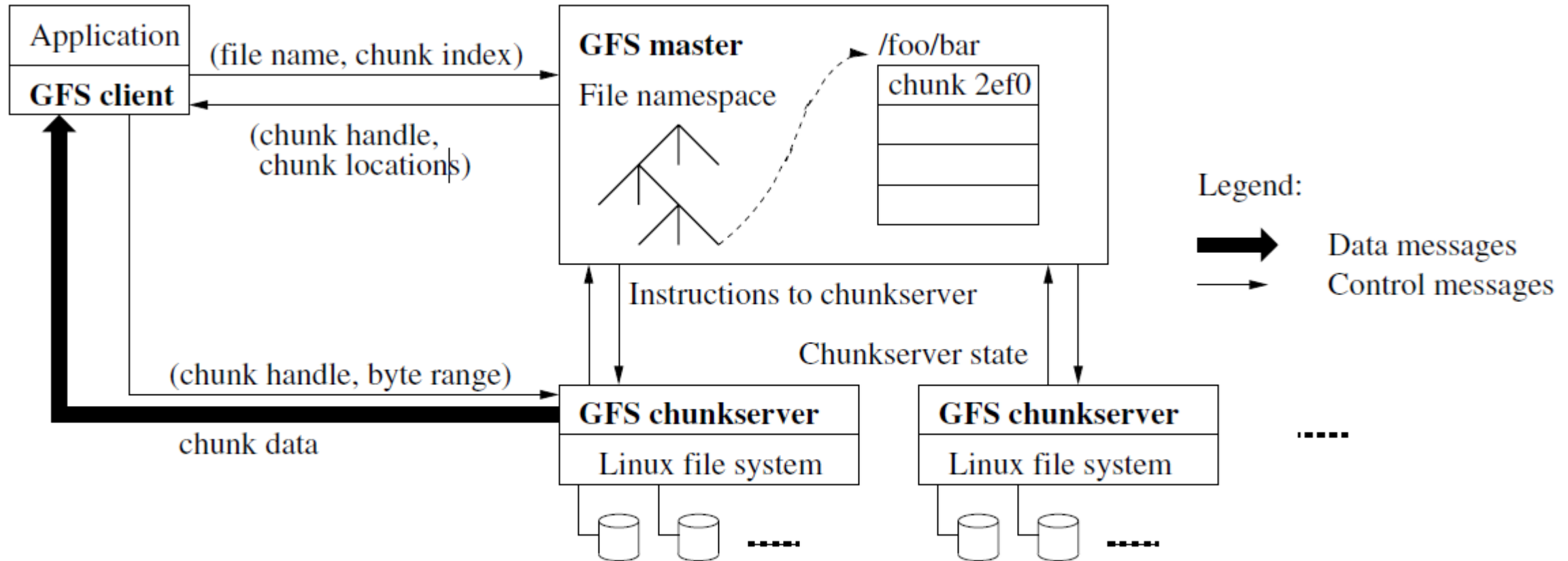


Figure 1: GFS Architecture

Google File System (GFS) - Overview and Key Concepts

Operations

Read/Write Operations:

- Clients query the master for the chunk locations, then communicate directly with the chunk servers.

Atomic Record Appends:

- Designed for concurrent writes where multiple clients can append to the same file simultaneously.

Snapshots:

- Efficiently copy data for backup or to create replicas.

Fault Tolerance

Heartbeat Messages:

- The master periodically exchanges heartbeat messages with each chunkserver to verify liveness, collect chunkserver state, and send instructions

Chunk Replication:

- Ensures that data is replicated and available even if some chunk servers fail.

Data Integrity:

- Each Chunkserver uses checksums to detect data corruption. Before returning any data, it verifies the checksum of the blocks. If a mismatch is found, it reports an error and the Master arranges to clone a new, valid replica

Google File System (GFS) - Overview and Key Concepts

Performance

Caching:

- Clients cache metadata to reduce master load.

Load Balancing:

- Distributes load evenly across chunk servers to optimize resource utilization.

Recovery:

- The master quickly re-replicates chunks when chunk servers fail to maintain redundancy.

Scalability

Master Node Bottleneck:

- The system is designed to minimize the load on the master by keeping operations involving the master to a minimum.

Adding Chunk Servers:

- Easily scalable by adding more chunkservers to the system..

Google File System (GFS) - Overview and Key Concepts

Diagnostic Tools

- Extensive and detailed diagnostic logging has helped immeasurably in problem isolation, debugging, and performance analysis, while incurring only a minimal cost. Without logs, it is hard to understand transient, non-repeatable interactions between machines.
- **What is Logged:** GFS servers generate diagnostic logs that record:
 - **Significant Events:** This includes critical system actions like Chunkservers going offline or coming back online.
 - **RPC Requests and Replies:** Every Remote Procedure Call (RPC) request and its corresponding reply is logged. This provides a complete chronological record of all client-server and master-chunkserver interactions
- These diagnostic logs can be freely deleted without affecting the correctness of the system. However, we try to keep these logs around as far as space permits.

Ghemawat, S., Gobioff, H. and Leung, S.T., 2003, October. The Google file system. In *Proceedings of the nineteenth ACM symposium on Operating systems principles* (pp. 29-43).

Google File System (GFS) - Overview and Key Concepts

Paper is **cited 10613** times. (as on Aug 1st , 2026)

Ghemawat, S., Gobioff, H. and Leung, S.T., 2003, October. The Google file system. In *Proceedings of the nineteenth ACM symposium on Operating systems principles* (pp. 29-43).

https://dl.acm.org/doi/pdf/10.1145/945445.945450?casa_token=E3-97mHbAkMAAAAA:dZ7Yk5RfZXWsa_hHI5aWI3-5PCfdd7tNuc19lxtWyr9kySFhy9GPI2PcQQj4SC90JySyVFZjhwO6aZ9Y

[PDF] [The Google file system](#)

[S Ghemawat](#), [H Gobioff](#), [ST Leung](#)

[Proceedings of the nineteenth ACM symposium on Operating systems principles, 2003](#) • [dl.acm.org](#)

Abstract

We have designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients. While sharing many of the same goals as previous distributed file systems, our design has been driven by observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure

SHOW MORE ▾

☆ Save 🔖 Cite Cited by 10613 Related articles All 261 versions

3

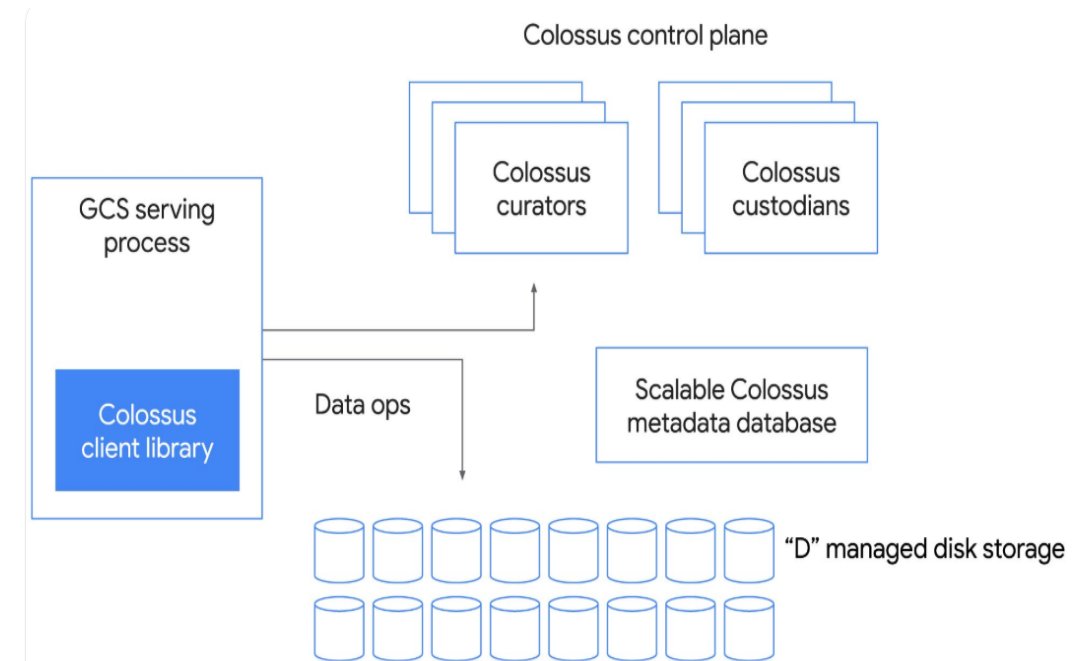
GFS, HDFS and what next?

Google internally replaced GFS with a next-generation cluster file system called **Colossus**

The Core Foundational Stack

- **Colossus File System:** This serves as Google's universal, underlying storage engine. It eliminated the single-master bottleneck of GFS by distributing metadata across Bigtable clusters, reducing chunk sizes from 64MB to 1MB, and replacing standard replication with Reed-Solomon erasure coding. [1, 2]
- **Borg & Spanner:** Colossus acts as the foundational layer alongside **Borg** (Google's resource orchestrator/ancestor to Kubernetes) and Spanner (their globally consistent relational database).

Almost all user-facing Google systems are built on top of this Colossus-driven infrastructure



<https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system>



THANK YOU

Prafullata Kiran Auradkar

Department of CSE
prafullatak@pes.edu



BIG DATA

Map Reduce Programming model & Architecture

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

What we have learnt so far..

- Large amounts of data to be processed.
- We have HDFS as a **distributed** store.
- We need to distribute the processing also.

What is Map-Reduce
?



Why do we do Map-Reduce ?

- It's the processing component of Apache Hadoop - a way to process extremely large data
- We are going to
 - Study Map-Reduce paradigm or the programming model for Map-Reduce
 - Study Hadoop architecture or how Map-Reduce works internally
 - Open Source implementation of Map-Reduce

What is MapReduce ?

- Origin from Google, [OSDI'04] & built on principles of parallel and distributed computing
- Hadoop is the adoption of open-source implementation by Yahoo (now Apache project)
- MapReduce is the central processing component of the Hadoop framework. It provides a programming model for processing vast amounts of data in a distributed environment, offering several key advantages:
 - **Parallel Processing:** MapReduce enables the parallel processing of data across multiple servers in a distributed computational environment, which significantly boosts performance.
 - **Data Locality:** This is a fundamental principle of MapReduce. Instead of moving large amounts of data to the processing unit, the framework moves the processing logic to where the data is already stored. This reduces expensive network traffic and improves efficiency.
- Its an execution framework for large-scale data processing
 - Which distributes the computing across distributed commodity servers and then pulls the results together
 - Programmer writes code as if writing for a single machine but the framework executes the process in distributed manner
 - Distributed implementation that hides all the messy details
 - Fault tolerance (thus provides High Availability)
 - I/O scheduling
 - parallelization

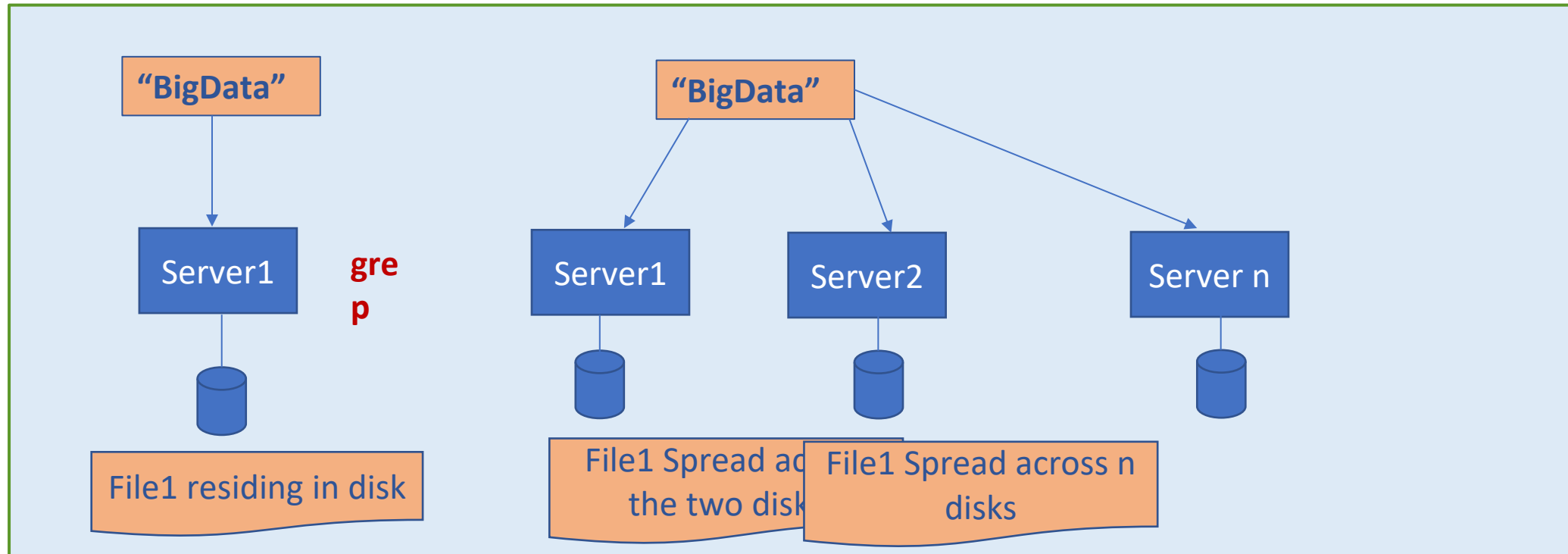
- Lots of demands for very large scale data processing leading to the need for Scaling
- Scale out not up - Symmetric multi-processing machines (SMP) with large number of processor sockets (100s), large shared memory (GBs) are significantly more expensive as compared to Cluster of low end machines which are 4x more cost effective but lots of machines needed
- Build levels of abstraction which supports beneficial division of labor
 - A simple Programming model/Interface : A map + reduce (separating the abstraction of **what** from **how** of data intensive processing)
 - An application programmer develops using well defined interfaces and passes on the data (**what**) and hence is isolated from system-level details like locking, starvation, etc.
 - The framework (Map-Reduce) which is designed once and verified for correctness, takes care of the **How**. This includes the system-level details of writing distributed programs with details of threads, processes, machines Code that runs concurrently and needing to factor in Deadlocks, race conditions, etc.

Big Data: Illustrative Example leading to Map Reduce

BIG DATA

A MapReduce Example

- Consider a very large text file and you want to determine if a word exists in this file? E.g. “BigData” in File1
 - Say if the file is stored in HDFS across two machines.
 - How will you search to check if the word “Big Data” is present in the file?



Browse Directory

Show 25 entries
 Search:

☐	↕ Permission	↕ Owner	↕ Group	↕ Size	↕ Last Modified	↕ Replication	↕ Block Size	↕ Name	↕
☐	-rw-r--r--	user	supergroup	3.18 GB	Aug 13 11:19	2	128 MB	book.txt	

Showing 1 to 1 of 1 entries

Previous
 1
 Next

File information - book.txt

[Download](#)
[Head the file \(first 32K\)](#)
[Tail the file \(last 32K\)](#)

Block information -- Block 1

Block ID: 1073748227
 Block Pool ID: BP-185901527-10.10.3.111-1720007864490
 Generation Stamp: 7403
 Size: 134217728
 Availability:

- worker-5
- worker-3

File information - book.txt

[Download](#)
[Head the file \(first 32K\)](#)
[Tail the file \(last 32K\)](#)

Block information -- Block 16

Block ID: 1073748242
 Block Pool ID: BP-185901527-10.10.3.111-1720007864490
 Generation Stamp: 7418
 Size: 134217728
 Availability:

- worker-4
- worker-3

File information - book.txt

[Download](#)
[Head the file \(first 32K\)](#)
[Tail the file \(last 32K\)](#)

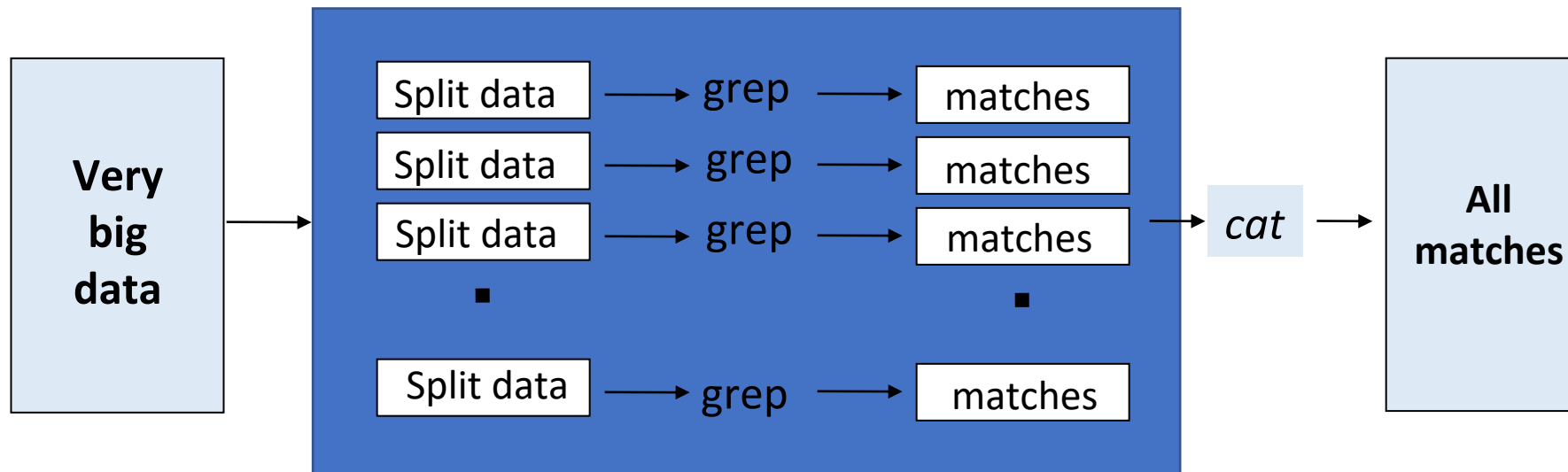
Block information -- Block 0

Block ID: 1073748226
 Block Pool ID: BP-185901527-10.10.3.111-1720007864490
 Generation Stamp: 7402
 Size: 134217728
 Availability:

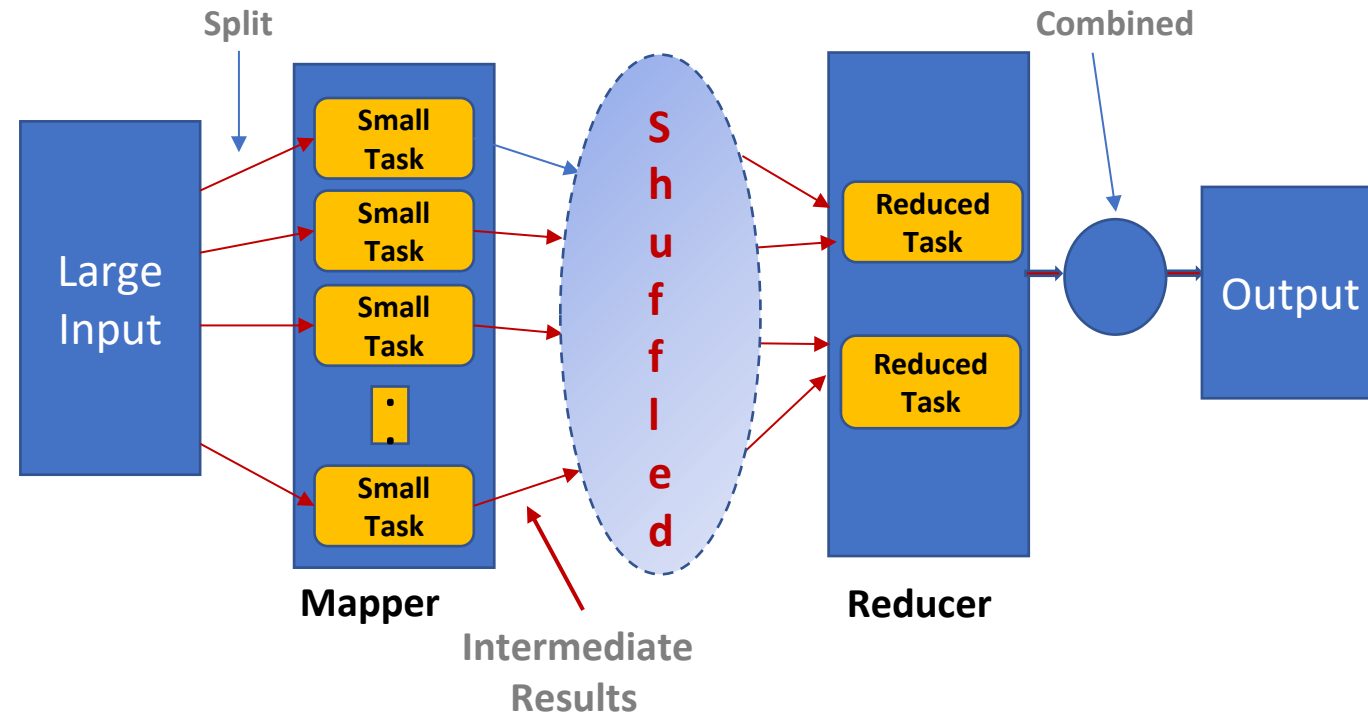
- worker-2
- worker-1

BIG DATA

Map Reduce: Distributed Grep-Solution



- Uses Divide and conquer – Partitions large problem into smaller subproblems
- Workers work on sub-problems in parallel (could be threads in a core or cores in multi-core processor, multiple processor in a machine, machines in a cluster) and produce intermediate results
- Intermediate results from workers are combined to form the final result
- Basic operations on the input
 - Map
 - Reduce
- To understand what Map/Reduce really are..



Lets consider a Distributed Word Count

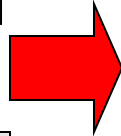


Menu 1

Idli	Vada
Pizza	

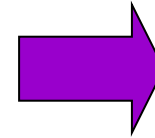
Menu 2

Dosa	Pizza
Burger	



Idli	1
Vada	1
Pizza	1

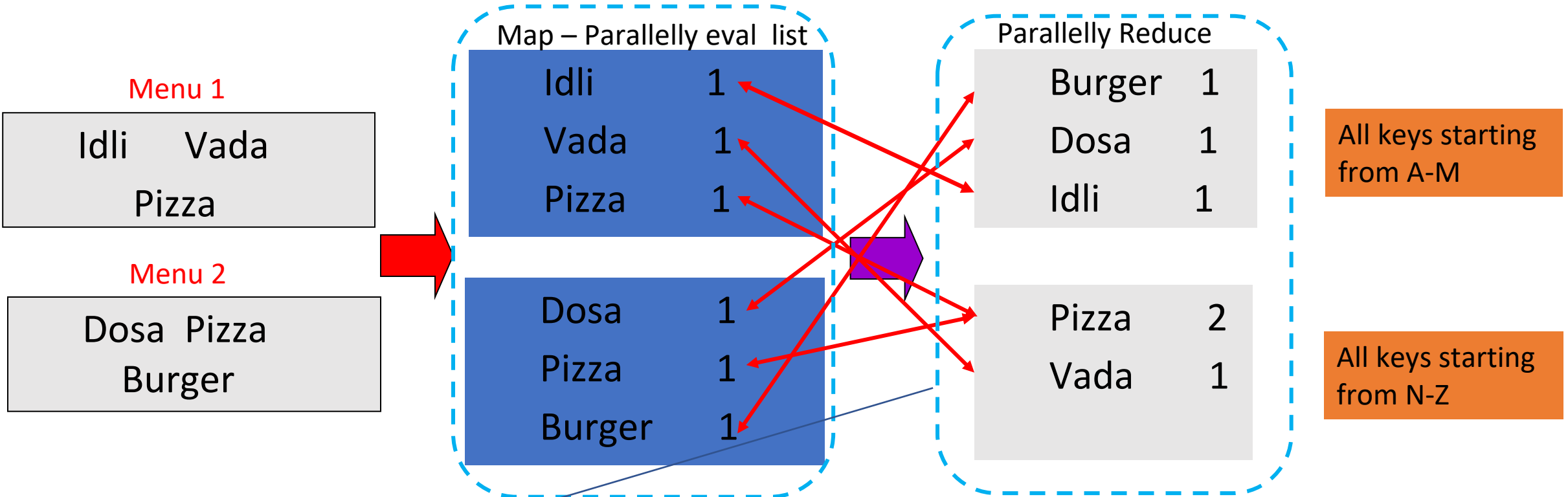
Dosa	1
Pizza	1
Burger	1



Merged results

Burger	1
Dosa	1
Idli	1
Pizza	2
Vada	1

- Suppose we need parallelism of the merge program.
- Would the earlier approach work?
- What do we need to do?

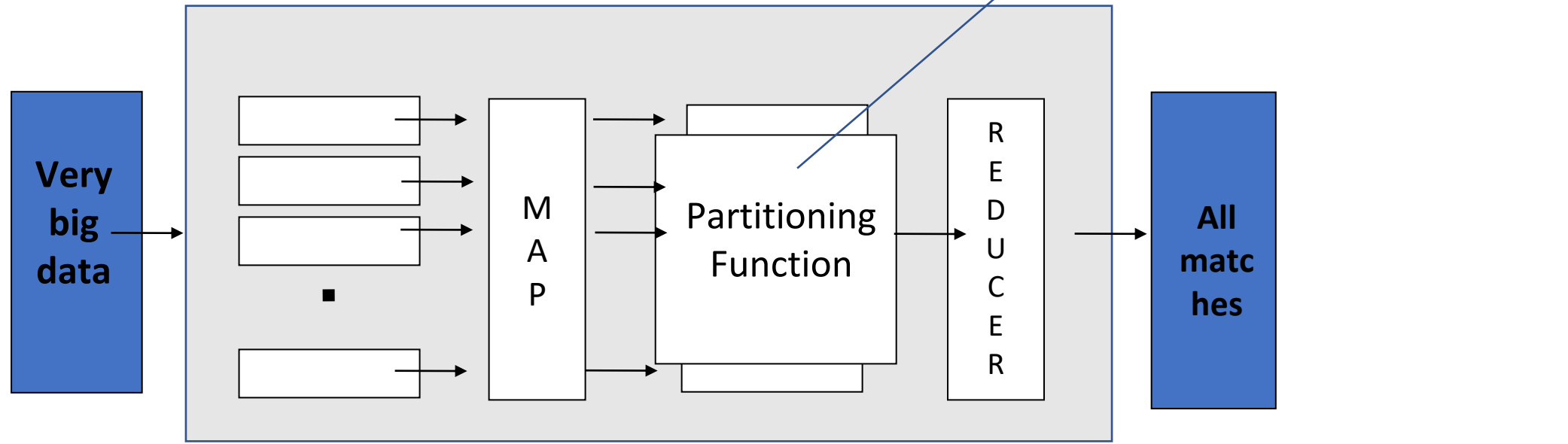


How do we know that all the “pizza” keys must be aggregated in server 2?

- Treat output of first program as a key value pair
- While partitioning the keys in the second program

BIG DATA

Map + Reduce



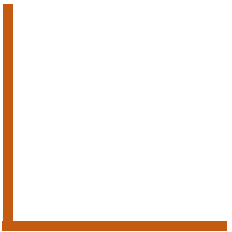
- Map:

- Accepts *input* key/value pair
- Emits *intermediate* key/value pair

- Reduce :

- Accepts *intermediate* key/value* pair
- Emits *output* key/value pair

Map Reduce: A look at the code



- **Data type:** key-value *records*

- **Map function:**

$(K_{in}, V_{in}) \rightarrow \text{list}(K_{inter}, V_{inter})$

- **Reduce function:**

$(K_{inter}, \text{list}(V_{inter})) \rightarrow \text{list}(K_{out}, V_{out})$

Pizza 1,1

Idli
Vada

Dosa Pizza
Burger

Idli 1
Vada 1
Pizza 1

Dosa 1
Pizza 1
Burger 1

Burger 1
Dosa 1
Idli 1

Pizza 2
Vada 1

Focus is on the Key-value pairs: NOT ON JAVA code or syntax

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
        ) throws IOException, InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

$(K_{in}, V_{in}) \rightarrow list(K_{inter}, V_{inter})$

Hadoop data type

List (Key ,value)

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Key ,List (value)
                      Context context
                      ) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result); List (Key ,value)
    }
}
```

$(K_{inter}, list(V_{inter})) \rightarrow list(K_{out}, V_{out})$

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] otherArgs = new GenericOptionsParser(conf, args).
        getRemainingArgs();
    if (otherArgs.length < 2) {
        System.err.println("Usage: wordcount <in> [<in>...] <out>");
        System.exit(2);
    }
    Job job = new Job(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);

    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    for (int i = 0; i < otherArgs.length - 1; ++i) {
        FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
    }
    FileOutputFormat.setOutputPath(job,
        new Path(otherArgs[otherArgs.length - 1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

**Set Mapper
and
Reducer class**

- This does not state the machine where this needs to be run in
- This also does not state how the input needs to be distributed across mappers

```
"""mapper.py"""
```

```
import sys
```

```
for line in sys.stdin:
```

```
    line = line.strip()           #Remove whitespaces
```

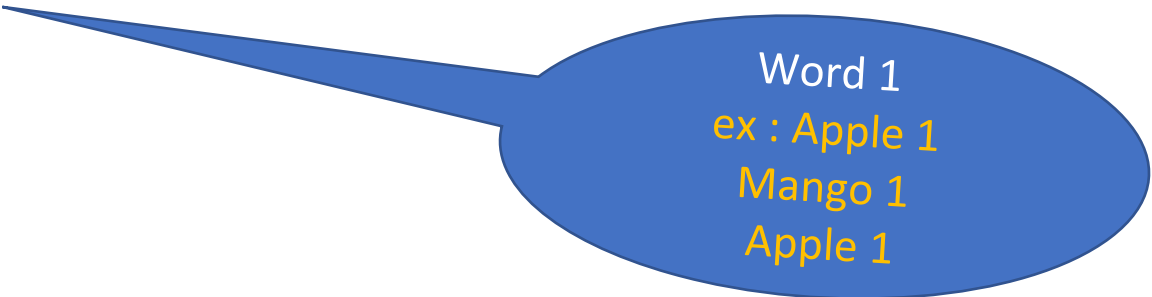
```
    words = line.split()         #line into words
```

```
    for word in words:
```

```
        # Write the results to standard output;
```

```
        print(f"{word}\t 1")      # "/" delimiter between key and
```

```
value
```



Word 1
ex : Apple 1
Mango 1
Apple 1

```
"""reducer.py"""
```

```
def reducer():  
    current_key = None  
    current_sum = 0  
    for line in sys.stdin:  
        key, value = line.strip().split("\t", 1) # Split the input line by tab  
  
        try:                                     # Convert value to integer  
            value = int(value)  
        except ValueError:                       #skip invalid value  
            continue  
  
        if current_key == key:  
            current_sum += value  
        else:  
            if current_key:  
                print(f"{current_key}\t{current_sum}") # Print the sum for the previous key  
            current_sum = value  
            current_key = key  
  
    if current_key == key:  
        print(f"{current_key}\t{current_sum}") # Print the sum for the last key
```

word 3

ex:

Apple 3

Mango 2

```

user@master:~/testHadoop$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar -files /home/user/testHadoop/wordCount/book.txt -output "/result2"
packageJobJar: [/tmp/hadoop-unjar6460106441101059560/] [] /tmp/streamjob8640532450010092031.jar tmpDir=null
2024-08-13 11:22:02,533 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/10.10.3.111:8030
2024-08-13 11:22:02,695 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/10.10.3.111:8030
2024-08-13 11:22:02,975 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/user/.staging
2024-08-13 11:22:03,568 INFO mapred.FileInputFormat: Total input files to process : 1
2024-08-13 11:22:03,824 INFO mapreduce.JobSubmitter: number of splits:26
2024-08-13 11:22:04,107 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1722926003558_0002
2024-08-13 11:22:04,107 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-08-13 11:22:04,348 INFO conf.Configuration: resource-types.xml not found
2024-08-13 11:22:04,349 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-08-13 11:22:04,965 INFO impl.YarnClientImpl: Submitted application application_1722926003558_0002
2024-08-13 11:22:05,055 INFO mapreduce.Job: The url to track the job: http://master:8088/proxy/application_1722926003558_0002/
2024-08-13 11:22:05,058 INFO mapreduce.Job: Running job: job_1722926003558_0002
2024-08-13 11:22:13,270 INFO mapreduce.Job: Job job_1722926003558_0002 running in uber mode : false
2024-08-13 11:22:13,274 INFO mapreduce.Job: map 0% reduce 0%
2024-08-13 11:22:33,489 INFO mapreduce.Job: map 11% reduce 0%
2024-08-13 11:22:35,503 INFO mapreduce.Job: map 17% reduce 0%
2024-08-13 11:22:39,533 INFO mapreduce.Job: map 21% reduce 0%
2024-08-13 11:22:41,548 INFO mapreduce.Job: map 26% reduce 0%
2024-08-13 11:22:45,576 INFO mapreduce.Job: map 32% reduce 0%
2024-08-13 11:22:47,590 INFO mapreduce.Job: map 35% reduce 0%
2024-08-13 11:22:51,617 INFO mapreduce.Job: map 42% reduce 0%
2024-08-13 11:22:53,632 INFO mapreduce.Job: map 44% reduce 0%
2024-08-13 11:22:57,662 INFO mapreduce.Job: map 47% reduce 0%
2024-08-13 11:22:58,669 INFO mapreduce.Job: map 49% reduce 0%
2024-08-13 11:22:59,711 INFO mapreduce.Job: map 52% reduce 0%
2024-08-13 11:23:03,740 INFO mapreduce.Job: map 55% reduce 0%
2024-08-13 11:23:04,747 INFO mapreduce.Job: map 56% reduce 0%
2024-08-13 11:23:05,753 INFO mapreduce.Job: map 59% reduce 0%

```

```

2024-08-13 11:23:28,926 INFO mapreduce.Job: map 91% reduce 0%
2024-08-13 11:23:29,934 INFO mapreduce.Job: map 97% reduce 0%
2024-08-13 11:23:30,941 INFO mapreduce.Job: map 97% reduce 6%
2024-08-13 11:23:33,964 INFO mapreduce.Job: map 98% reduce 6%
2024-08-13 11:23:34,976 INFO mapreduce.Job: map 100% reduce 6%
2024-08-13 11:23:36,988 INFO mapreduce.Job: map 100% reduce 9%
2024-08-13 11:23:43,027 INFO mapreduce.Job: map 100% reduce 13%
2024-08-13 11:23:49,064 INFO mapreduce.Job: map 100% reduce 17%
2024-08-13 11:23:55,103 INFO mapreduce.Job: map 100% reduce 21%
2024-08-13 11:24:01,142 INFO mapreduce.Job: map 100% reduce 24%
2024-08-13 11:24:07,180 INFO mapreduce.Job: map 100% reduce 29%
2024-08-13 11:24:13,214 INFO mapreduce.Job: map 100% reduce 33%

```


Map Reduce: Sample Exercise

Context : Search

- We have a set of files each representing a web site
- Every file contains a line number and information as records.
- We are looking or searching for a pattern in the file
- Output should be all the line numbers which has the pattern

Input: file(lineNumber, line) records and pattern

Output: lines matching a given pattern

What will be the mapper and reducer? What will be the keys?

Map:

Reduce:

Input: file (lineNumber, line) records and pattern

Output: lines matching a given pattern

Map:

```
if(line matches pattern):  
    output(lineNumber)
```

Reduce: identity function

–Alternative: no reducer (map-only job)

- **Map**

- Process a key/value pair to generate intermediate key/value pairs
- Sorts all key/value pairs before sending to reducer

- **Reduce**

- Merge all intermediate values associated with the same key
- Runs after all Map tasks are finished (why?)

- **Partition**

- By default : **hash (key) mod R** (Well balanced)
- There are cases where this can be more complex

Map Reduce: Revision exercise

Input: (key, value) records

Output: same records, sorted by key

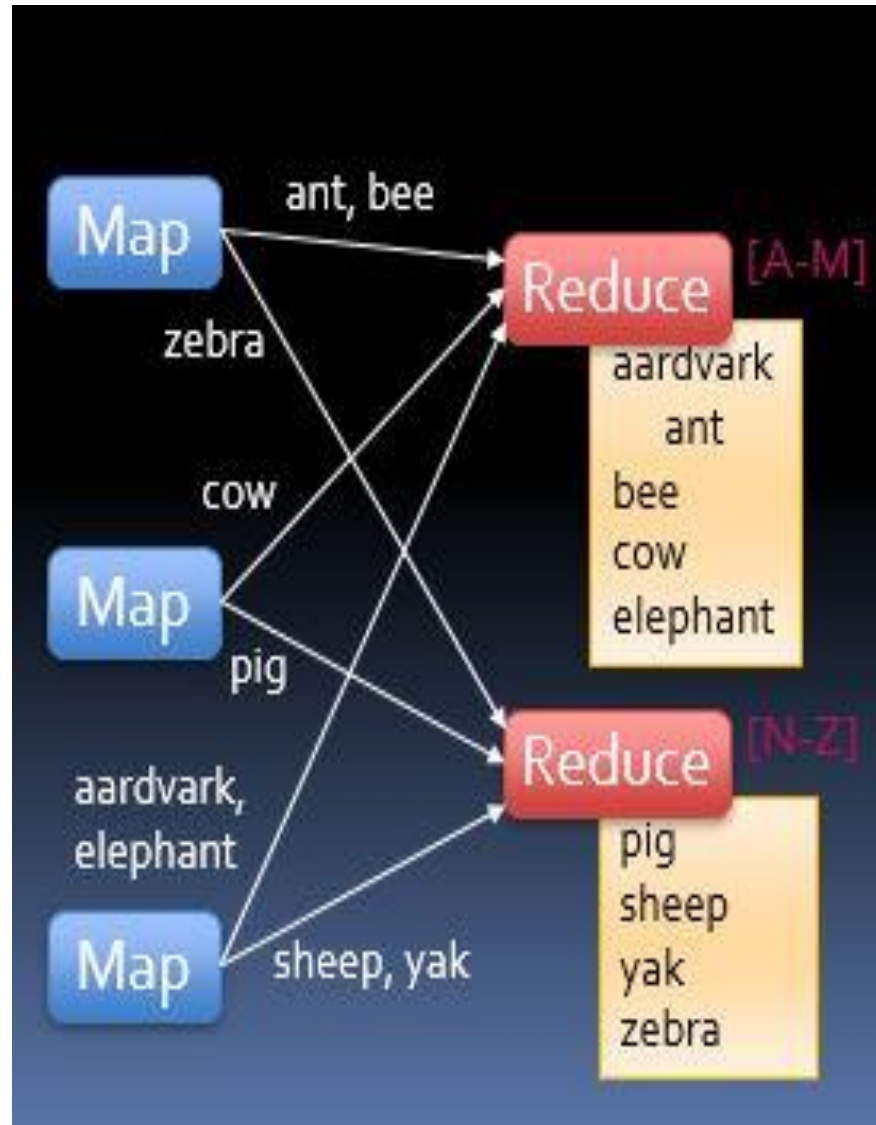
What will be the mapper and reducer?

What will be the partition function?

Map:

Reduce:

- **Input:** (key, value) records
- **Output:** same records, sorted by key
- **Map:** identity function
- **Reduce:** identity function
- **Trick:** Pick partitioning function p so that
$$k_1 < k_2 \Rightarrow p(k_1) < p(k_2)$$
- Works because map sorts output keys.

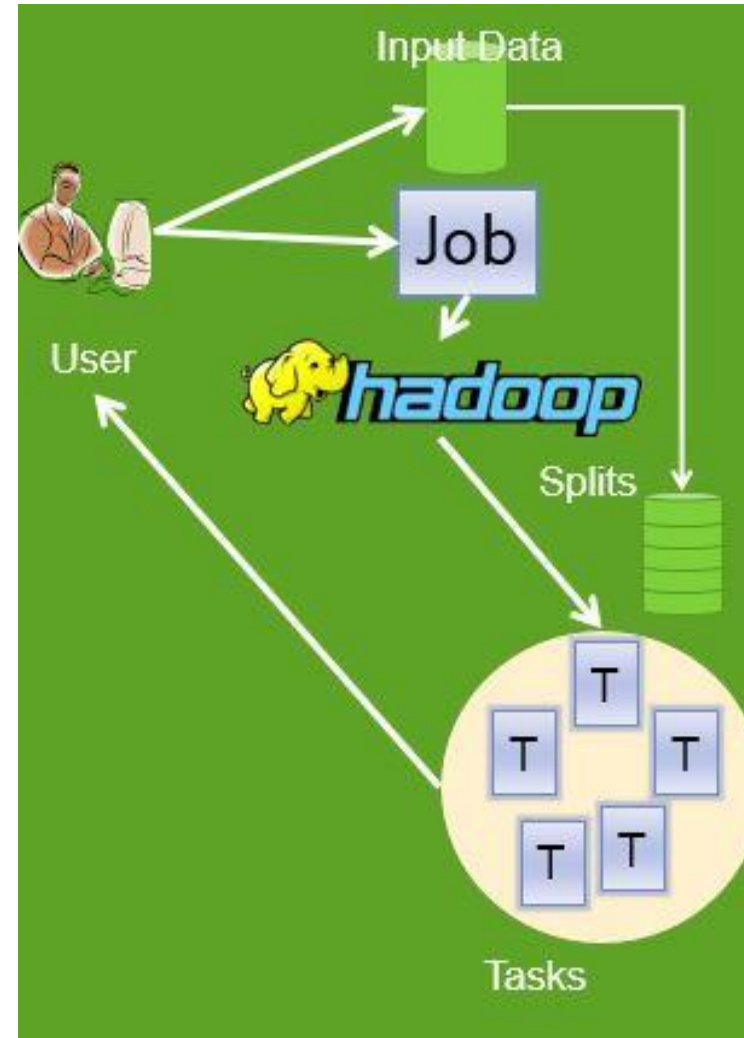


Map Reduce: Job Submission Flow

BIG DATA

Map Reduce , Hadoop Flow.

- User submits job
 - input data, MapReduce program, and configuration information
- How is parallelization achieved?
 - Divide input into smaller chunks – called **input splits**
- Hadoop divides jobs into tasks
 - Map tasks, reduce tasks
 - One map task per split
 - Tasks run in parallel



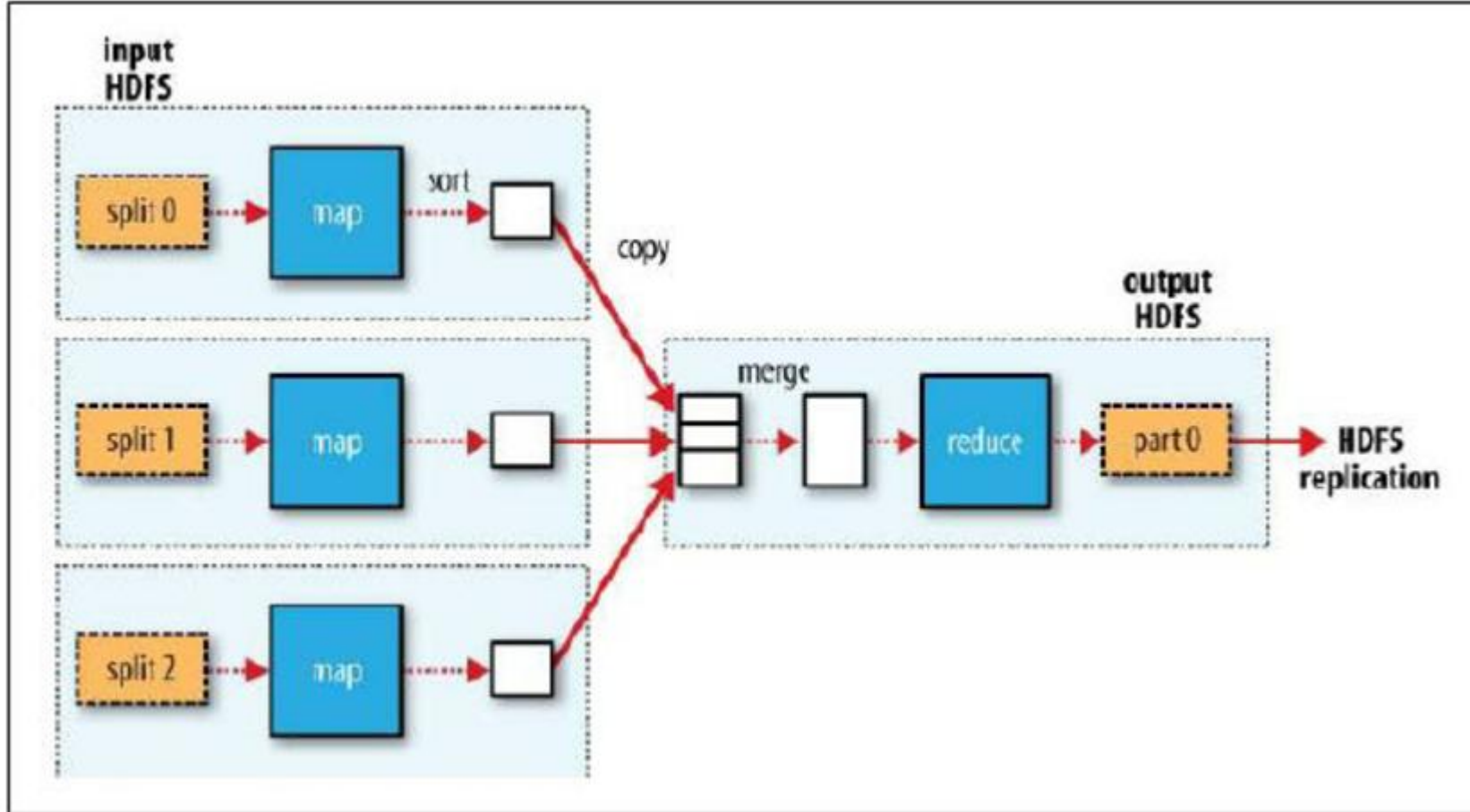


Figure: Map reduce data flow for single reducer task.

Map Reduce:

Exercise : Job Submission Flow with two Reducers

- How will it work when there are two reducers?
- Where will the outputs be?

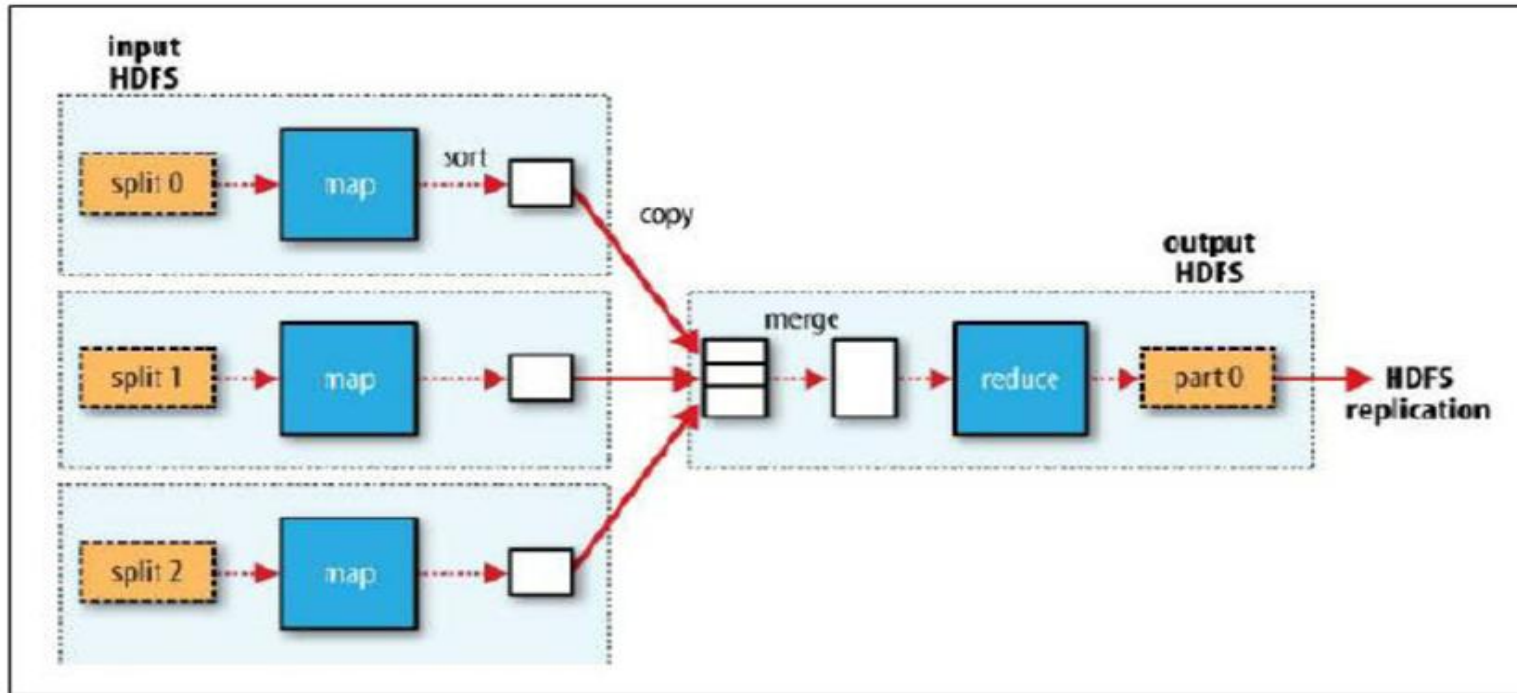
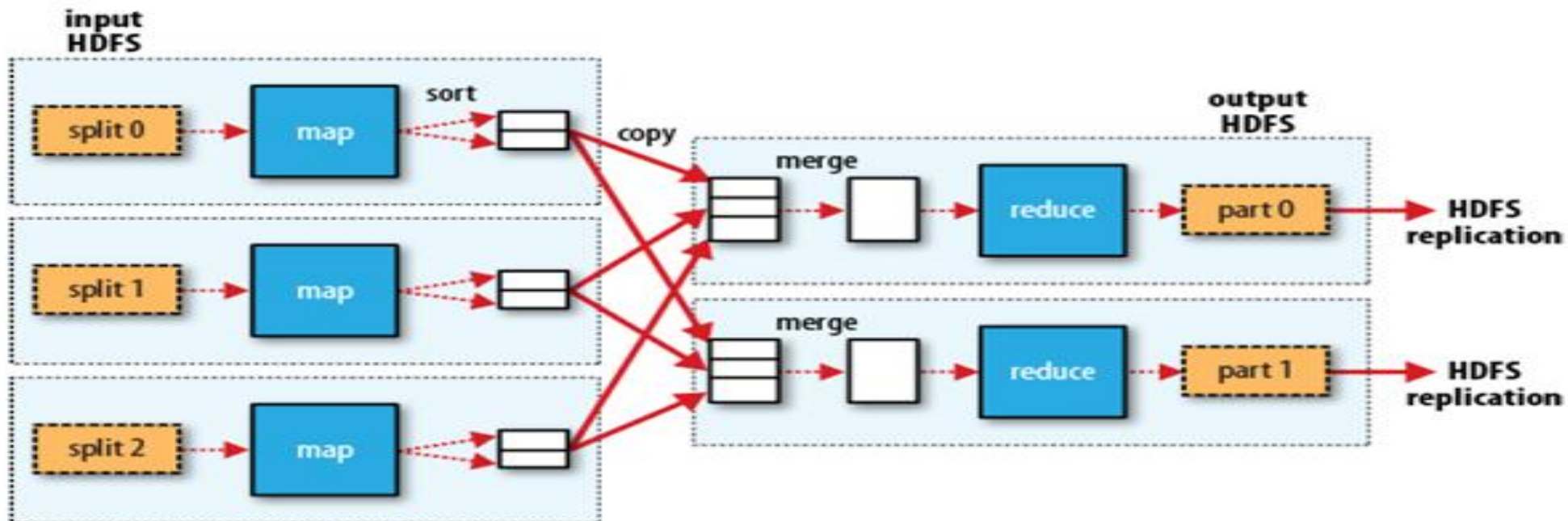


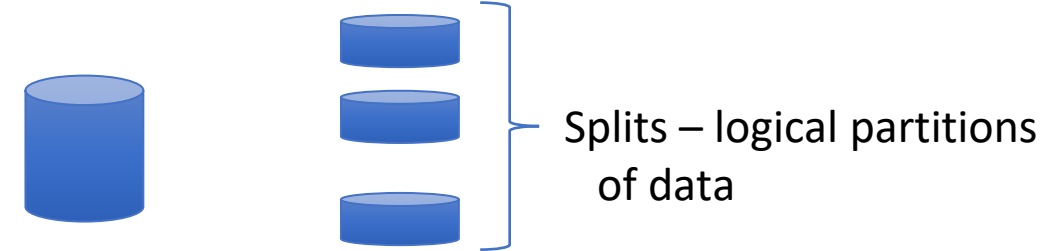
Figure: Map reduce data flow for single reducer task.



- Outputs are created on two different nodes and have to be merged.
- But available through HDFS on any node

Map Reduce: Splits

- Split size proportional to parallelism
- Small split size
 - Advantages
 - Large #splits
 - Increased parallelism
 - Increased load balancing
 - Disadvantages
 - the overhead of managing the splits and of map task creation
 - begins to dominate the total job execution time.



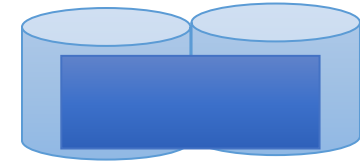
Optimal split size == size of HDFS block (128MB)
(default) on Hadoop v3

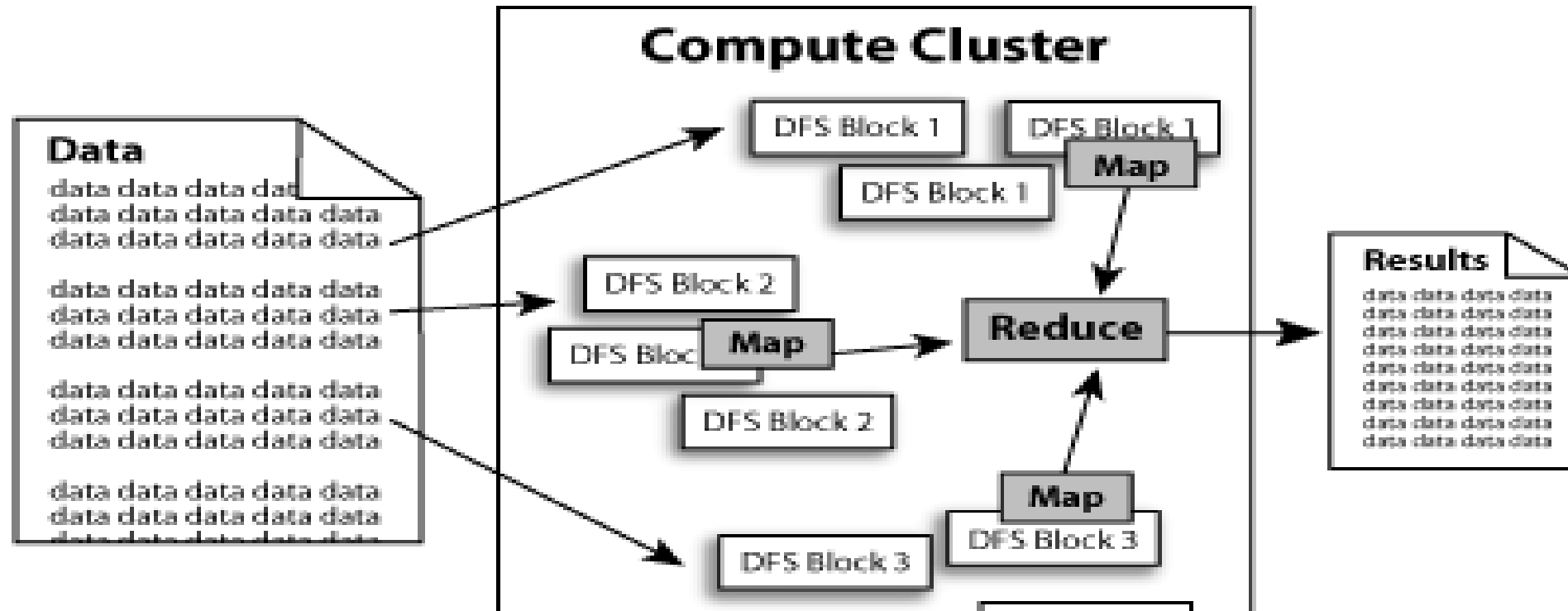
Split == Block size

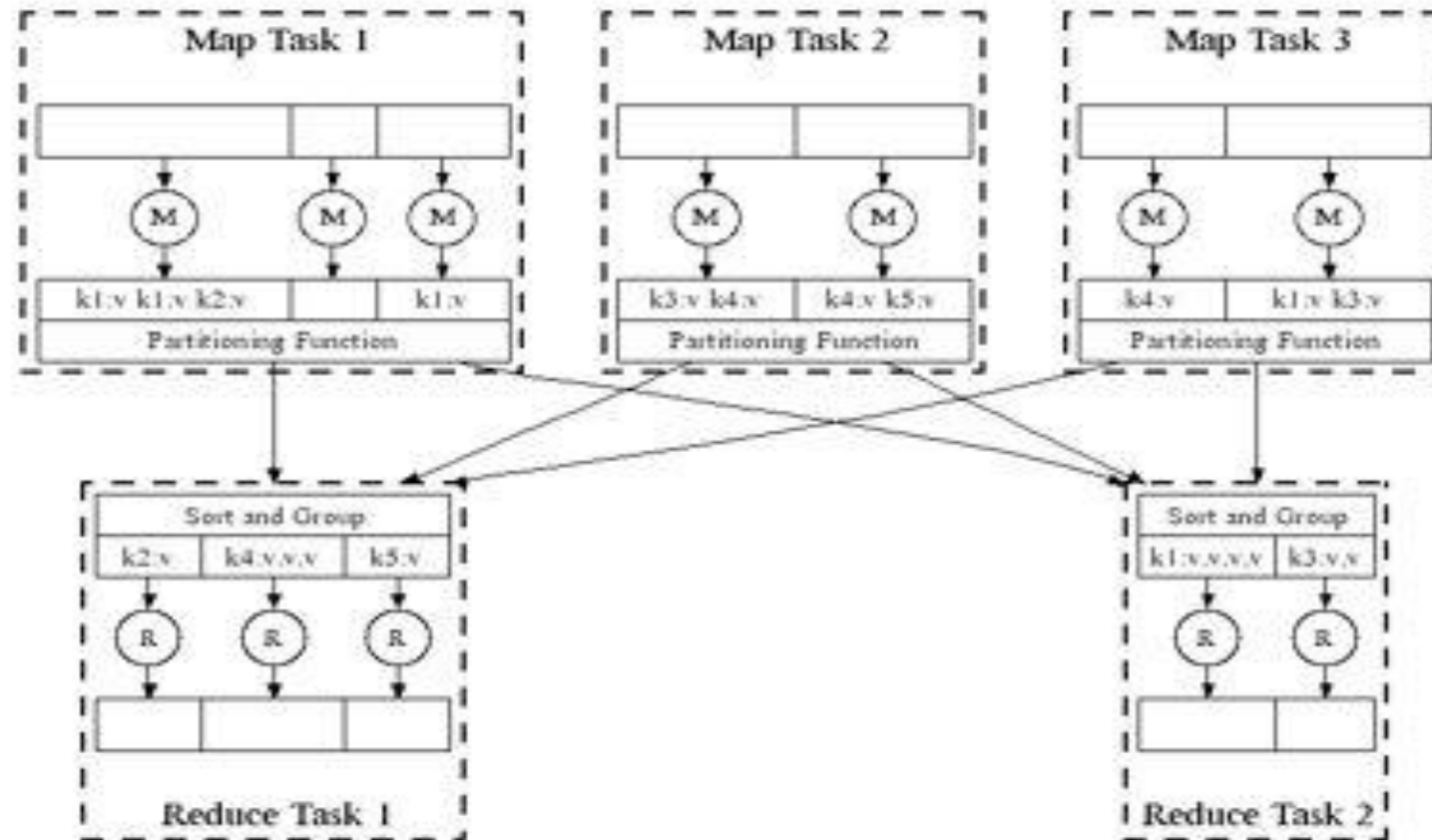
- All data required for Map
 - In the same node
 - No inter-node data transfer is required

Split != block size

- Data transfer across multiple nodes
- Impacts performance







Parallel Execution

- **Where is Map output written to?**
 - Local disk and not HDFS
 - Why? Temporary output to be discarded after reduce.
- **Failure**
 - If the node running the map task fails
 - before the output has been consumed by reducer
 - Automatically rerun map task on another node

- Reduce tasks don't have the advantage of data locality
 - the input to a single reduce task is normally the output from all mappers.
- Sorted map outputs
 - have to be transferred across the network
 - Where to?
 - To the node where the reduce task is running
 - Merge data from different mappers
 - Then passed to the user-defined reduce function.
- The output of the reduce is normally stored in HDFS for reliability.
- Where is the reduce output stored
 - 1 on the local node where the reduce happens
 - Other replicas on off-rack nodes.
 - Consumes network bandwidth

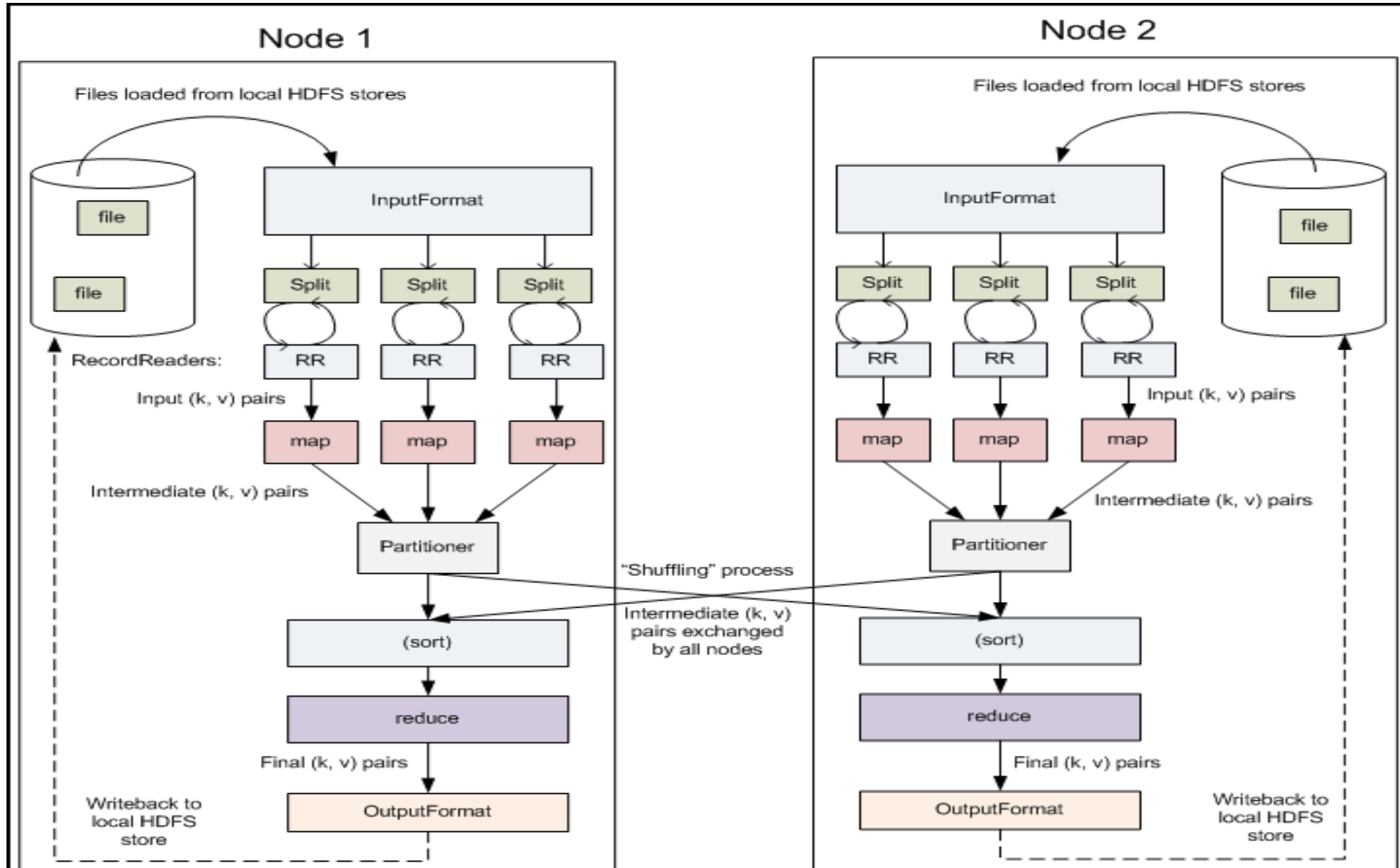
Summary: MapReduce

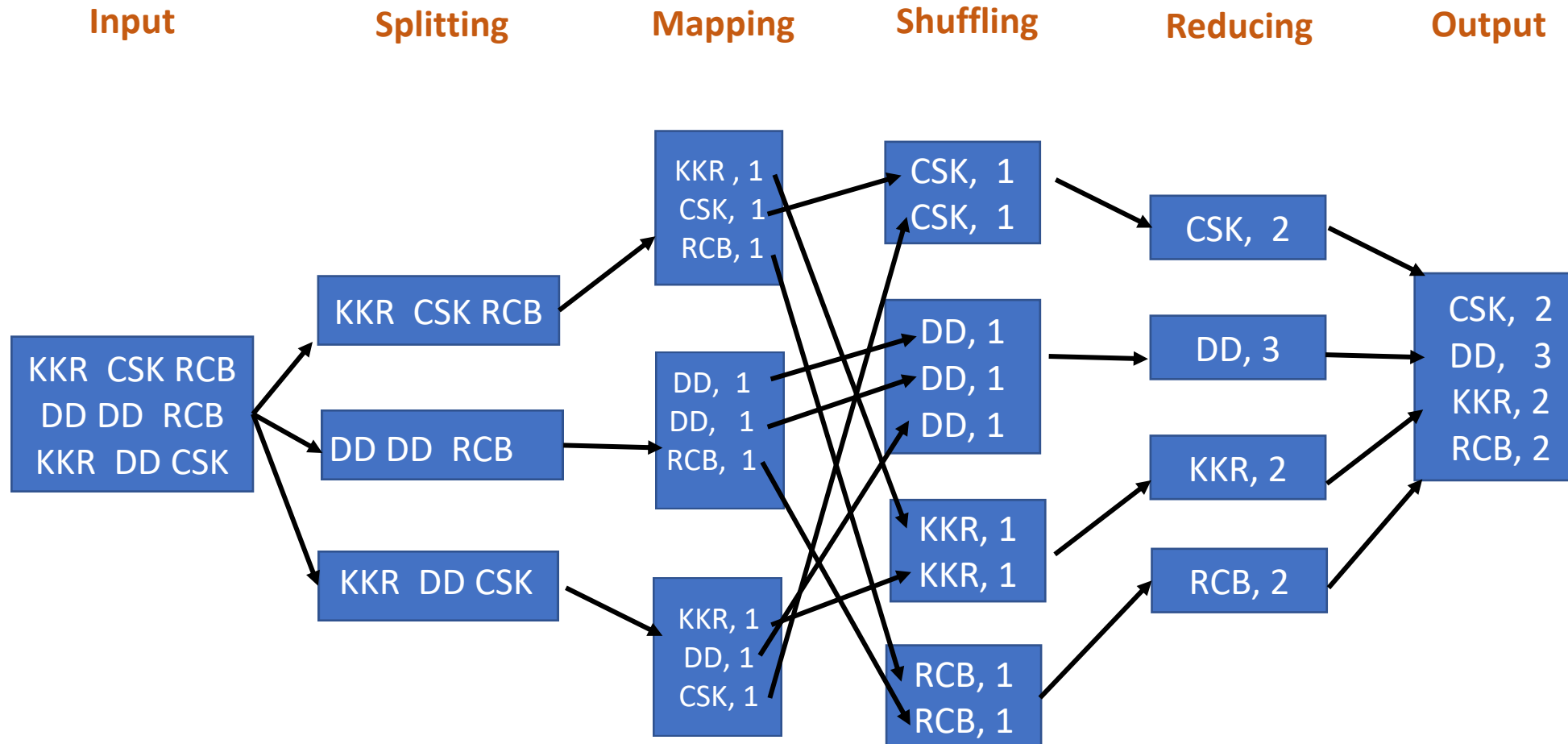
1. Parallelize Map: **easy!** each map task is independent of the other!
 - All Map output records with same key assigned to same Reducer
2. Transfer data from Map to Reduce:
 - Shuffle data
 - All Map output records with same key assigned to same Reduce task
 - use **partitioning function, e.g., $\text{hash}(\text{key})\% \text{number of reducers}$**
3. Parallelize Reduce: **easy!** each reduce task is independent of the other!
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output
 - Map input: from **distributed file system (in our case HDFS)**
 - Map output: to local disk (at Map node); uses **local file system (in our case Linux FS)**
 - Reduce input: from (multiple) remote disks; uses local file systems
 - Reduce output: to distributed file system

Map Reduce: Working

BIG DATA

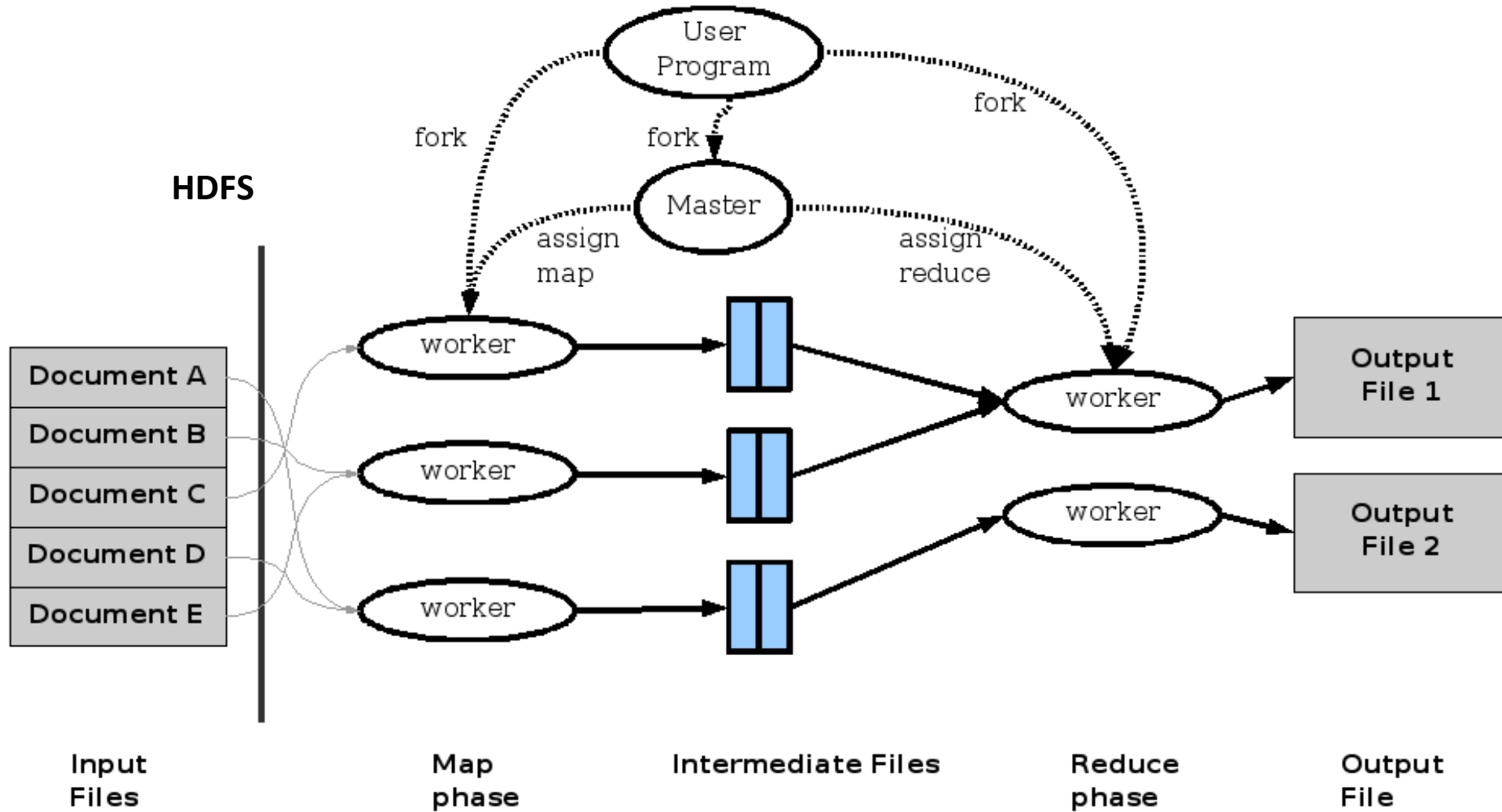
High level view



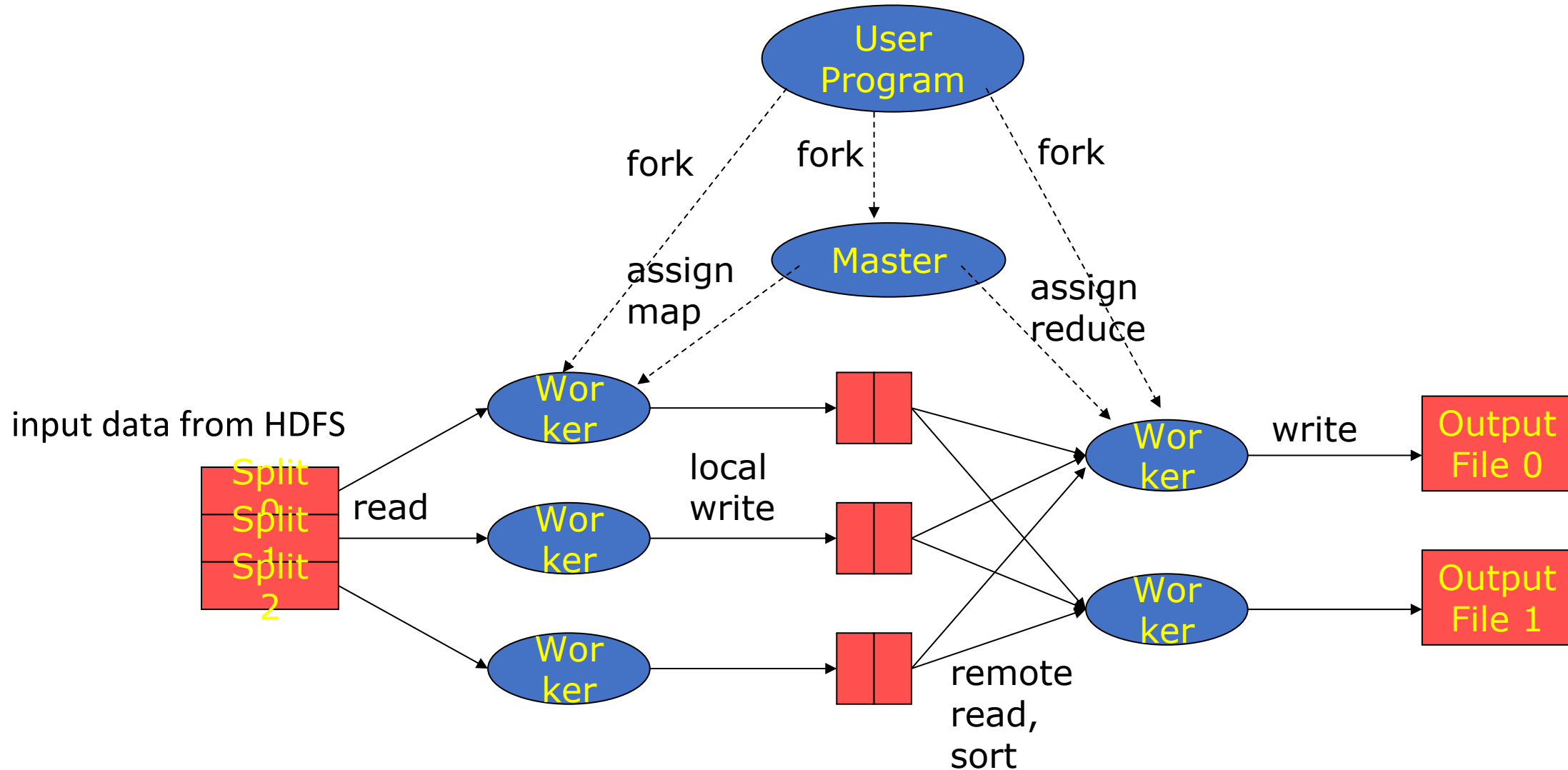


The overall Map-Reduce word count Process

MapReduce: Execution overview

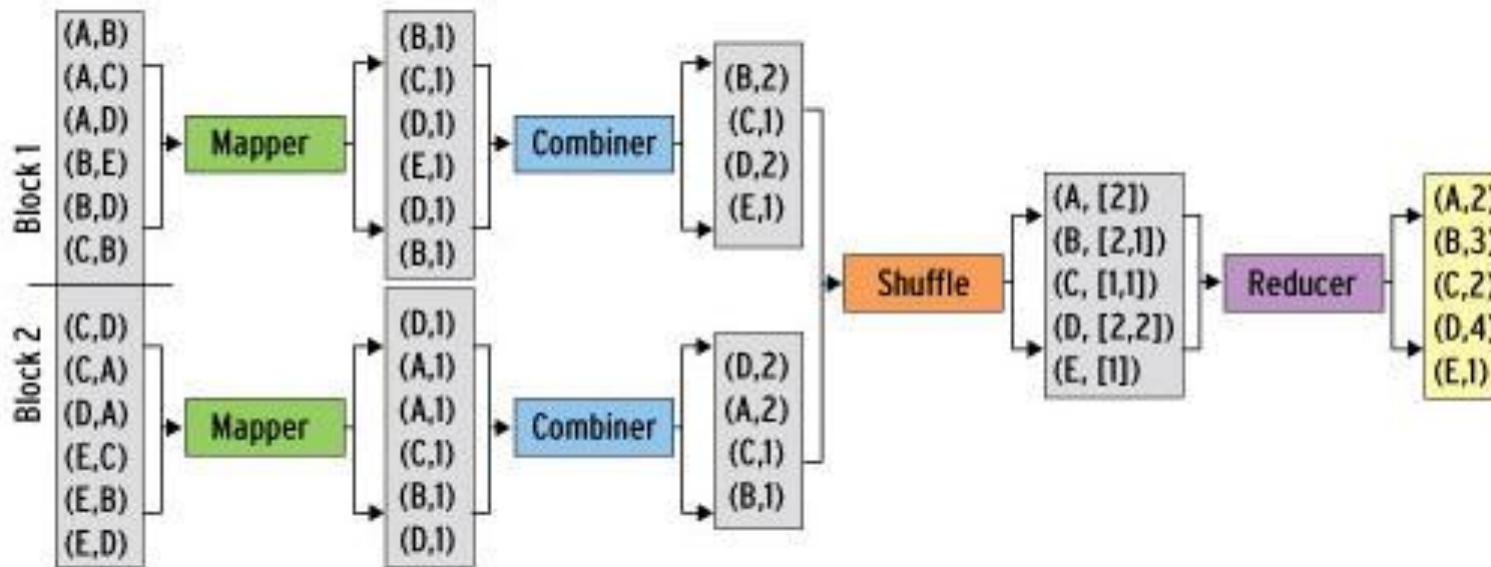


Distributed Execution Overview

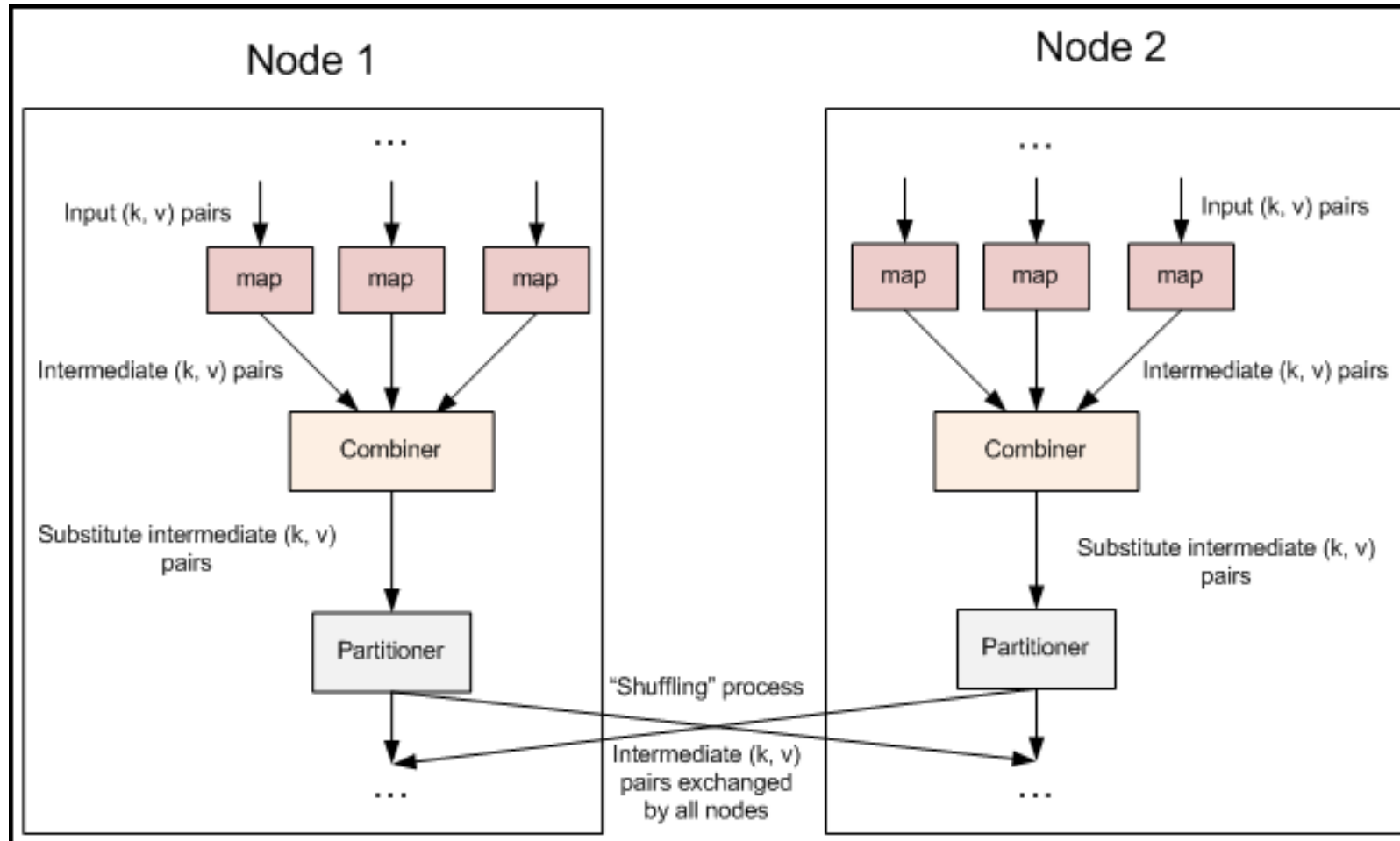


From Jeff Ullman's course slides

Map Reduce: Combiners



- Combine multiple map outputs before doing a reduce
- Can write a combiner function in program
 - Combiner will be run before reduce
- Mini-reducer



```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] otherArgs = new GenericOptionsParser(conf, args).
        getRemainingArgs();
    if (otherArgs.length < 2) {
        System.err.println("Usage: wordcount <in> [<in>...] <out>");
        System.exit(2);
    }
    Job job = new Job(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    for (int i = 0; i < otherArgs.length - 1; ++i) {
        FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
    }
    FileOutputFormat.setOutputPath(job,
        new Path(otherArgs[otherArgs.length - 1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

**Combiner is
set here**



Combiner is
set here



```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar -  
input /sample2 -output /output -mapper "python3 wc_map.py" -combiner  
"python3 wc_red.py" -reducer "python3 wc_red.py" -file wc_map.py -file  
wc_red.py
```

```
prafullata@prafullata-VirtualBox:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \  
-input /sample \  
-output /output \  
-mapper "python3 wc_map.py" \  
-combiner "python3 wc_red.py" \  
-reducer "python3 wc_red.py" \  
-file wc_map.py \  
-file wc_red.py  
2025-08-19 09:26:07,582 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.  
packageJobJar: [wc_map.py, wc_red.py, /tmp/hadoop-unjar13182960491298246448/] [] /tmp/streamjob676250065174612109.jar tmpDir=  
2025-08-19 09:26:08,936 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8  
2025-08-19 09:26:09,246 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8  
2025-08-19 09:26:09,644 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/prafu  
a/.staging/job_1755575505405_0001  
2025-08-19 09:26:10,835 INFO mapred.FileInputFormat: Total input files to process : 1  
2025-08-19 09:26:10,891 INFO net.NetworkTopology: Adding a new node: /default-rack/127.0.0.1:9866  
2025-08-19 09:26:11,473 INFO mapreduce.JobSubmitter: number of splits:2  
2025-08-19 09:26:12,494 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1755575505405_0001  
2025-08-19 09:26:12,495 INFO mapreduce.JobSubmitter: Executing with tokens: []  
2025-08-19 09:26:12,765 INFO conf.Configuration: resource-types.xml not found  
2025-08-19 09:26:12,766 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.  
2025-08-19 09:26:13,104 INFO impl.YarnClientImpl: Submitted application application_1755575505405_0001  
2025-08-19 09:26:13,177 INFO mapreduce.Job: The url to track the job: http://prafullata-VirtualBox:8088/proxy/application_1755
```



PES
UNIVERSITY

CELEBRATING 60 YEARS

Review

- Suppose
 - We have a 2 GB file
 - Split size is 128 MB
 - We have 4 disks (one per node)
- How many splits are there?
- How many splits per disk?
- How many map tasks?
- How many map tasks per node?
- How many reduce tasks?

- Suppose
 - We have a 2 GB file
 - Split size is 128 MB
 - We have 4 disks (one per node)
- How many splits are there? = $2048\text{mb}/128\text{mb} = 16$
- How many splits per disk? = $16 \text{ splits}/4 \text{ disks} = 4$
- How many map tasks? - 16 splits = 16 map tasks
- How many map tasks per node? - $16 \text{ maps} / 4 \text{ nodes} = 4$
- How many reduce tasks? – User specified

Block 1

Far out in the uncharted backwaters of the unfashionable end of the Western Spiral arm of the Galaxy lies a small unregarded yellow sun. Orbiting this at a distance of roughly ninety-eight mi

Block 2

llion miles is an utterly insignificant little blue-green planet whose ape-descended life forms are so amazingly primitive that they still think digital watches are a pretty neat idea

- You run Word count using Hadoop on this data
- We know each block is an input split
- And each split is processed by a different mapper
- Do we get the right result?
- How will you solve this?

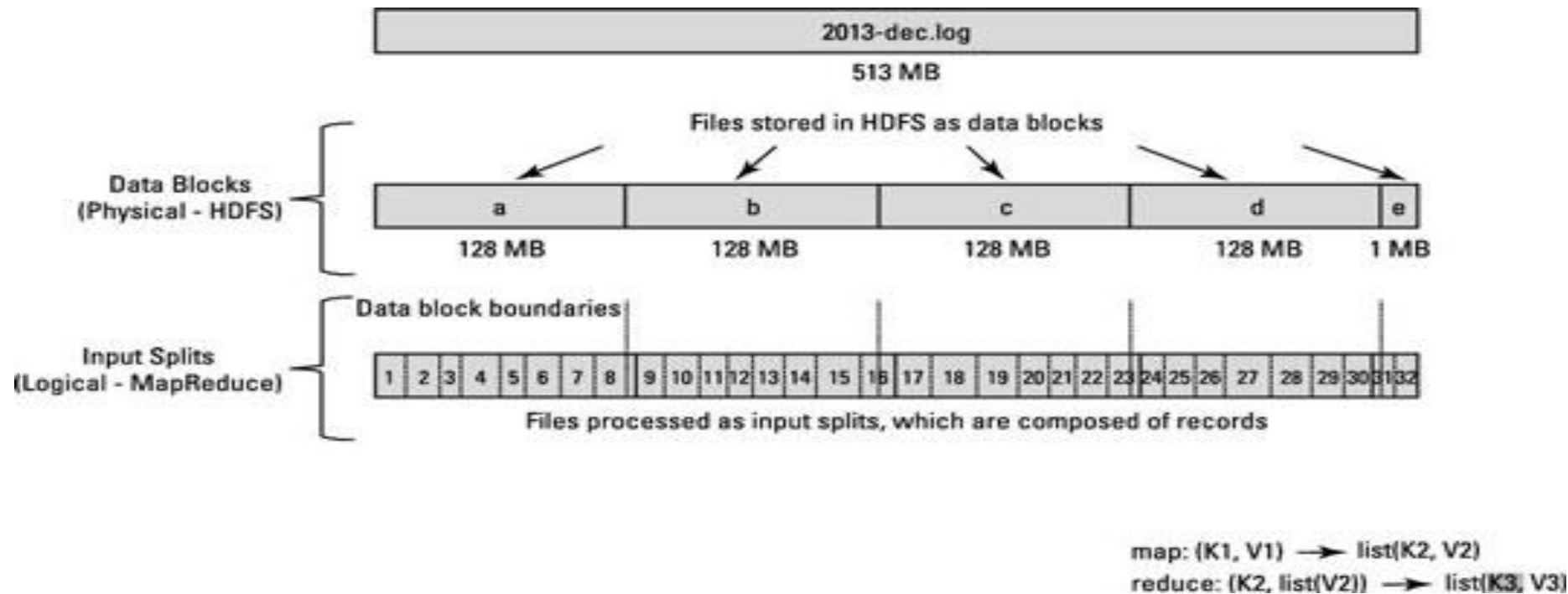
Block 1

Far out in the uncharted backwaters of the unfashionable end of the Western Spiral arm of the Galaxy lies a small unregarded yellow sun. Orbiting this at a distance of roughly ninety-eight mi

Block 2

llion miles is an utterly insignificant little blue-green planet whose ape-descended life forms are so amazingly primitive that they still think digital watches are a pretty neat idea

- Each split further broken into records
- In this case the records would look like
 - Far
 - Out
 - in. ...
- How to handle incomplete records?
 - Mapper 1 will process million
 - By looking for EOR (end of record) separator.



<https://stackoverflow.com/questions/14291170/how-does-hadoop-process-records-split-across-block-boundaries>

End of Record boundary $\backslash n$ by default

Q: How can this be changed?

- How many mappers and reducers will get started when trying to process a 230 MB file with Hadoop v3?
 - Ans: Block size = 128MB, so there will be two blocks. Assuming one block per split there will be 2 mappers
 - #reducers is configurable.
- Where is a combiner executed?
 - On the mapper.
- Write mappers and reducer pseudo code showing keys for counting #unique words in a file?
 - Similar to word count. Just that reducer does not have to write the count.

Additional Notes, References and Videos

- Chapter 2, 6 and 7 from T1
- Tom white's book is an excellent reference for the programming component.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



Hands-on 1 → HDFS commands & Map-Reduce

Step 1 - Starting Hadoop

Navigate to the `hadoop` folder and execute the following commands. `start-all.sh` is a shell script that is used to start all the processes that hadoop requires.

```
cd
cd hadoop-3.3.6/sbin/
./start-all.sh
```

```
pes2ug22cs094@pes2ug22cs094:~/hadoop-3.3.6/sbin$ ./start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as pes2ug22cs094 in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [pes2ug22cs094]
Starting resourcemanager
Starting nodemanagers
pes2ug22cs094@pes2ug22cs094:~/hadoop-3.3.6/sbin$
```

Type `jps` to find all the Java Processes started by the shell script. You should see a total of 6 processes, including the `jps` process. Note that the order of the items and the process IDs will be different.

```
pes2ug22cs094@pes2ug22cs094:~/hadoop-3.3.6/sbin$ jps
14416 DataNode
14305 NameNode
14961 NodeManager
14597 SecondaryNameNode
15340 Jps
14847 ResourceManager
pes2ug22cs094@pes2ug22cs094:~/hadoop-3.3.6/sbin$
```



If you cannot see datanode, try the following steps:

1. in the `/hadoop-3.3.6/sbin` folder, run `./stop-all.sh`
2. then type in `cd`
3. and then change to the folder dfsdata using `cd dfsdata`
4. you should see 2 folders :

```
pes2ug22cs094@pes2ug22cs094:~/dfsdata$ ls
datanode  namenode
pes2ug22cs094@pes2ug22cs094:~/dfsdata$
```

5. `rm -rf datanode` and `rm -rf namenode`
6. `mkdir datanode` and `mkdir namenode`
7. to give the required permissions :
 - a. `sudo chmod -R 755 namenode`
 - b. `sudo chmod -R 755 datanode`
8. now, do `cd` and run `hdfs namenode -format`
9. Finally ,

```
cd
cd hadoop-3.3.6/sbin/
./start-all.sh
```

Step 2 - Accessing Hadoop from the Browser

You can access Hadoop on `localhost` on the following ports

- NameNode - <http://localhost:9870>
- DataNode - <http://localhost:9864>
- YARN Manager - <http://localhost:8088>

Step 3 - Hadoop Examples

We will be using the Wordcount example to demonstrate the usage of Hadoop. Create a text file named `input.txt` with any content you want. Next, we will put this to the HDFS folder `/example` with the following command.

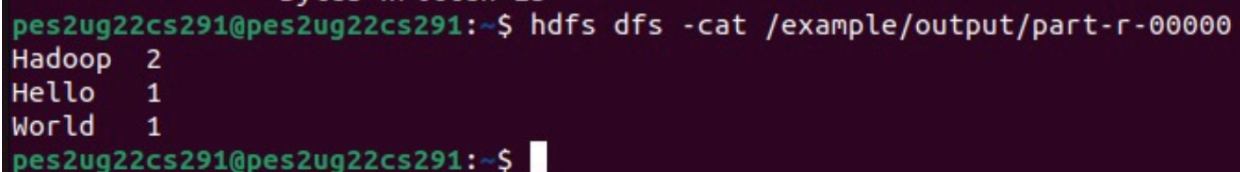
```
cd
hdfs dfs -mkdir /example
hdfs dfs -put input.txt /example
```

Run the following command for the wordcount example.

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount /example/input.txt /example/output
```

You can check the output with the following command.

```
hdfs dfs -cat /example/output/part-r-00000
```

A terminal window with a dark background. The prompt is 'pes2ug22cs291@pes2ug22cs291:~\$'. The command 'hdfs dfs -cat /example/output/part-r-00000' has been entered. The output shows three lines: 'Hadoop 2', 'Hello 1', and 'World 1'. The prompt is now 'pes2ug22cs291@pes2ug22cs291:~\$' with a cursor.

```
pes2ug22cs291@pes2ug22cs291:~$ hdfs dfs -cat /example/output/part-r-00000
Hadoop 2
Hello 1
World 1
pes2ug22cs291@pes2ug22cs291:~$
```

Step 4 - Simulating MapReduce in Local System

Example 1 - Word Count

a.) input.txt

```
Hello Hello World Big Big Big Data Data  
This is an example example
```

b.) mapper.py

```
#!/usr/bin/env python3  
  
import sys  
  
for line in sys.stdin:  
    words = line.split()  
    for word in words:  
        print(f"{word},1")
```

c.) reducer.py

```
#!/usr/bin/env python3  
import sys  
  
prev_word = None  
count = 0  
  
for line in sys.stdin:  
    curr_word, value = line.split(",")
```

```
value = int(value)
if(curr_word == prev_word):
    count += value
else:
    if prev_word:
        print(prev_word,count)
    count = 1
    prev_word = curr_word
print(prev_word,count)
```

Install dos2unix if you haven't before

```
sudo apt install dos2unix
```

Convert the files to unix format using the following command :

```
dos2unix *.py
```

Pass input to a mapper file using `echo` command

```
cat input.txt | ./mapper.py
```

```
pes2ug22cs094@pes2ug22cs094:~/BD25$ cat input.txt | ./mapper.py
Hello 1
Hello 1
World 1
Big 1
Big 1
Big 1
Data 1
Data 1
This 1
is 1
an 1
example 1
example 1
pes2ug22cs094@pes2ug22cs094:~/BD25$
```

You can then sort the output from the mapper file using :

```
cat input.txt | ./mapper.py | sort
```

```
pes2ug22cs094@pes2ug22cs094:~/BD25$ cat input.txt | ./mapper.py | sort
an 1
Big 1
Big 1
Big 1
Data 1
Data 1
example 1
example 1
Hello 1
Hello 1
is 1
This 1
World 1
pes2ug22cs094@pes2ug22cs094:~/BD25$
```

Now, finally call the reducer file

```
cat input.txt | ./mapper.py | sort | ./reducer.py
```



```
pes2ug22cs094@pes2ug22cs094:~/BD25$ cat input.txt | ./mapper.py | sort | ./reducer.py
an      1
Big     3
Data    2
example 2
Hello   2
is      1
This    1
World   1
pes2ug22cs094@pes2ug22cs094:~/BD25$
```

Example 2 - Average Rating of a Movie

a.) input.txt

```
interstellar,5
inception,3
interstellar,5
interstellar,3
interstellar,4
inception,3
inception,4
inception,1
```

b.) mapper.py

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    try:
        movie, rating = line.split(",")
        print(f"{movie}\t{rating}")
```

```
except:
    continue
```

c.) reducer.py

```
#!/usr/bin/env python3
import sys

current_movie = None
total = 0
count = 0

for line in sys.stdin:
    line = line.strip()
    movie, rating = line.split("\t")
    rating = float(rating)

    if current_movie == movie:
        total += rating
        count += 1
    else:
        if current_movie:
            avg = total / count
            print(f"{current_movie}\t{avg:.2f}")
            current_movie = movie
            total = rating
            count = 1

if current_movie:
    avg = total / count
    print(f"{current_movie}\t{avg:.2f}")
```

Run the command after you save the mapper and reducer files.

```
cat input.txt | ./mapper.py | sort | ./reducer.py
```

```
pes2ug22cs094@pes2ug22cs094:~/bdc1ass$ cat input.txt | ./mapper.py | sort | ./reducer.py
inception      2.75
interstellar    4.25
pes2ug22cs094@pes2ug22cs094:~/bdc1ass$
```



BIG DATA

Job Management and YARN

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

What we have learnt so far..

- Data processing distributed over a cluster – Map Reduce
- Job Submission Flow
- How does job management actually happen?
- How is failure management addressed?

Lecture Overview

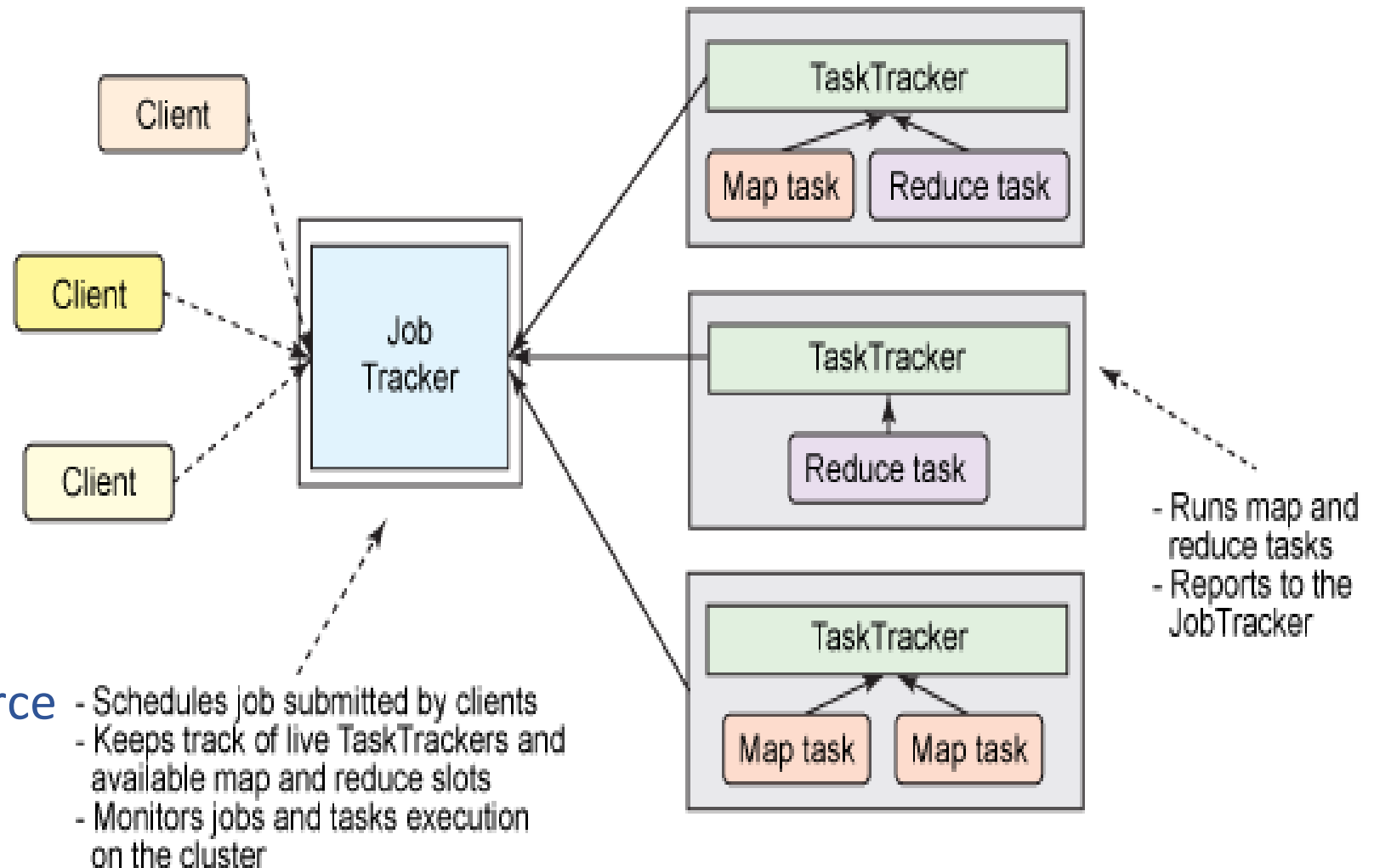
- Need for YARN - history
- YARN Architecture
- Job submission lifecycle – YARN
- Scheduling
- Failure Handling
- Benefits of YARN



Big Data: The need for YARN

- Recall
 - Job – the entire map reduce application
 - Task – Individual mappers/reducers
- How do we
 - Allocate resources – determine which nodes will run the jobs
 - Monitor the tasks – start new tasks or restart failed/slow tasks
 - Monitor the overall state of the job?

- Job Tracker
 - Manage Cluster resources
 - Job scheduling
- Task Tracker
 - One per task
 - Manage the task
- Fault Tolerance, Cluster resource management and scheduling handled by JobTracker



Limits scalability

- Job tracker runs on a single machine and is responsible for cluster management, scheduling and monitoring

Availability

- JobTracker is the single point of availability/failure

Resource utilization problems

- Predefined #map/reduce slots. Utilization issues because map slots may be full but reduce slots are free.

Limitation in running MR applications

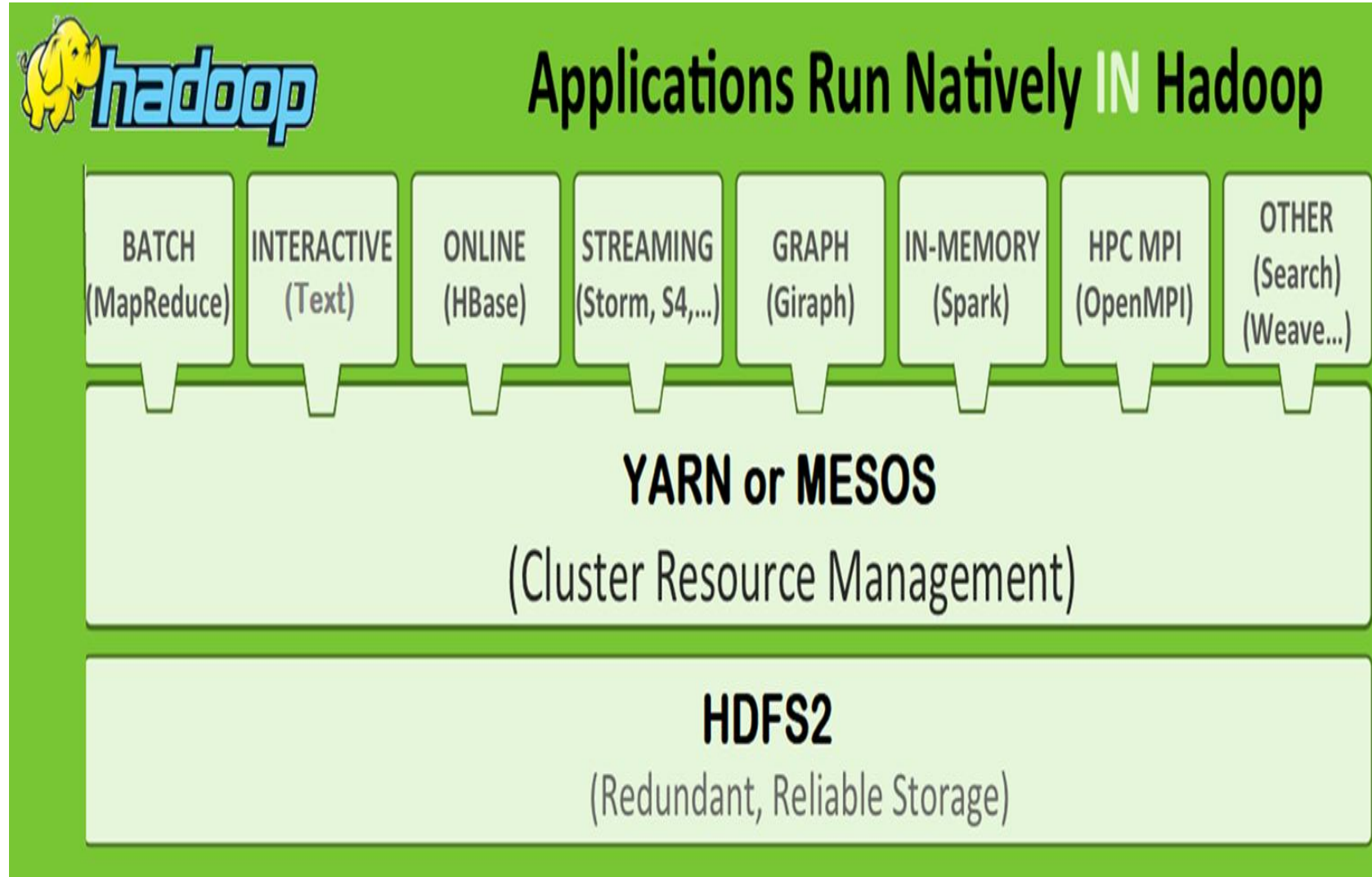
- Tightly integrated with Hadoop. Only MR apps can run. Can't coexist with other applications.

Big Data: YARN Architecture

BIG DATA

Map Reduce - Motivation

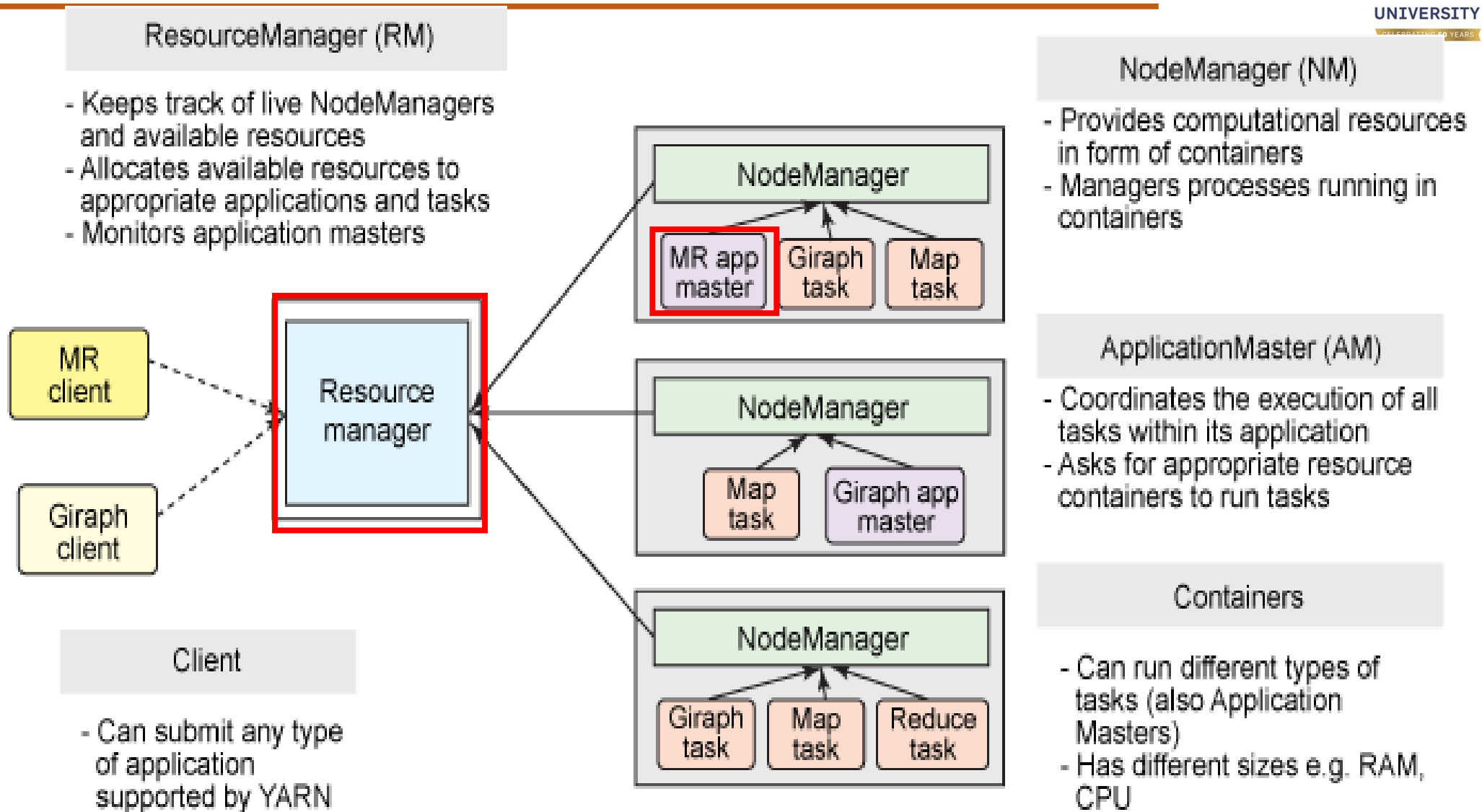
- Issues in managing clusters
> 4000 nodes
- 2010 – MapReduce v2 with YARN
 - Yet Another Resource Negotiator
 - YARN Application Resource Negotiator!!



BIG DATA

YARN Architecture

- Split responsibility of Job Tracker
- Resource Manager – manage cluster wide resources
- Application Master – manage lifecycle of application



Resource Manager

- Arbitrates resources amongst all applications of the system

Node Manager

- Per machine slave
- Responsible for launching application containers
- Monitors resource usage

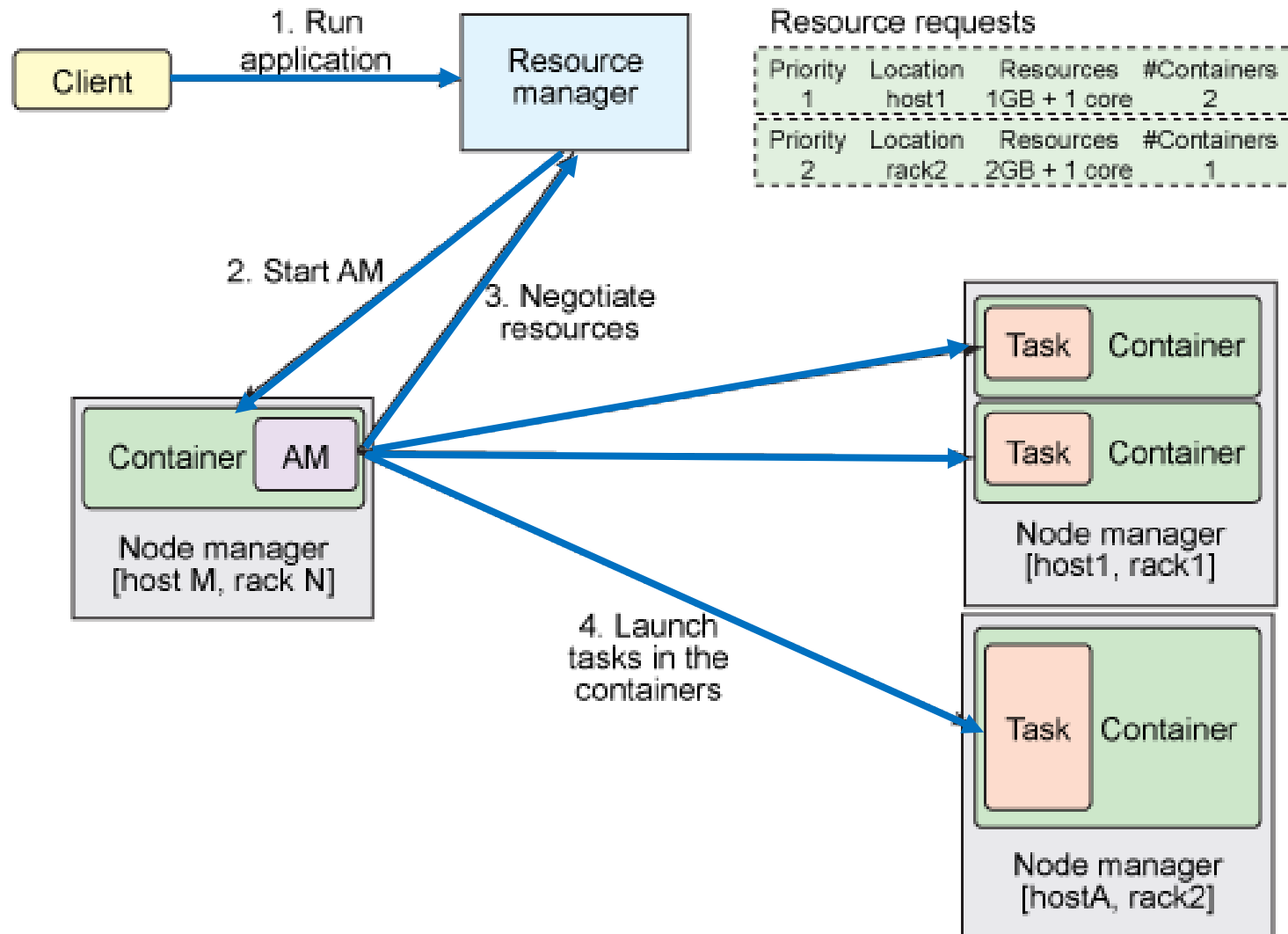
Application Master

- Negotiate appropriate resource containers from the scheduler
- Track and monitor the progress of the containers

Container

- Unit of allocation incorporating resources such as memory, CPU, disk

Big Data: Job Submission - YARN



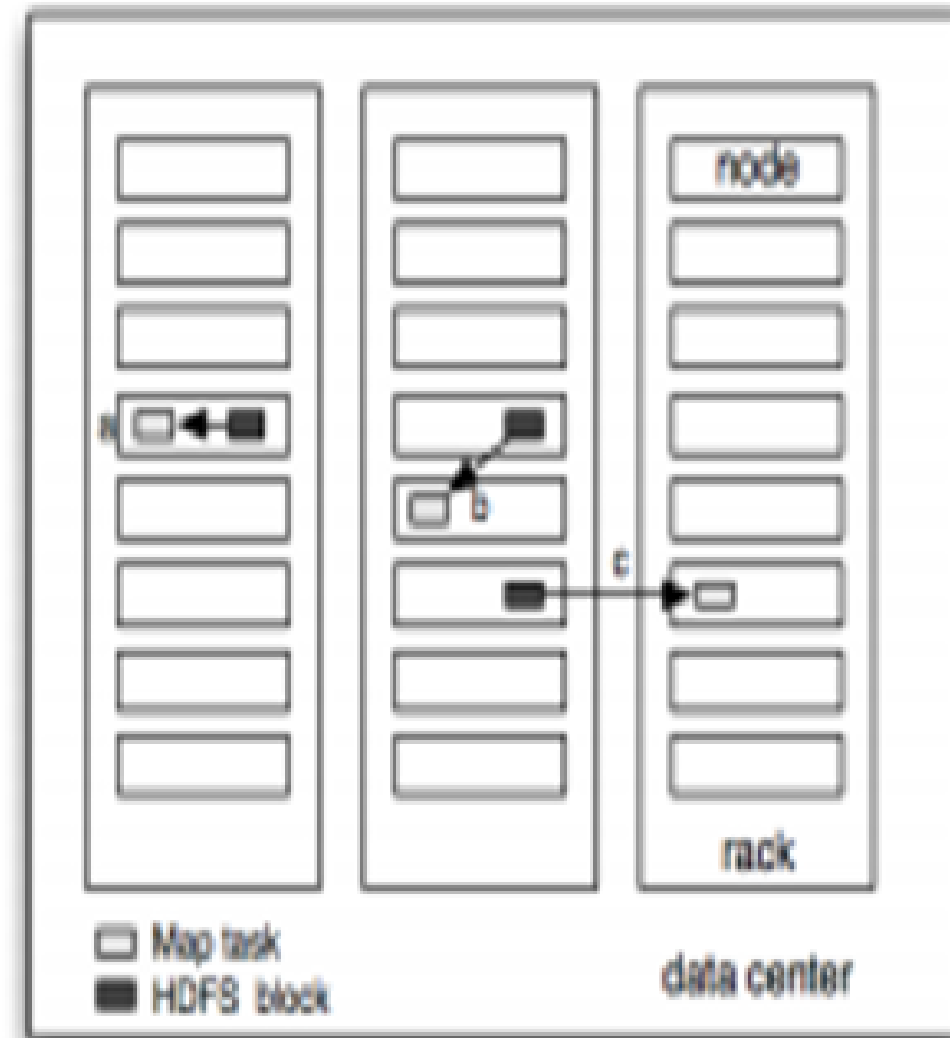

```
user@master:~$ yarn node -list
2024-08-13 11:22:13,976 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/10.10.3.111:8032
Total Nodes:5
  Node-Id          Node-State Node-Http-Address      Number-of-Running-Containers
worker-2:42905      RUNNING    worker-2:8042          0
worker-3:42529      RUNNING    worker-3:8042          13
worker-5:39031      RUNNING    worker-5:8042          10
worker-1:40041      RUNNING    worker-1:8042          0
worker-4:39113      RUNNING    worker-4:8042          4
```

```

user@master:~$ yarn node -list -showDetails
2024-08-13 11:55:11,619 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at master/10.10.3.111:8032
2024-08-13 11:55:11,963 INFO conf.Configuration: resource-types.xml not found
2024-08-13 11:55:11,963 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
Total Nodes:5
  Node-Id          Node-State Node-Http-Address      Number-of-Running-Containers
  worker-2:42905    RUNNING    worker-2:8042          7
Detailed Node Information :
  Configured Resources : <memory:50768, vCores:28>
  Allocated Resources : <memory:7168, vCores:7>
  Resource Utilization by Node : PMem:6677 MB, VMem:6677 MB, VCores:15.661447
  Resource Utilization by Containers : PMem:1927 MB, VMem:18685 MB, VCores:20.421
  Node-Labels :
  worker-3:42529    RUNNING    worker-3:8042          0
Detailed Node Information :
  Configured Resources : <memory:32768, vCores:28>
  Allocated Resources : <memory:0, vCores:0>
  Resource Utilization by Node : PMem:3499 MB, VMem:3499 MB, VCores:0.019993335
  Resource Utilization by Containers : PMem:0 MB, VMem:0 MB, VCores:0.0
  Node-Labels :
  worker-5:39031    RUNNING    worker-5:8042          10
Detailed Node Information :
  Configured Resources : <memory:32768, vCores:28>
  Allocated Resources : <memory:10240, vCores:10>
  Resource Utilization by Node : PMem:10725 MB, VMem:10725 MB, VCores:19.51
  Resource Utilization by Containers : PMem:1887 MB, VMem:26326 MB, VCores:27.631
  Node-Labels :
  worker-1:40041    RUNNING    worker-1:8042          0
Detailed Node Information :
  Configured Resources : <memory:32768, vCores:28>
  Allocated Resources : <memory:0, vCores:0>
  Resource Utilization by Node : PMem:3890 MB, VMem:3890 MB, VCores:0.013328891
  Resource Utilization by Containers : PMem:0 MB, VMem:0 MB, VCores:0.0
  Node-Labels :
  worker-4:39113    RUNNING    worker-4:8042          10
Detailed Node Information :
  Configured Resources : <memory:50768, vCores:28>
  Allocated Resources : <memory:11264, vCores:10>
  Resource Utilization by Node : PMem:12732 MB, VMem:12732 MB, VCores:18.516666
  Resource Utilization by Containers : PMem:2052 MB, VMem:26578 MB, VCores:27.332998

```

- Attempts to run the map task on a node where the input data resides in HDFS.
 - *data locality optimization* - it doesn't use valuable cluster bandwidth.
- What happens when all nodes hosting the block replicas are busy?
 - look for a free map slot on a node in the same rack as one of the blocks.
- Very occasionally even this is not possible, so an off-rack node is used, which results in an inter-rack network transfer.





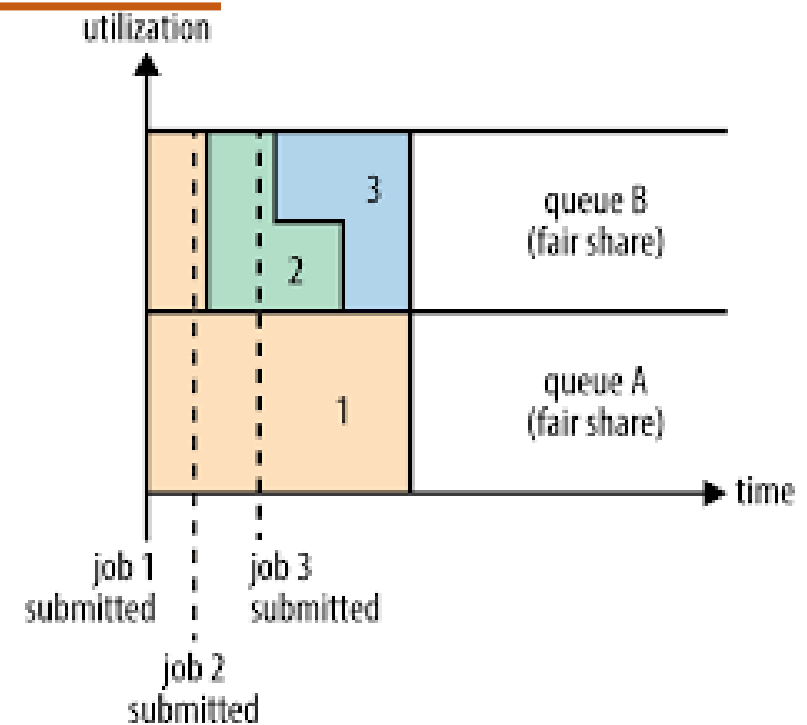
PES
UNIVERSITY

CELEBRATING 60 YEARS

Scheduling in YARN

- Early Hadoop versions, simplistic FIFO scheduler
 - In order of submission
 - each job would use the whole cluster
 - Resulting with jobs waiting for their turn.
- How to share resources fairly?
- Balance between
 - Production jobs
 - Ad-hoc jobs

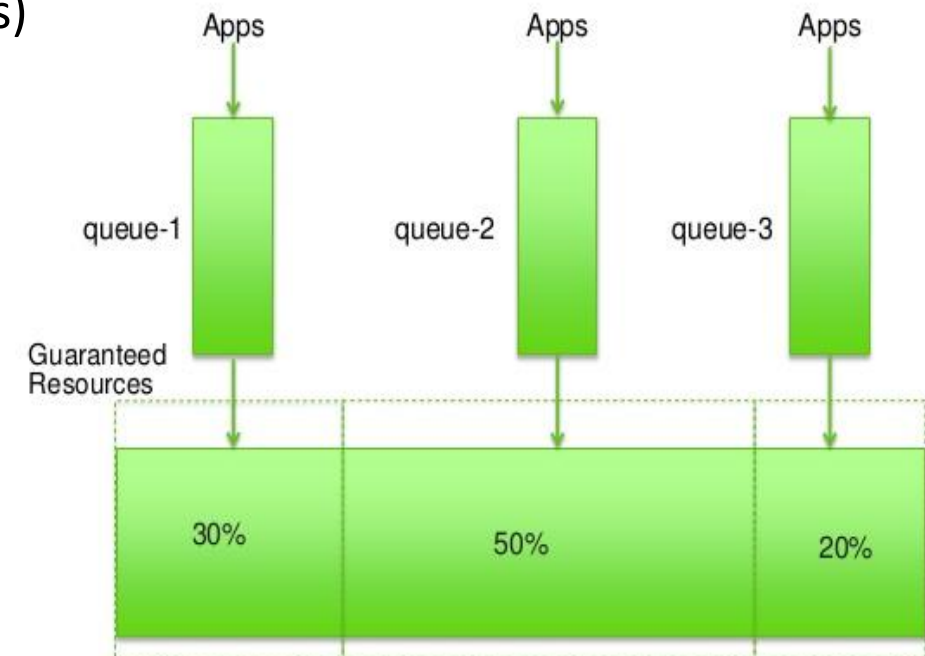
- Aims to give every user a fair share of the cluster capacity over time.
- Jobs are placed in pools,
 - Default each user gets their own pool.
- If a single job is running, it gets all of the cluster.
- As more jobs are submitted,
 - free task slots are given to the jobs in such a way as to give each user a fair share of the cluster.
- Short job – completes in reasonable time
- Long job – can continue making progress.



- Consider a user who submits more jobs
 - Scheduler ensures that user does not hog the cluster
- Custom pools
 - Guaranteed minimum capacities with map/reduce slots
 - It is also possible to define custom pools with guaranteed minimum capacities defined in terms of the number of map
- The Fair Scheduler supports preemption
 - If pool not received its fair share over certain time
 - scheduler will kill tasks in pools running over capacity

- Different approach
- number of queues (like the Fair Scheduler's pools),
 - Has an allocated capacity
 - Can be hierarchical
 - Within each queue scheduled using FIFO (with priorities)
- Cannot use free or spare capacity even if it exists
- Like breaking up cluster into smaller clusters

Capacity Scheduler



BIG DATA

Capacity Scheduler



Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources	Reserved Resources	Physical Mem Used %
3	0	1	2	26	<memory:27 GB, vCores:26>	<memory:195.16 GB, vCores:140>	<memory:0 B, vCores:0>	15

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes
5	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority	Scheduler Busy %	RM Dispatcher EventQueue Size	Scheduler C
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0	0	0	0

Dump scheduler logs 1 min

Show 100 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB	Allocated GPUs	Reserved CPU VCores	Reserved Memory MB	Reserved GPUs	% of Queue	% of Cluster	Progre
application_1722926003558_0004	user	streamjob16064623285584792112.jar	MAPREDUCE		root.default	0	Tue Aug 13 13:09:31 +0550 2024	Tue Aug 13 13:09:31 +0550 2024	N/A	RUNNING	UNDEFINED	26	26	27648	-1	0	0	-1	13.8	13.8	

Showing 1 to 1 of 1 entries

Aggregate scheduler counts

Total Container Allocations(count)	Total Container Releases(count)	Total Fulfilled Reservations(count)	Total Container Preemptions(c
83	57	0	0

Last scheduler run

Time	Allocations(count - resources)	Reservations(count - resources)	Releases(count - re
Tue Aug 13 13:10:26 +0530 2024	0 - <memory:0, vCores:0>	0 - <memory:0, vCores:0>	0 - <memory:0, vCores:0>

Last Preemption

Time	Container Id	Node Id	Queue
N/A	N/A	N/A	N/A

Last Reservation

Time	Container Id	Node Id	Queue
N/A	N/A	N/A	N/A

Last Allocation

Time	Container Id	Node Id	Queue
Tue Aug 13 13:09:38 +0530 2024	N/A	worker-1.40041	root.default

Last Release

Time	Container Id	Node Id	Queue
Tue Aug 13 13:10:22 +0530 2024	container_1722926003558_0004_01_000011	worker-5.39031	root.d



PES
UNIVERSITY

CELEBRATING 60 YEARS

Handling Failures

- Task
- Application Master
- Resource Manager
- Node Manager

Due to runtime exceptions

- JVM reports error back to parent application master

Hanging tasks

- Progress updates not happening for 10 mins
- Timeout value can be set.

Killed tasks

- Speculative duplicates can be killed

Recovery

- AM tries restarting task on a different node

When can failure occur?

- Due to hardware or network failures

How to detect for failures?

- AM sends periodic heartbeats to Resource Manager

Restart

- Max-attempts to restart application
- Default = 2

When can failure occur?

- Hardware, crashing, slow network

How to detect for failures?

- When a heartbeat is not received by RM for 10mins

Restart

- Tasks of incomplete jobs will be rerun – maybe on different node

How is failure handled?

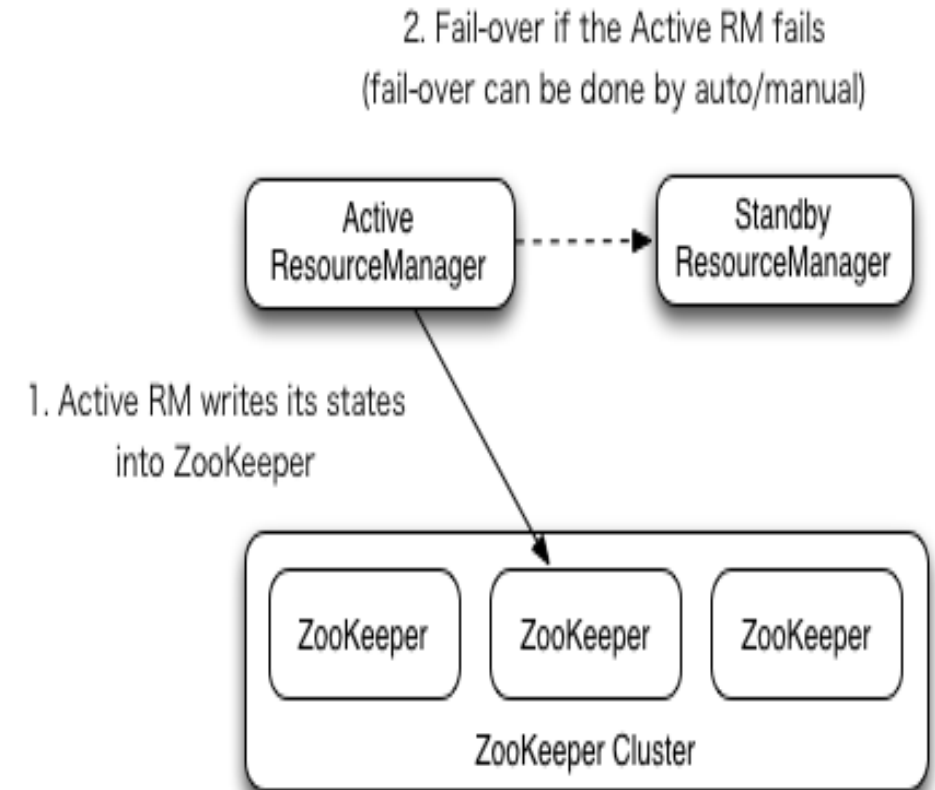
- Active Standby configuration

Impact

- More serious as all tasks fail

Restart

- Handled by failover controller





PES
UNIVERSITY

CELEBRATING 50 YEARS

Benefits of YARN

- YARN manages a very large cluster at Yahoo
 - Scalability – to over 40,000 servers with 100,000 CPUs, 455 PB of data
 - Runs over 850,000 jobs per day
 - Flexibility
 - Same cluster has Hadoop, Storm and Spark (100 node cluster) sharing resources using YARN

- A 1000 node YARN cluster has no jobs running. Two pools are configured with max of 50% of the resources. A new job requiring 600 nodes is submitted and on starting consumes all 600 nodes. Which YARN scheduler is active?
 - Either FIFO or Fair because they will use the entire cluster if there is no other job.
- Will the failure of task result in failure of the entire job?
 - No. Task will be restarted
- What are speculative duplicates?
 - Tasks that are started when AM determines that there is a slow running task.

Additional Notes, Reference Material and Notes

- Chapter 4 of T1
- <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>
- Also follow links in the slide deck



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Google Big Query

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Google Big Query- From data warehouse to autonomous data and AI platform

- BigQuery is the autonomous data to AI platform, automating the entire data life cycle, from ingestion to AI-driven insights, so you can go from data to AI to action faster.
- **Features:**
 - ❖ AI-powered conversational and agentic experience
 - ❖ Unified data and AI platform for analytics on multimodal data
 - ❖ Flexible with low cost ML and interoperability with open source

Some more features of Google BigQuery

1) **Connect your data to AI:**

Create and run ML models using familiar SQL right within BigQuery. Leverage built-in pre-trained models, Vertex AI-trained models, or your custom models, and register them in Vertex AI model registry. Use latest AI models to perform multimodal data analysis such as document processing, image annotation and audio transcription, generative AI tasks such as text summarization and sentiment analysis, and embeddings generation for vector search use cases such as anomaly detection and product recommendations.

2) **Built-in AI agents and workflow automation:**

Get AI-powered experiences and automation for your workflows with Gemini in BigQuery. Find, join, and query datasets and visualize results using natural language prompts in data canvas. Automate data preparation, error detection, and transformations. Automatically uncover queries from table metadata and get context-aware coding assistance. Save costs and optimize data infrastructure with intelligent recommendations for partitioning, clustering, and materialized views.

3) **Your choice of open source and open formats:**

Run serverless Spark alongside SQL workloads in BigQuery, with unified security, runtime metadata, and governance. BigQuery's fully managed capabilities combined with managed Apache Iceberg tables powered by BigLake enable streaming, advanced analytics, and AI use cases, making it easy to work with open formats.

Some more features of Google BigQuery

4) **Built-in data to AI governance:**

BigQuery provides contextual governance that is powered by Dataplex Universal Catalog. All the key capabilities such as automatic metadata harvesting, data profiling, data quality and lineage are integrated and are available in the BigQuery experience. Customers can use gen AI-powered capabilities such as semantic search, metadata augmentation, and data insights to discover, document and get faster insights for all your BigQuery assets.

5) **Built for enterprise scale and efficiency:**

BigQuery's unique architecture decouples storage and compute for petabyte-scale analysis while optimizing costs with compressed storage, compute autoscaling, flexible pricing, and more. Under the hood, BigQuery employs a vast set of Google infrastructure technologies like Borg, Colossus, Jupiter, and Dremel. For mission-critical workloads, BigQuery also provides managed disaster recovery in the case of a total region outage relying on cross-region dataset replication capabilities.

- **Borg** → Cluster management (compute).
- **Colossus** → Distributed storage (file system).
- **Jupiter** → Networking backbone (connectivity).
- **Dremel** → Data analysis engine (querying).

Uses of Google BigQuery



1)Unlock generative AI use cases with BigQuery and Gemini models:

Build data pipelines that blend structured data, unstructured data, and generative AI models together to create a new class of analytical applications. BigQuery integrates with the latest Gemini models through Vertex AI integrations, unlocking a wide range of tasks like text summarization and sentiment analysis using simple SQL



Uses of Google BigQuery

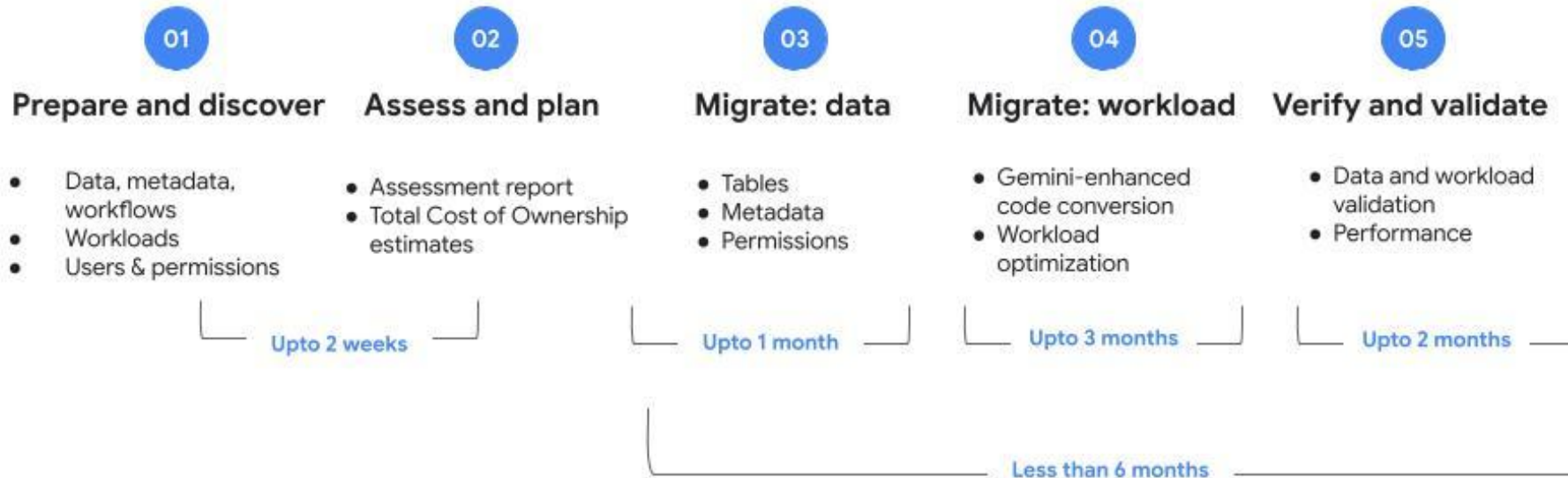


2) Migrate data warehouses to BigQuery:

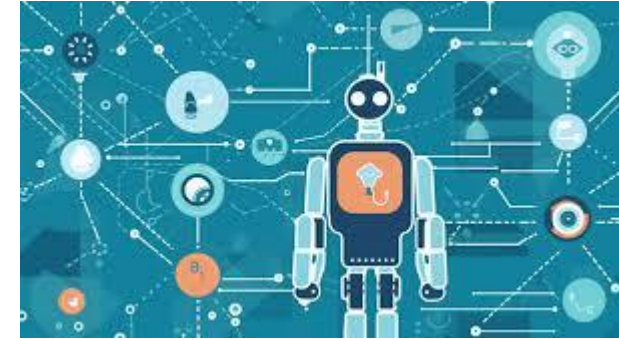
Solve for today's analytics demands and tomorrow's AI use cases by migrating your data warehouse to BigQuery. Streamline your migration path from Netezza, Oracle, Redshift, Teradata, Snowflake, or Databricks to BigQuery using the free and fully managed BigQuery Migration Service.



Migration automation via BigQuery Migration Service



Uses of Google BigQuery

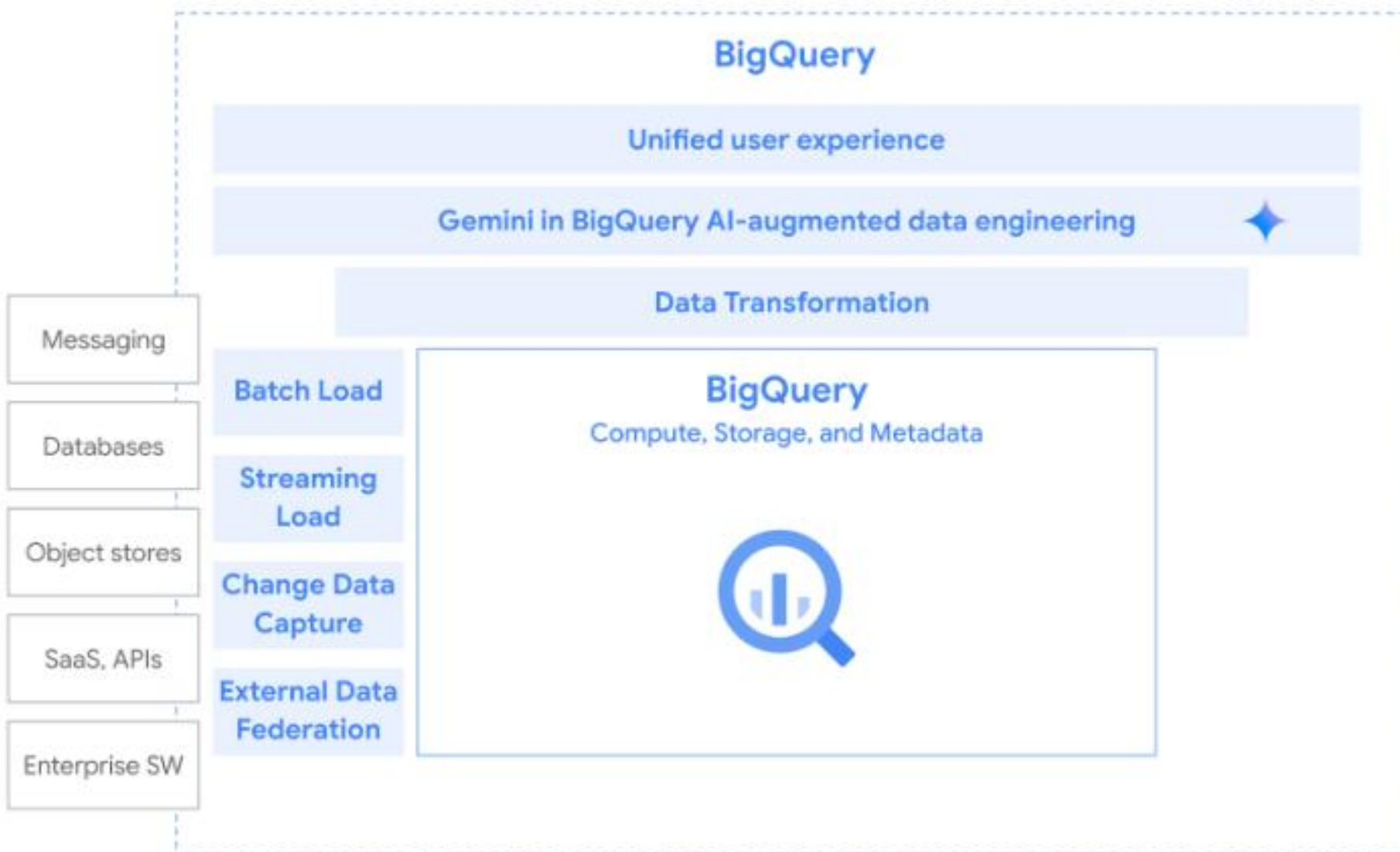


3) Data integration and ELT:

ELT is the recommended pattern for bringing data into BigQuery. There are many tools that offer flexibility for data integration. For batch load, use BigQuery Data Transfer Service (DTS) to automate the bulk load of data from supported data sources into BigQuery. For streaming load, Pub/Sub BigQuery subscriptions writes Pub/Sub messages to an existing BigQuery table as they are received. For Change data capture (CDC), Datastream enables non-intrusive change data capture (CDC) from databases into BigQuery. Finally, you can federate to a number of external data sources that don't require data movement.



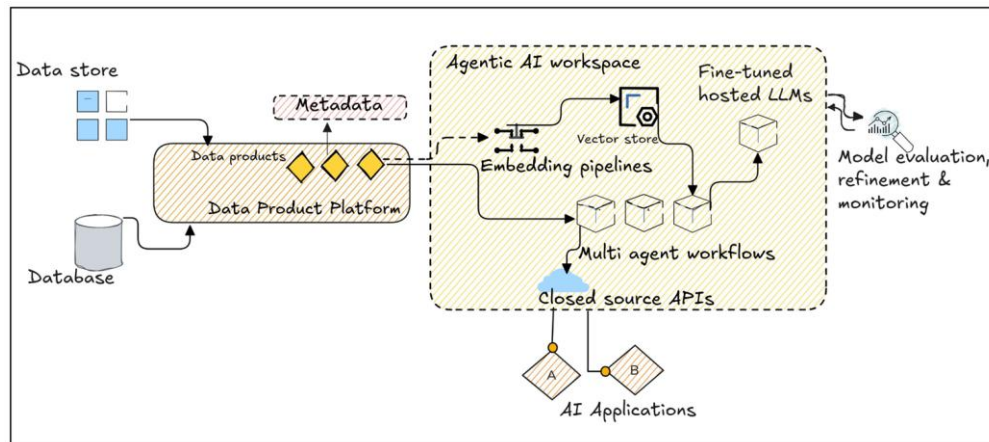
Uses of BigQuery



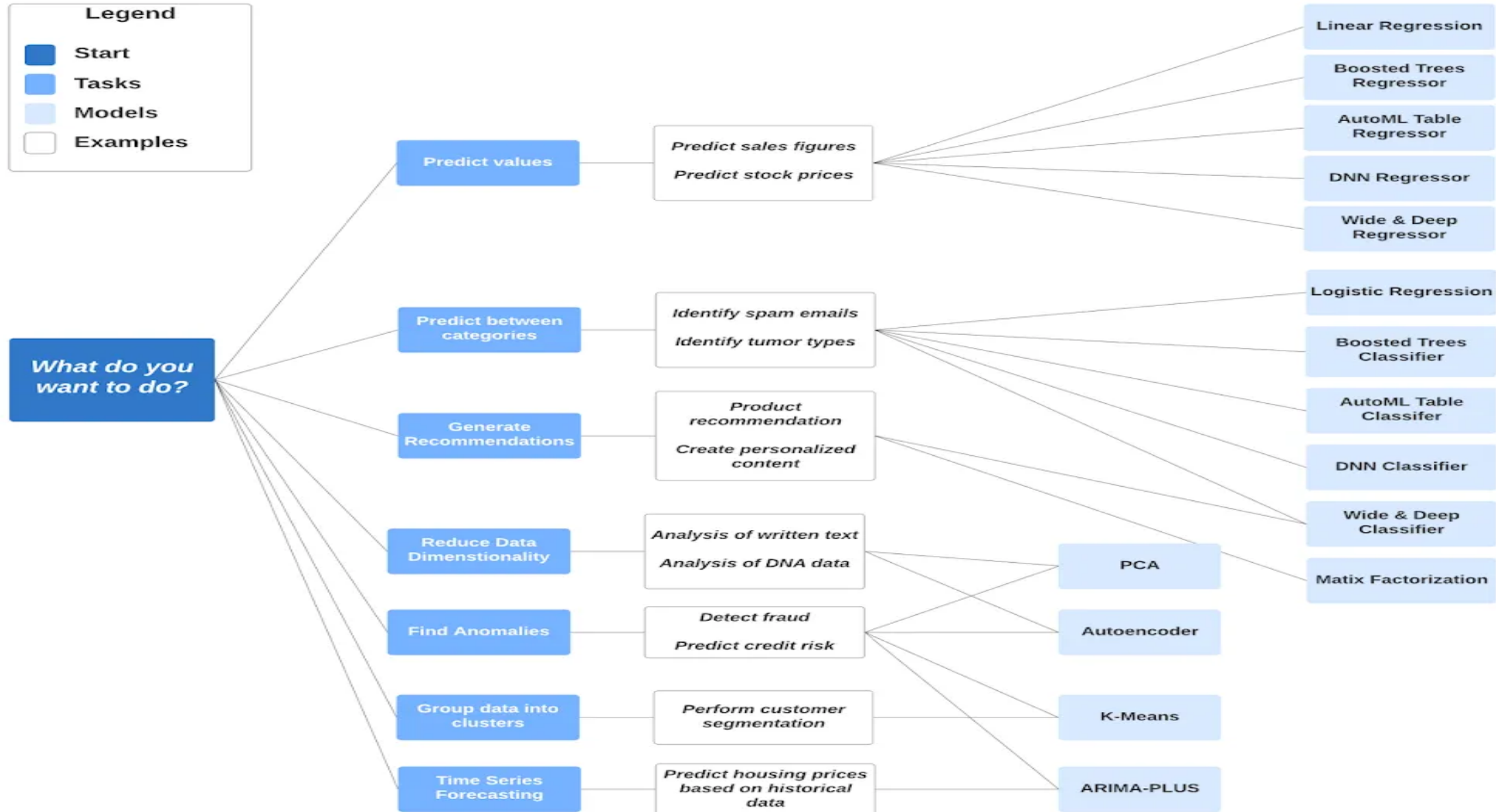


4) Simplify data to AI workflows

Harness the flexibility to use Colab Enterprise notebooks, open source Python libraries through BigQuery DataFrames, Jupyter notebooks, and programmatic analysis tools with BigQuery. Streamline complete machine learning flow for each model, such as feature preprocessing, model creation, hyperparameter tuning, inference, evaluation, and model export.



Uses of BigQuery



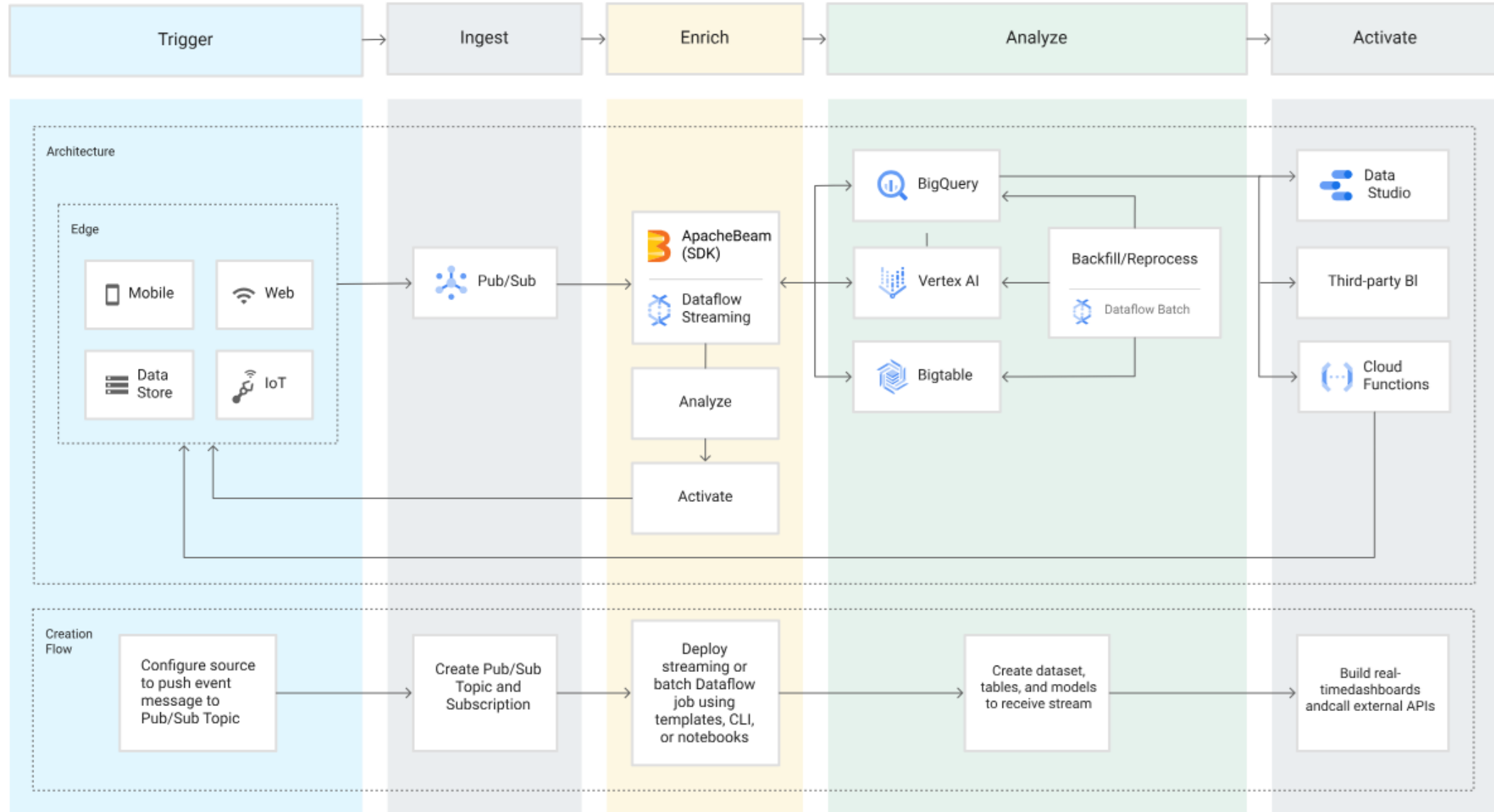


5)Real-time analytics

Gain a competitive advantage by responding to business events in real time with event-driven analysis. Built-in streaming capabilities automatically ingest streaming data and make it immediately available to query. This allows you to stay agile and make business decisions based on the freshest data. Or use Dataflow to enable fast, simplified streaming data pipelines for a comprehensive solution.



Uses of BigQuery



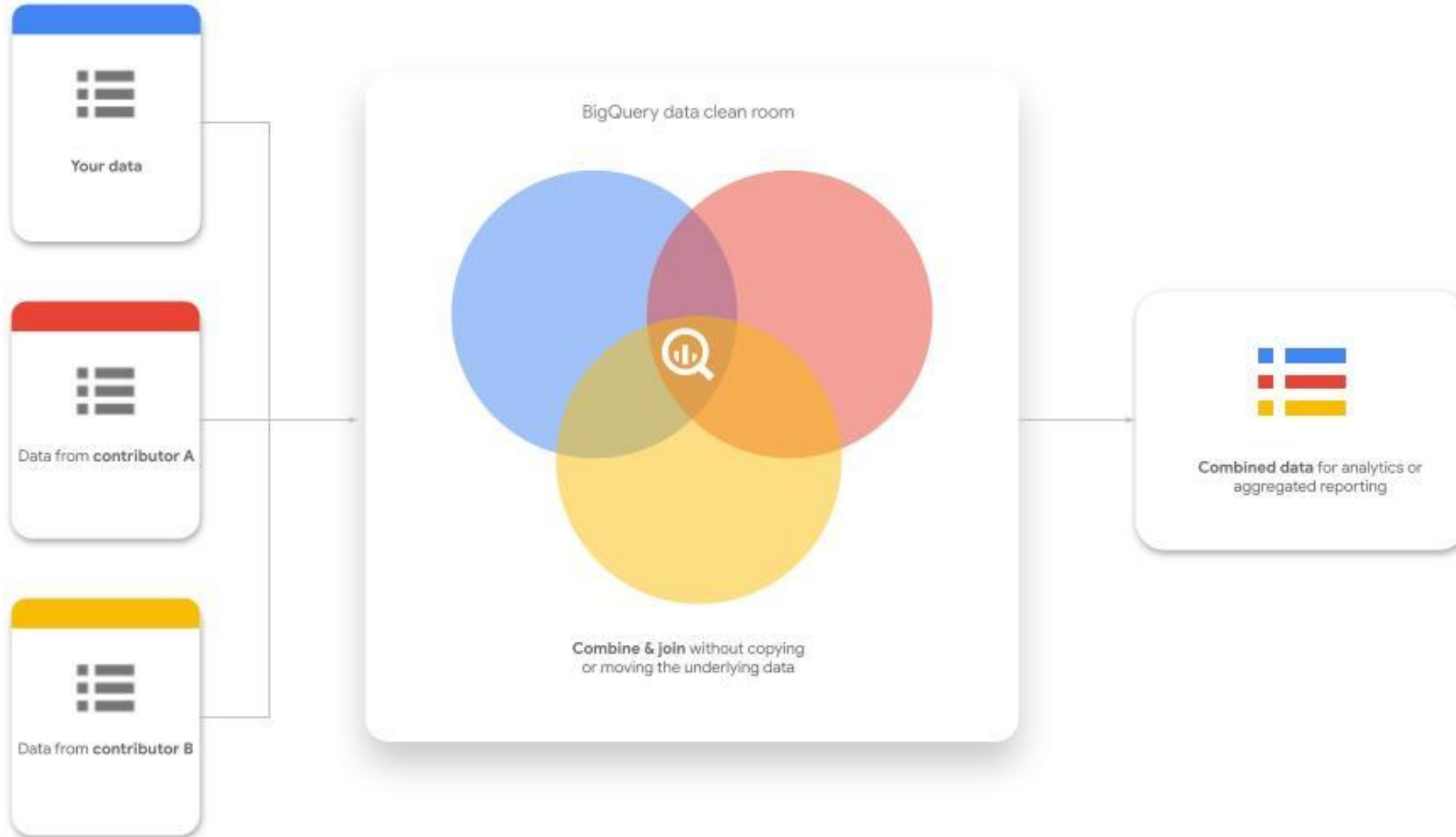


6) BigQuery data clean rooms for privacy-centric data sharing

Create a low-trust environment for you and your partners to collaborate without copying or moving the underlying data right within BigQuery. This allows you to perform privacy-enhancing transformations in BigQuery SQL interfaces and monitor usage to detect privacy threats on shared data. Benefit from BigQuery scale without needing to manage any infrastructure and built-in BI and AI/ML.



Uses of BigQuery



Uses of BigQuery



7) Geospatial analytics: Unlock planetary-scale insights with rich, easy-to-use geospatial datasets

Access a portfolio of rich geospatial data, powerful cloud computing, and built-in AI tools that make it easier for you to unlock insights that lead to more informed and faster business and sustainability decisions, without needing remote sensing or GIS expertise. Seamlessly integrate analysis-ready imagery and datasets from Earth Engine, and Places, Routes, Street View, and satellite data from Google Maps Platform into your existing BigQuery workflows, using data clean rooms.



Resource Link

For more information about Google BigQuery, you can refer to the [Official BigQuery Website](#)



THANK YOU

Prafullata Kiran Auradkar

Department of CSE
prafullatak@pes.edu



BIG DATA

Big Data Algorithms - From Matrix-Vector Multiplication to Page Rank

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

BIG DATA

From Big Data Infrastructure to Big Data Algorithms

What we have learnt so far:

HDFS

Store massive datasets across machines

MapReduce

Process those datasets in parallel

YARN

Manage resources and execution

Now, let's look at Big Data algorithms

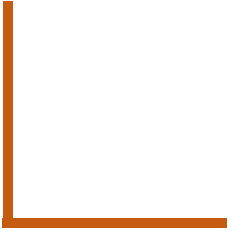
We have the infrastructure to **store and process large amounts of data**.

How can we use this infrastructure to solve one of the Big Data problems - Page Rank?

Before we look at the PageRank algorithm, we first need to understand **matrices and matrix-vector multiplication** in a Big Data setting.



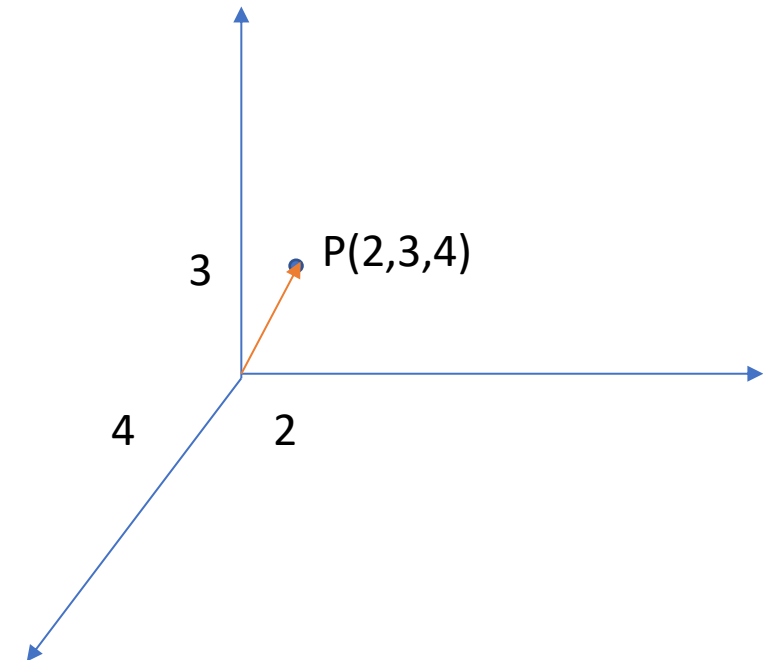
Matrices and Matrix Multiplication in Big Data



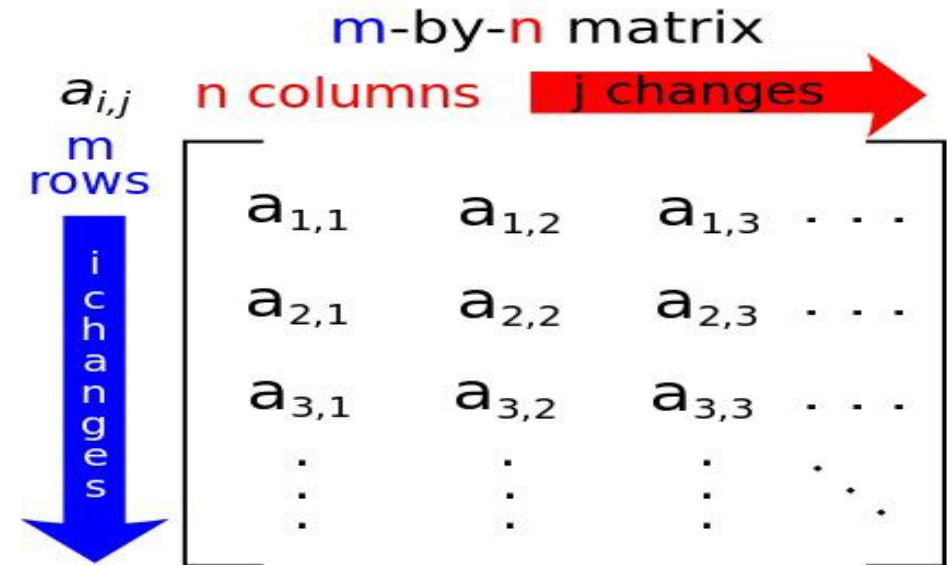
- Matrix Multiplication algorithms
 - Fundamental to many computations, including Page Rank
- Source
 - Leskovec, Jure, Anand Rajaraman, and Jeffrey David Ullman. *Mining of massive datasets*. Cambridge University Press, 2014.
 - <http://infolab.stanford.edu/~ullman/mmds/book.pdf>

Matrices and Vectors - Introduction

- Vectors
 - Can be defined as an ordered list of numbers
 - Visualization
 - An arrow where the direction of the vector is given by the relative size of the components
- Some Common Operations
 - Addition: $\mathbf{v} + \mathbf{w}$
 - Add components
 - Scalar multiplication $a\mathbf{v}$
 - Multiply each component by constant



- Rectangular array of numbers.
- The numbers are called the elements of the matrix.
- An $m \times n$ matrix has m rows and n columns
 - Can be considered as a collection of
 - m row vectors
 - n column vectors
 - An $n \times n$ matrix is called a square matrix.
- Vector can be considered as a
 - $1 \times n$ matrix (row matrix)
 - $n \times 1$ matrix (column matrix)



Each element of a matrix is often denoted by a variable with two **subscripts**. For example, $a_{2,1}$ represents the element at the second row and first column of a matrix **A**.

- Multiply each row vector of **A** by the corresponding elements of **x** and sum
- Multiplying a $m \times n$ matrix by an n element vector gives an m element vector

$$\begin{aligned}
 A\mathbf{x} &= \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \\
 &= \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{bmatrix}.
 \end{aligned}$$

$$A = \begin{bmatrix} 1 & 0 & -1 \\ 3 & 1 & 2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

2x3 multiplied by 3x1

$$\begin{aligned}
 A\mathbf{x} &= \begin{bmatrix} 1 & 0 & -1 \\ 3 & 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x - z \\ 3x + y + 2z \end{bmatrix} \\
 &= (x - z, 3x + y + 2z).
 \end{aligned}$$

- Typically, matrices are stored as multi-dimensional arrays in programs
- `int A[10][10]`
allocates 100 integers and is accessed as a 10x10 matrix

Space required to store the matrix = $10 \times 10 \times \text{sizeof}(\text{int}) = 100 \times \text{sizeof}(\text{int}) = 100 \times 4 = 400$ bytes.

In general, we need n^2 integers to store an ***nxn*** matrix

Matrix Representation of WWW

BIG DATA

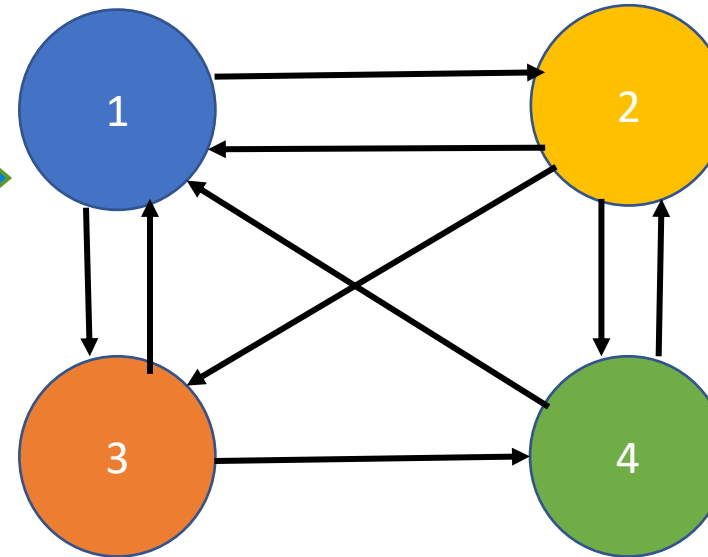
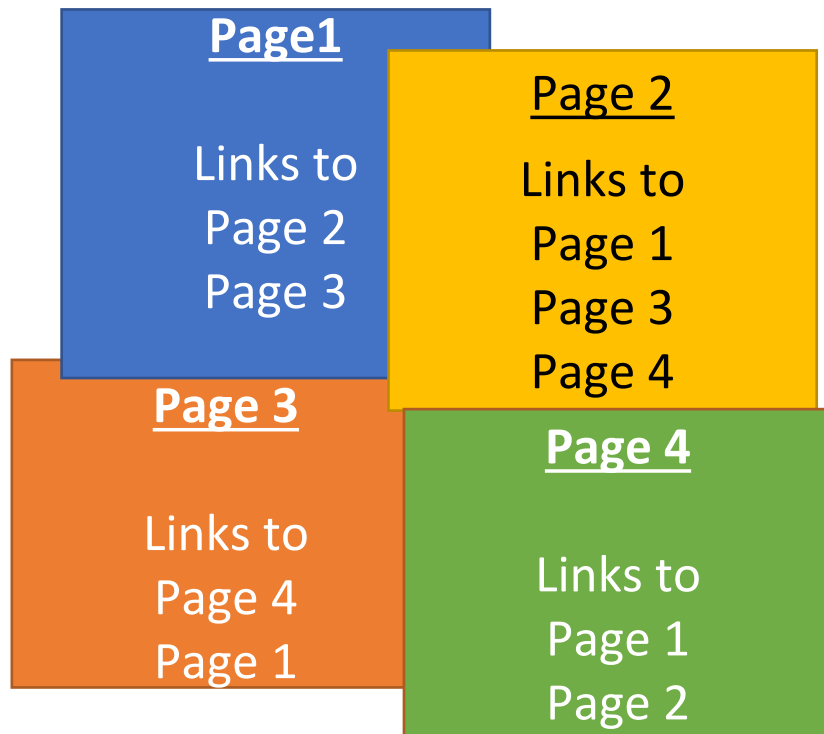
How do you represent the pages in WWW?

<u>Page1</u> Links to Page 2 Page 3	<u>Page 2</u> Links to Page 1 Page 3 Page 4
<u>Page 3</u> Links to Page 4 Page 1	<u>Page 4</u> Links to Page 1 Page 2

Consider a sample of the internet that contains 4 pages

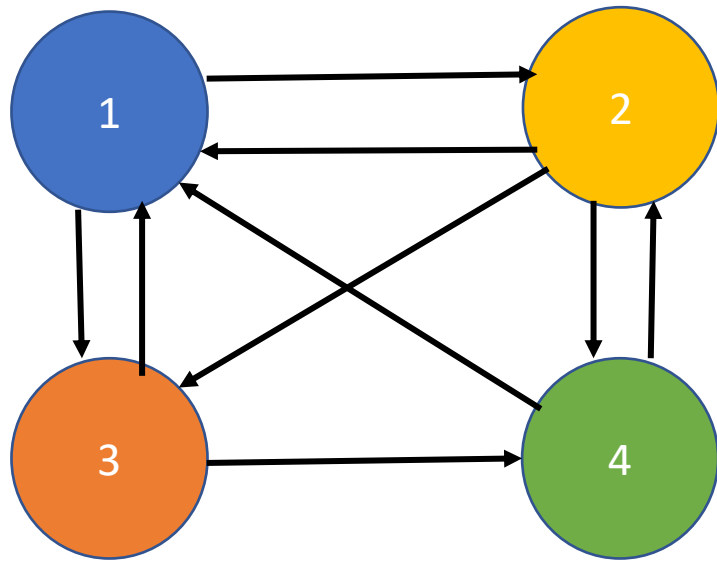
- How should we represent this?

Pages in the WWW

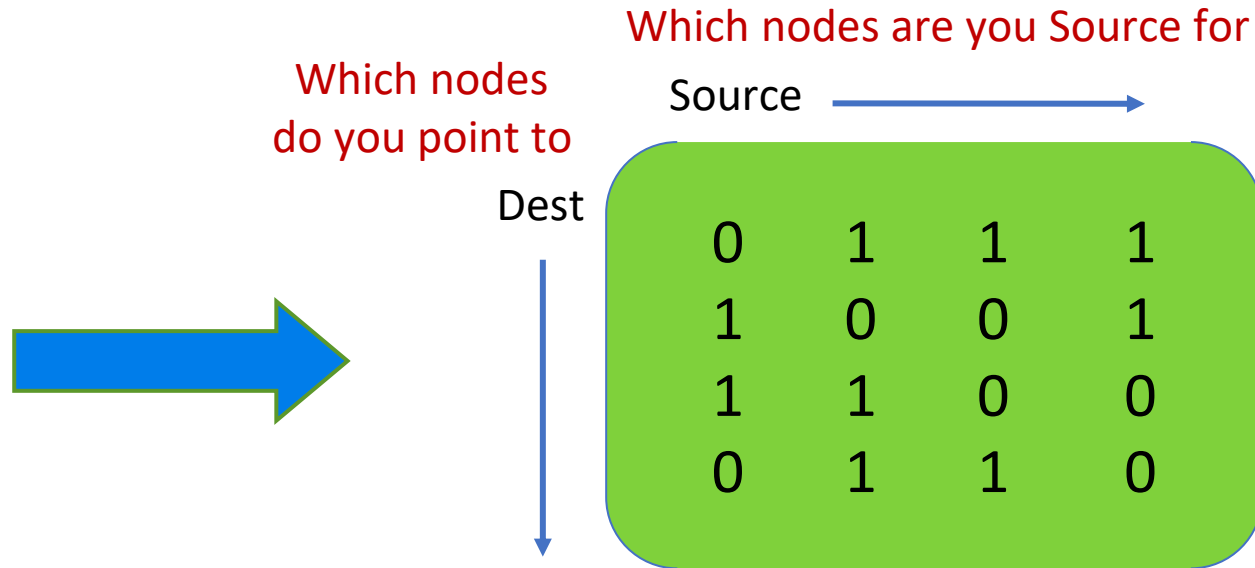


Represented as a directed graph

Pages in the WWW



Directed Graph



- This is fine for a small graph – 4 pages.
- But internet is large – billions of pages.
- This will lead to very large matrixes
- How much storage will we require?

Large scale matrix representations

BIG DATA

Representing the graph as an Adjacency Matrix



Directed Graph

- Internet is large – billions of pages.
- Note – In the above example significant number of the entries are 0 for the $N \times N$ adjacency matrix
- In the internet significant number of the entries in such an adjacency matrix will have values of 0 as not every page has a link to another page. There will be few links.
- This makes this adjacency matrix to be a *sparse* matrix.

In Big Data

- We deal with large matrices e.g. n will be the order of 10^{10} if n is number of web pages
 - Won't fit in the memory (DRAM)
 - And this will have to be stored in HDFS
 - We then bring it into memory for computation
 - Since it will be a sparse matrix, its logical that we want to store only the non-zero values or store values where there exists a link.
- How do we store this? There are number of ways in which Sparse matrix's can be represented to reduce the total memory requirement. E.g. Dictionary of Keys (DoK), Coordinate List (COO). All of these consider and represent the non-zero values present in the sparse matrix
- We consider non-zero values along with their row and column values

- Store only non-zero elements as a separate record in CSV format
- For each element we store
 - $\langle \text{row_number}, \text{column_number}, \text{value} \rangle$
 - As the format
- As many entries as there are links
- Exercise –Store the graph given on the right into a HDFS CSV file

		Column number			
		1	2	3	4
Row Number	1	0	1	1	1
	2	1	0	0	1
	3	1	1	0	0
	4	0	1	1	0

1, 2, 1
1, 3, 1
1, 4, 1
2, 1, 1
2, 4, 1
3, 1, 1
3, 2, 1
4, 2, 1
4, 3, 1

		Column number			
		1	2	3	4
Row Number	1	0	1	1	1
	2	1	0	0	1
	3	1	1	0	0
	4	0	1	1	0

As an exercise, try saving this in a file and loading it onto HDFS that you have installed.

Matrix Vector Multiplication

- To multiply an $m \times n$ matrix **MA** with an n -element vector **v**, compute

$$x_i = \sum_{j=1}^n ma_{ij} v_j$$

Check?

Where i is the row number in the range of 1 to total number of rows ' m ' in the matrix and j is the column number in the range of 1 to total number of columns ' n ' in the matrix

Eg. $x_1 = ma_{11}x_1 + ma_{12}x_2 + \dots + ma_{1n}x_n$

	Matrix MA	Vector v	Resultant Matrix x
$A \mathbf{x} =$	$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$	$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$	$= \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \end{bmatrix}$

- Let us assume that the **vector v fits into memory**
- Vector v is shared by all the mappers
- M_{ij} is stored as a CSV file on HDFS and is distributed across multiple nodes

$$x_i = \sum_{j=1}^n m_{ij} v_j$$

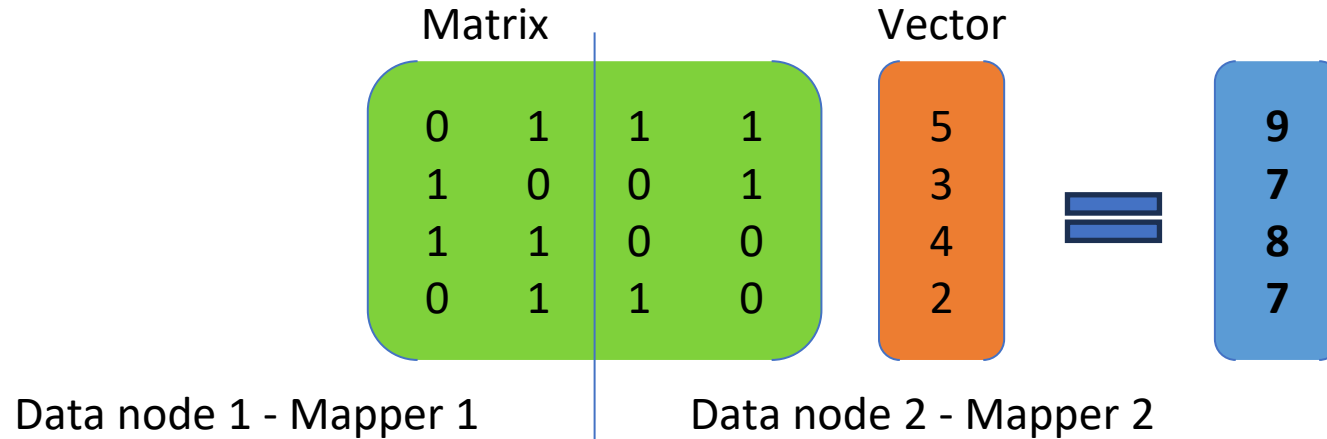
$$A\mathbf{x} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \end{bmatrix}$$

← 2

- **map:**
 - Computes the partial product
 - Uses the key as i the index into the target vector
 - output $(i, m_{ij}v_j)$
- **reduce:**
 - Sums all the partial products.

$$x_i = \sum_{j=1}^n m_{ij} v_j$$

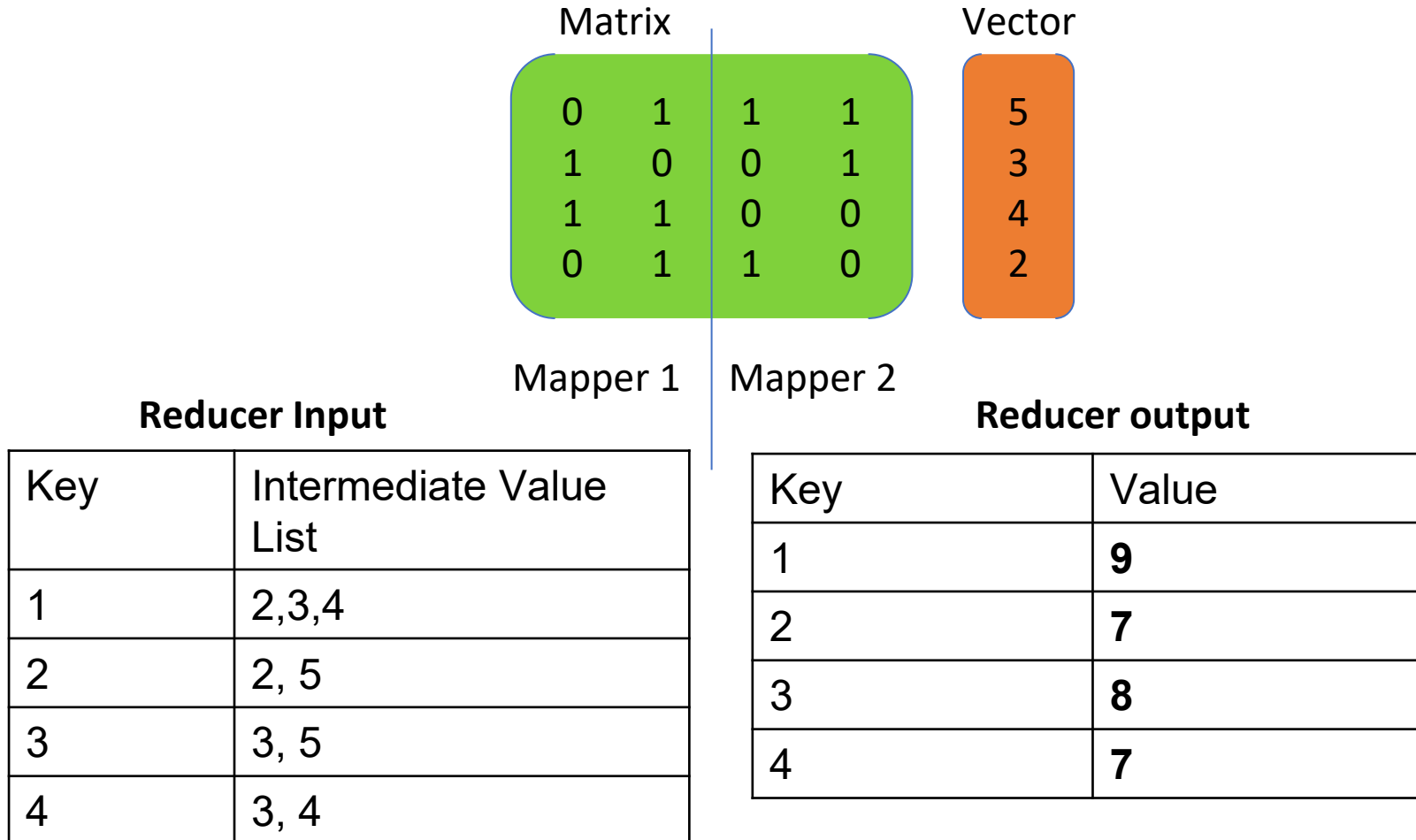
$$A\mathbf{x} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \end{bmatrix}$$



Key	Value
1	$1 \times 3 = 3$
2	$1 \times 5 = 5$
3	$1 \times 5 = 5$
3	$1 \times 3 = 3$
4	$1 \times 3 = 3$

Key	Value
1	$1 \times 4 = 4$
1	$1 \times 2 = 2$
2	$1 \times 2 = 2$
4	$1 \times 4 = 4$

Note that the key is the index into the vector where this value will contribute



Matrix

0	0	3	8
0	9	5	0
0	10	0	0
5	0	0	1

Vector

1
2
3
4

- Assuming rows 1 and 3 are in datanode1 and Rows 2 and 4 are in datanode 2, perform a matrix multiplication using Map Reduce
- Show inputs/outputs of mappers/reducers as discussed in previous slides

Matrix Vector Multiplication - extensions

- Case 2: $\mathbf{v}$ doesn't fit into main memory.
- Partition $\mathbf{M}$ and $\mathbf{v}$ into stripes.

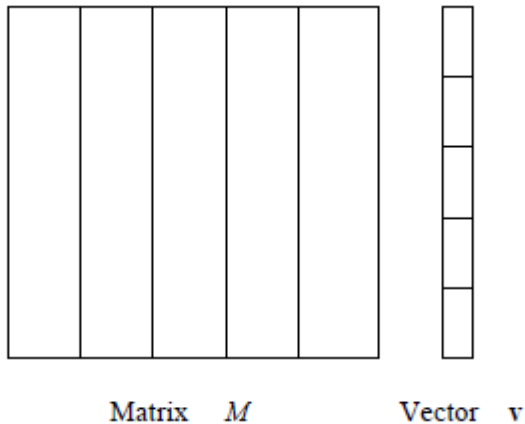


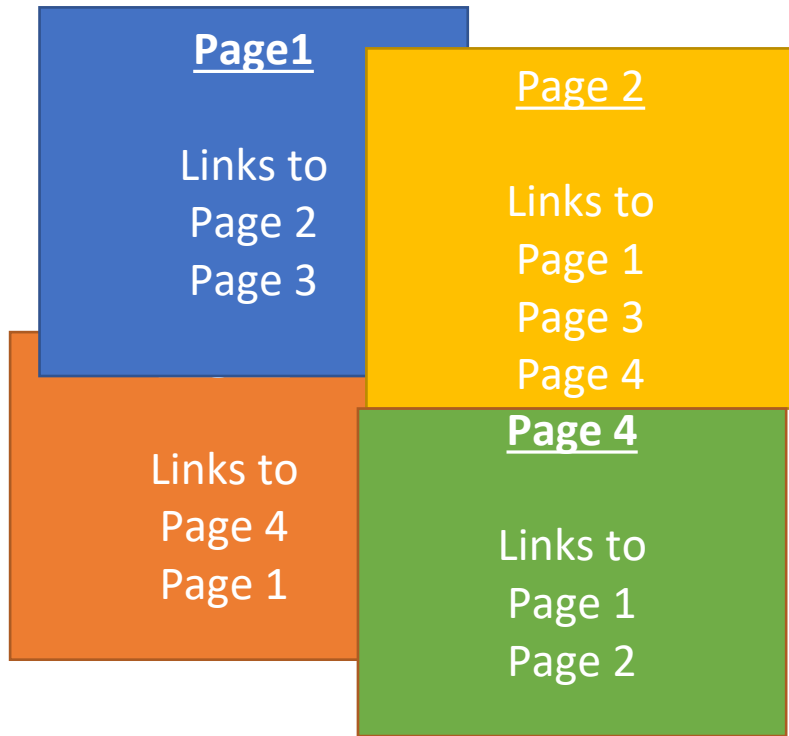
Figure 2.4: Division of a matrix and vector into five stripes

- The same MapReduce algorithm can be used.

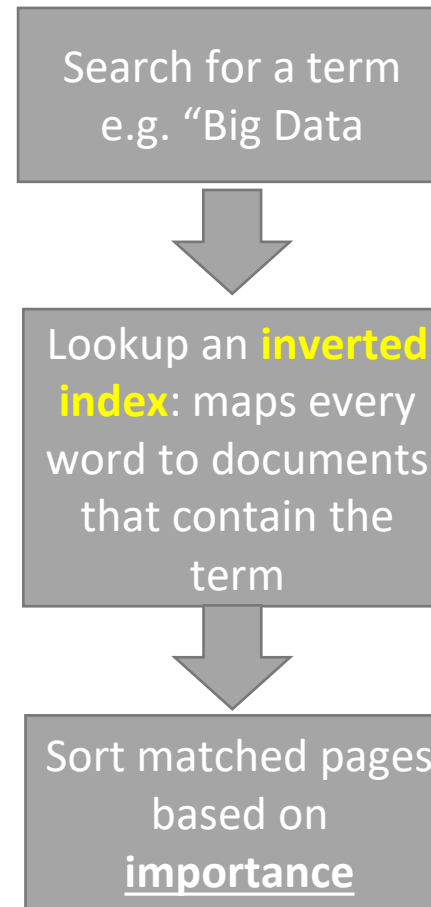
Page Rank

- Page Rank
- Source
 - Leskovec, Jure, Anand Rajaraman, and Jeffrey David Ullman. *Mining of massive datasets*. Cambridge University Press, 2014.
 - <http://infolab.stanford.edu/~ullman/mmds/book.pdf>

Page Rank : Overview



Pages in the WWW



How do we Compute the Importance.
It could be computation of frequency and usage of the term within the pages

Documents

We study the complexity of influencing elections through bribery: How computationally complex is it for an external actor to determine whether by a certain amount of bribing voters a specified candidate can be made the election's winner? We study this problem for election systems as varied as scoring ...

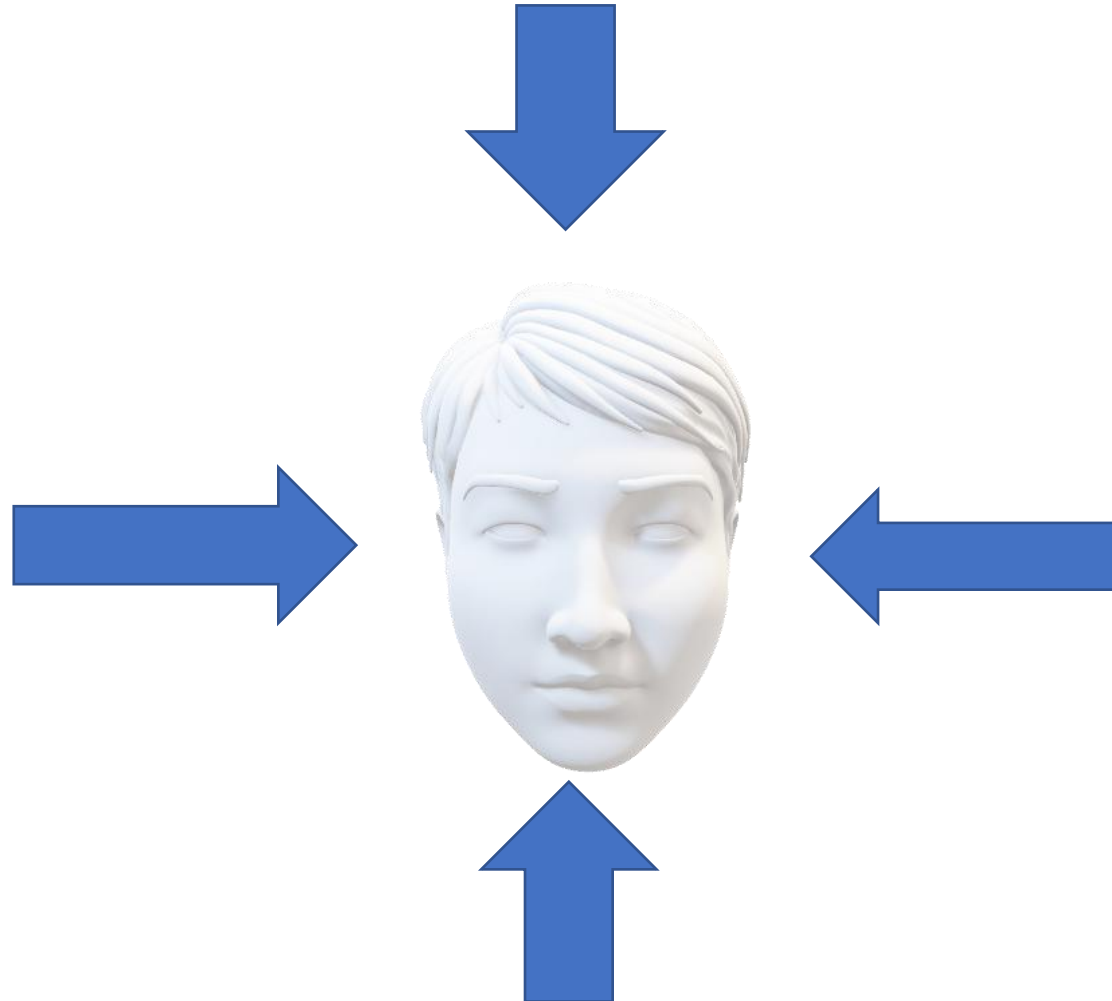
Vector-space representation

	D1	D2	D3	D4	D5
complexity	2		3	2	3
algorithm	3			4	4
entropy	1			2	
traffic		2	3		
network		1	4		

Term-document matrix

- Term Spam
 - Unethical use ☐ add terms multiple times
- Example
 - I could setup a page with PES University occurring a 1000 times in the page.
 - Search Engine would think that my page is an authority for PES University ☐ mark it as *important*
 - How to fix this?



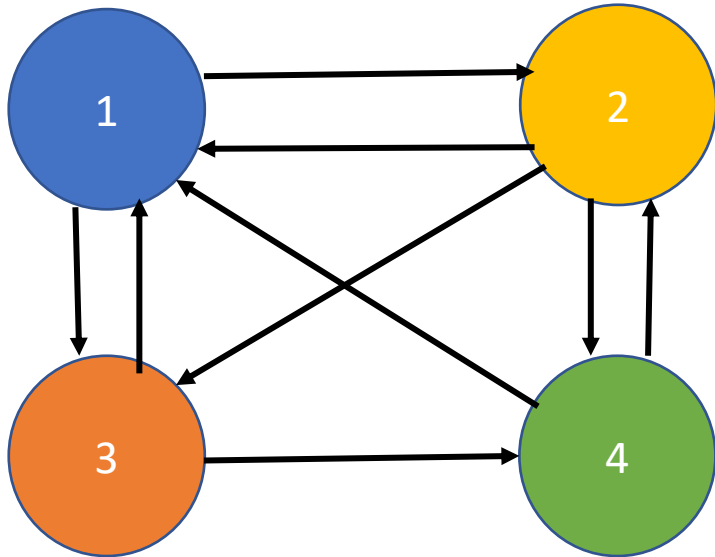


While searching for the term
how
Many are clicking on me??

- Instead of taking terms in my page
- Consider how many pages are pointing into my page?
- Will this not solve the problem.
- NO!!
- Can add many spam pages that point to my page.

- Two techniques
 1. Consider random surfers starting at any of the random pages
 - What fraction of surfers end up at my page?
 - This gives an indication of the importance of my page.
 - Page Rank
 2. Take into account the terms near the links present in pages that point to my page
 - Gives an indication of how relevant my page is.

Page Rank : Transition Matrix



Which nodes do you point to

Which nodes are you Source for

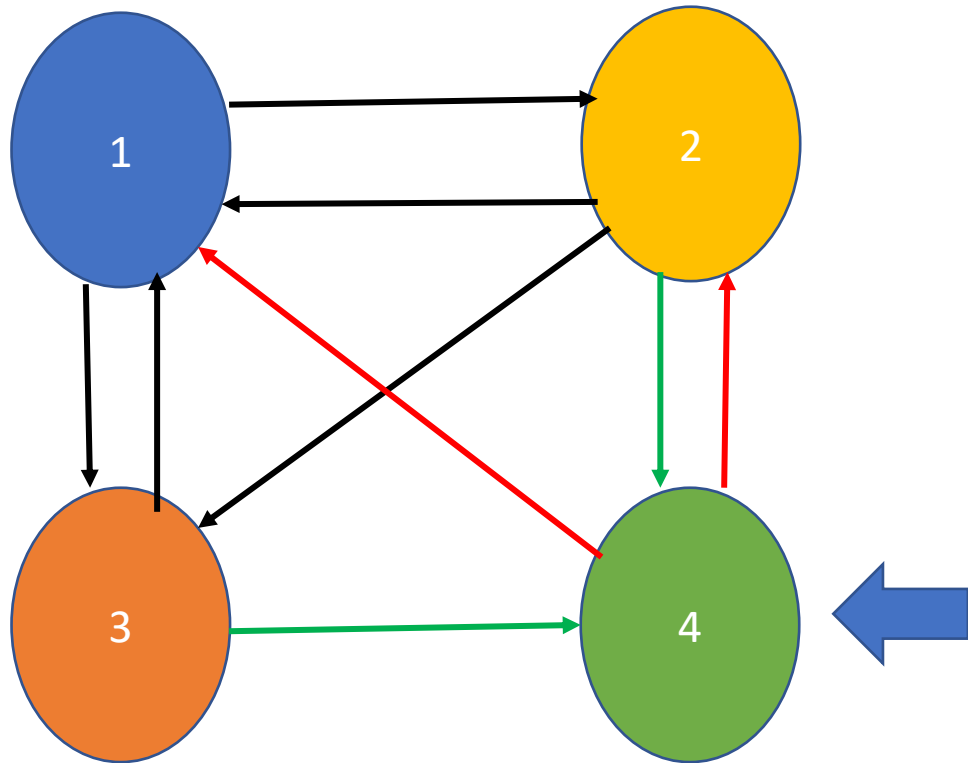
Source nodes/pages →

Dest ↓

	1	2	3	4
1	0	1	1	1
2	1	0	0	1
3	1	1	0	0
4	0	1	1	0

Let us start from our graph model of the internet

- Adjacency matrix represents which pages are reachable from a given page



If we are at node 4 (page 4) and we follow a link out
Then

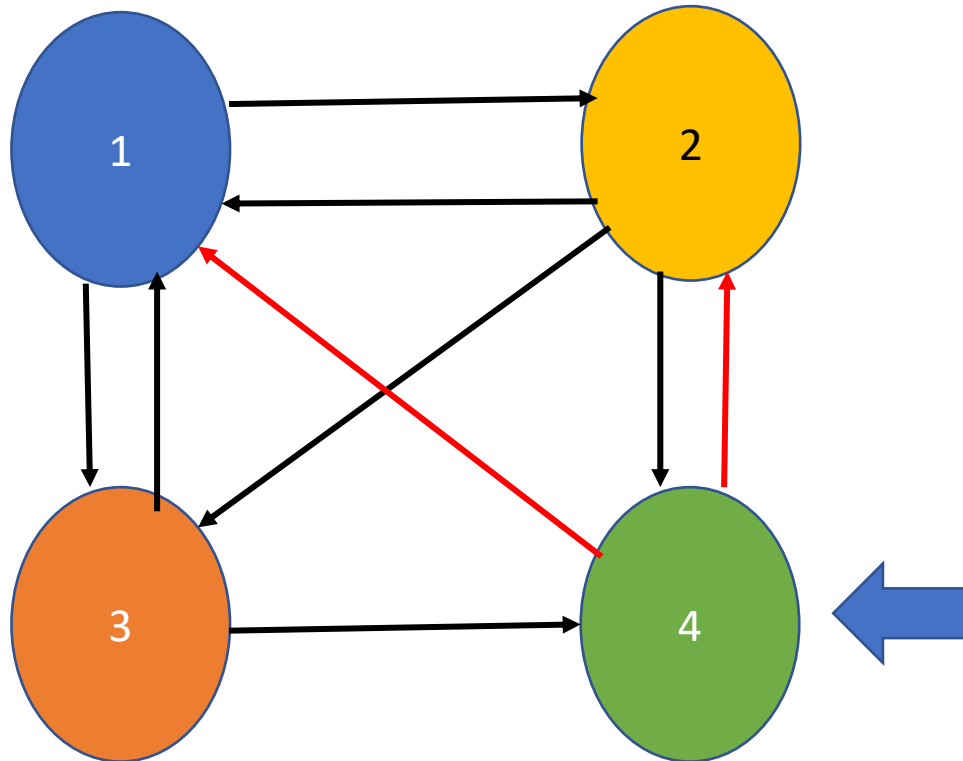
- the pages we can visit are 1 and 2
- 3 is not reachable directly from 4

To summarize for node 4

#in links = 2

#out links = 2

Probability of a Random surfer going to a web link/Node



If we have n links from any page (represented as node) and the user can go to any of the links, then we can assume to have $1/n$ probability of taking any of the links.

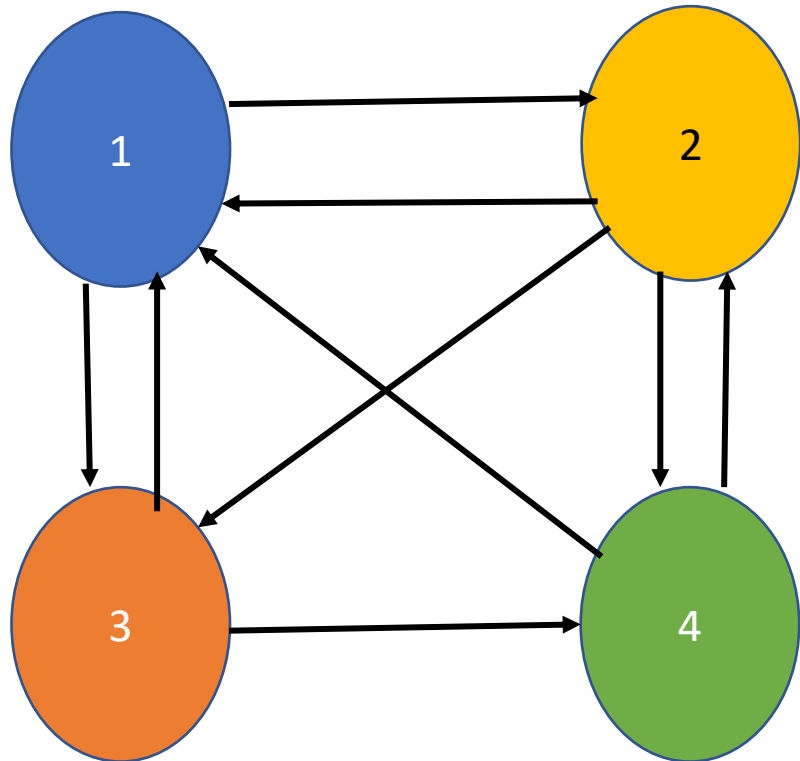
Consider, a random surfer starting from page 4

Assume, that we have equal probability of taking either of the out link

So,

$$P_{(\text{transition at node 4})} = 1/\text{\#outlinks} \\ = \frac{1}{2}$$

Direct transition probability of the random Surfer to any other page at a node

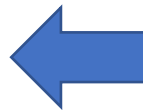


For any node we can represent the probabilities of directly transitioning

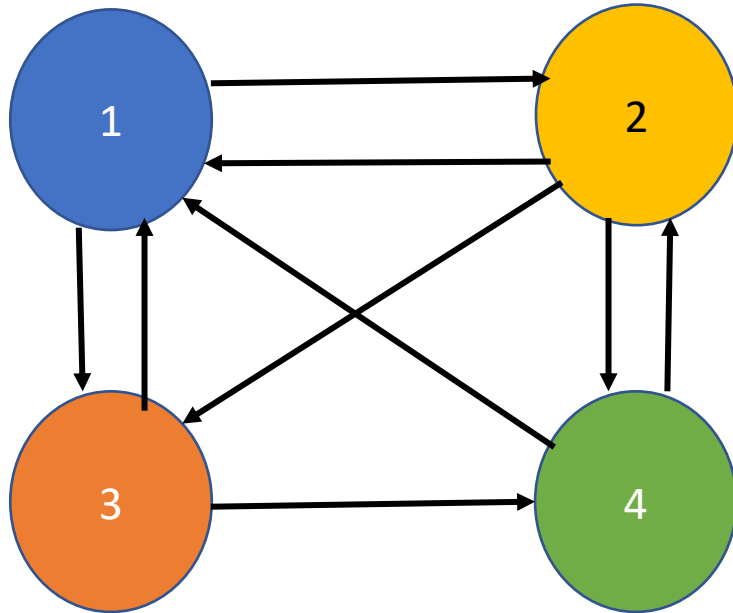
- To every other page as a *column vector*

Source	
Dest	
1	1/2
2	1/2
3	0
4	0

Example for node 4



Transition Matrix with the probability of the random Surfer going to any link/node



For the entire graph with n nodes, this would be a $n \times n$ matrix

- Each entry represents the probability of transition from a source to a destination
- Source column #
- Destination row #

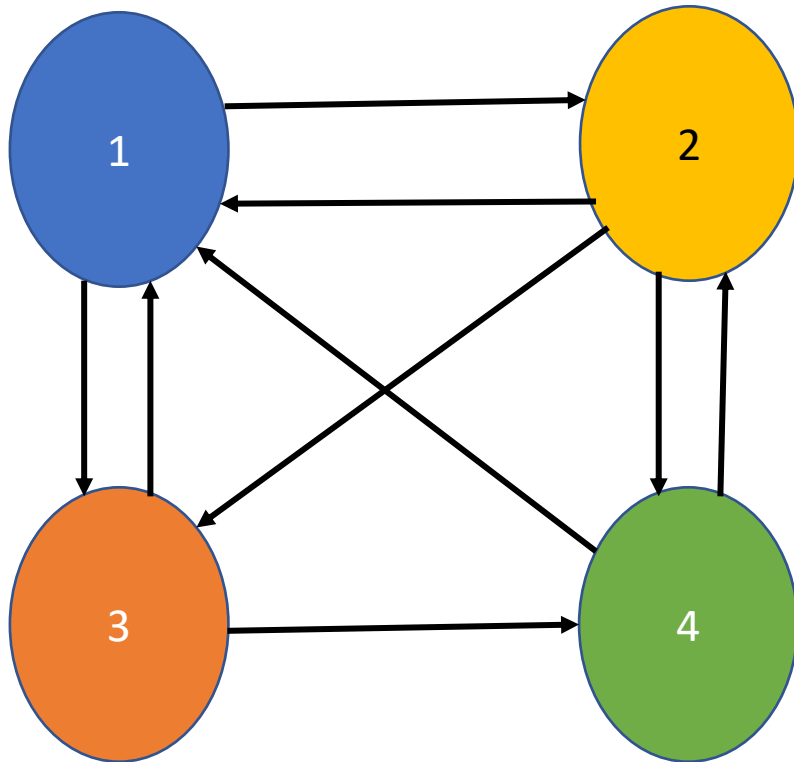
Transition matrix (M)

Source nodes/pages →

	1	2	3	4
1	0	1/3	1/2	1/2
2	1/2	0	0	1/2
3	1/2	1/3	0	0
4	0	1/3	1/2	0

Dest ↓

Page Rank : Following the random surfer



The random surfer can start off on any of the nodes

So each node has equal chance of being a starting node.

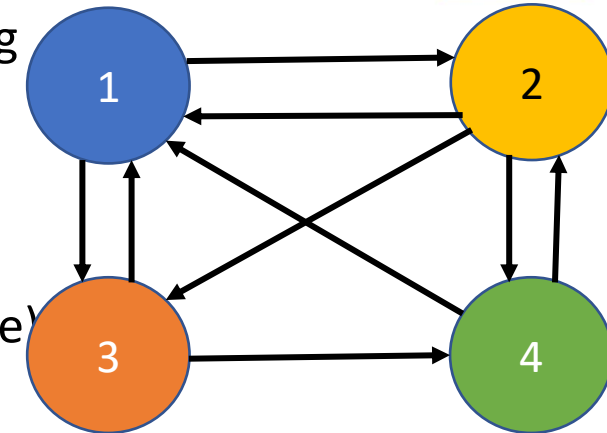
Lets call the relative importance of each node - ***importance*** represented as vector $\mathbf{v}$

Our first guess at this is called $\mathbf{v}_0$

$\mathbf{v}_0 =$

1/4
1/4
1/4
1/4

Node importance



As the random surfer moves once, we compute the chance of the random Surfer landing up in each of the link/nodes based on previous node.

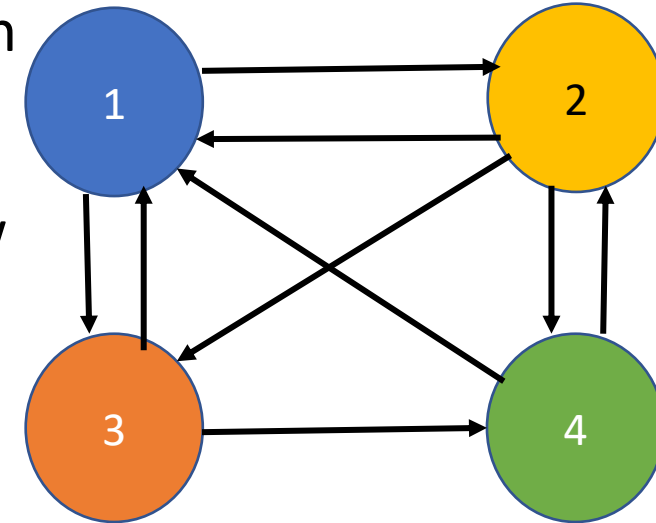
- We consider the generic probability transition matrix for this scenario, matrix **M** as derived earlier.
- We start with the initial probability of the choice of a page (indicating the importance) to be $\frac{1}{4}$ which is represented as discussed earlier for this scenario as vector **v₀**
- For each node we compute the probability of ending up at that node based on previous node (represented in the transition matrix **M**) and the weightage or importance associated which is vector **v** by Multiplying transition matrix **M** and **v** (So initially **M** and **v₀**)
- Illustrating lets start by looking at the probability of the surfer landing into (or destination) being Node 1 ->

	M	v₀	=	v₁	=	v₁	
1	0	1/4		$\frac{1}{3} \cdot \frac{1}{4} + \frac{1}{2} \cdot \frac{1}{4} + \frac{1}{2} \cdot \frac{1}{4}$		$\frac{1}{3}$	Probability of ending up at each of the nodes 1 to 4
2	1/2	1/4		$\frac{1}{2} \cdot \frac{1}{4} + \frac{1}{2} \cdot \frac{1}{4}$		$\frac{1}{4}$	
3	1/2	1/4		$\frac{1}{2} \cdot \frac{1}{4} + \frac{1}{3} \cdot \frac{1}{4}$		$\frac{5}{24}$	
4	0	1/4		$\frac{1}{3} \cdot \frac{1}{4} + \frac{1}{2} \cdot \frac{1}{4}$		$\frac{5}{24}$	

With each move of the random surfer, we will repeat the computation and get say $\mathbf{v}_2$

Multiplying the generic transition matrix $\mathbf{M}$ and the new $\mathbf{v}$ (or the new computed importance vector indicating the weightage/chance of a node in this case $\mathbf{v}_1$) will give an indication of what will happen when the surfer moves once again.

For every movement the surfer makes, from $\mathbf{v}_i$ we get $\mathbf{v}_{i+1}$ we can tell after n moves, which page the user lands up and that indicates how important that page is.



$$\begin{matrix}
 & \mathbf{M} & & \mathbf{v}_1 & & \\
 \begin{pmatrix}
 0 & 1/3 & 1/2 & 1/2 \\
 1/2 & 0 & 0 & 1/2 \\
 1/2 & 1/3 & 0 & 0 \\
 0 & 1/3 & 1/2 & 0
 \end{pmatrix}
 & &
 \begin{pmatrix}
 1/3 \\
 1/4 \\
 5/24 \\
 5/24
 \end{pmatrix}
 & = &
 \mathbf{v}_2
 \end{matrix}$$

Page Rank : As an Eigen vector problem

- Suppose we want to find x satisfies the equation (or set of equations) to the right.
- How do we solve for x ?
- We use **Iterative Algorithm** (widely used in Big Data). Here:
 - i = number of times we have looped,
 - x_i = value of x on i^{th} iteration,
 - At each iteration:
 1. Calculate an “error term” based upon x_i .
 - a. Shows how far x_i is from the correct value.
E.g., $Ax_i - x_i$
 1. Derive x_{i+1} based upon x_i and error term.
 - Repeat until error term is small.
 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots$
which eventually converges to a good solution.

$$A \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

- A common form of the iterative problem is:

$$x = f(x)$$

- Here, the solution x is a **fixed point** of the function f : applying f to x gives x back. x may be a matrix or vector. We can find this solution using **Fixed point iteration**:
 - Initialize x_0 to some value
 - Calculate $x_{i+1} = f(x_i)$
 - Loop over step 2 until either
 - error term = $|x_{i+1} - x_i|$ is small i.e., not much change between x_{i+1} and x_i
 - or a maximum number of iterations is reached.

$$x_0 \rightarrow f(x_0) \rightarrow f(x_1) \rightarrow f(x_2) \rightarrow \dots$$

- We have already seen that PageRank repeatedly computes:

$$v_{i+1} = M * v_i$$

- Compare this with the general fixed-point iteration:

$$x_{i+1} = f(x_i)$$

- For PageRank:

$$f(v) = Mv ; f \text{ is a matrix multiplication operation}$$

where:

M = transition matrix, v_i = PageRank vector at iteration i

Page Rank as Eigenvector Problem – Eigenvector Computation

So we start with an initial PageRank vector v_0 and repeatedly apply the matrix multiplication:

$v_0 \rightarrow Mv_0 \rightarrow Mv_1 \rightarrow Mv_2 \rightarrow \dots$ until the values change very little.

At convergence:

$$v_{i+1} \approx v_i$$

Therefore:

$$M \cdot v = v$$

Thus:

$$M \cdot v = 1 \cdot v$$

Therefore:

- v is an **eigenvector** of M
- corresponding **eigenvalue**: $\lambda = 1$
- the values in v are the **PageRanks**

Recall: Eigenvector Equation

For a square matrix A : $A \cdot v = \lambda \cdot v$

- v is the eigenvector
- λ is the corresponding eigenvalue
- Av is the same vector as v , scaled by λ

- We need support for
 - Handling multiple files as input to the mapper
 - The initial page rank
 - Part of the Transition Matrix
 - Iteration over multiple matrix-vector multiplication rounds

BIG DATA

Hadoop Multiple Input Files – mapper

```
public class multiInputFile extends Configured implements Tool
{

    public static class CounterMapper extends Mapper
    {
        public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException
        {
            String[] line=value.toString().split("\t");

            context.write(new Text(line[0]), new Text(line[1]));
        }
    }

    public static class CountertwoMapper extends Mapper
    {
        public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException
        {
            String[] line=value.toString().split("\t");
            context.write(new Text(line[0]), new Text(line[1]));
        }
    }
}
```

Mappers
write first
word as key,
2nd word as
value

```
public static class CounterReducer extends Reducer
{
    String line=null;

    public void reduce(Text key, Iterable values, Context context )
    throws IOException, InterruptedException
    {
        for(Text value:values)
        {
            line = value.toString();

            context.write(key, new Text(line));
        }
    }
}
```

Loop over values
Write each value as
separate record

```
public int run(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = new Job(conf, "aggprog");
    job.setJarByClass(multiInputFile.class);
    MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class, CounterMapper.class);
    MultipleInputs.addInputPath(job, new Path(args[1]), TextInputFormat.class, CountertwoMapper.class);

    FileOutputFormat.setOutputPath(job, new Path(args[2]));
    job.setReducerClass(CounterReducer.class);
    job.setNumReduceTasks(1);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    return (job.waitForCompletion(true) ? 0 : 1);
}

public static void main(String[] args) throws Exception {

    int ecode = ToolRunner.run(new multiInputFile(), args);
    System.exit(ecode);

}
}
```

Multiple input class

Set input file
for each
mapper

- One MR step produces a estimate of v
 - *The file is stored in HDFS (graph.txt)*
- _At the end of the iteration, we need to compare this with the previous iteration estimate of v
- After that, we use this file as the input for the next step of MR.
- Sample Python code example of computing Page Rank with Hadoop Map Reduce iterative computing will be demonstrated!!!
- code and the example – screenshots to the slide

- graph.txt (4 node graph, tab separated adjacency list)

A	B	C
B	C	
C	A	
D	C	

```
prafullata@prafullata-VirtualBox:~$ cat graph.txt
A      B      C
B      C
C      A
D      C
```

```
hdfs dfs -rm -r /pr_out1
```

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-
3.3.6.jar \
-input /graph.txt \
-output /pr_out1 \
-mapper pagerank_mapper.py \
-reducer pagerank_reducer.py \
-file pagerank_mapper.py \
-file pagerank_reducer.py
```


BIG DATA

Iteration

```
prafullata@prafullata-VirtualBox:~$  
prafullata@prafullata-VirtualBox:~$ cat pagerank_mapper.py  
#!/usr/bin/env python3  
import sys  
  
N = 4 # total nodes in the graph  
  
def try_float(s):  
    try:  
        return float(s)  
    except Exception:  
        return None  
  
for line in sys.stdin:  
    line = line.rstrip("\n")  
    if not line:  
        continue  
  
    # First try tab-splitting (Hadoop-friendly); fallback  
    cols = line.split("\t")  
    if len(cols) == 1:  
        cols = line.split()  
  
    if not cols:  
        continue  
  
    node = cols[0]
```

```
prafullata@prafullata-VirtualBox:~$ cat pagerank_reducer.py  
#!/usr/bin/env python3  
import sys  
  
d = 0.85  
N = 4  
  
current_node = None  
outlinks = []  
rank_sum = 0.0  
  
def emit(node, outlinks, rank_sum):  
    new_rank = (1 - d)/N + d*rank_sum  
    print(f"{node}\t{'|'.join(outlinks)}\t{new_rank}")  
  
for line in sys.stdin:  
    line = line.strip()  
    if not line:  
        continue  
  
    try:  
        node, value = line.split("\t", 1)  
    except ValueError:  
        continue  
  
    if current_node and node != current_node:  
        emit(current_node, outlinks, rank_sum)  
        outlinks = []
```

```
prafullata@prafullata-VirtualBox:~$ hdfs dfs -cat /pr_out1/part-00000
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further de
tails.
```

A	B C	0.25
B	C	0.14375
C	A	0.56875
D	C	0.03750000000000000006

```
prafullata@prafullata-VirtualBox:~$ hdfs dfs -cat /pr_out2/part-00000
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further de
tails.
```

A	B C	0.5209374999999999
B	C	0.14375
C	A	0.297812500000000004
D	C	0.03750000000000000006

```
prafullata@prafullata-VirtualBox:~$
```



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Issues with Hadoop

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Issue with Small Files

- Hadoop does not suit for small data files due to its high capacity design
- A small file is significantly smaller than the HDFS block size (default 128MB)
- Too many small files cause problem for managing the Namenode.

Slow Processing Speed

- Data is distributed and processed over the cluster in MapReduce which increases the time and reduces processing speed.

Support for Batch Processing only and No Real-time Data Processing

- Hadoop supports batch processing only, it does not process streamed data, and hence overall performance is slower

No Delta Iteration

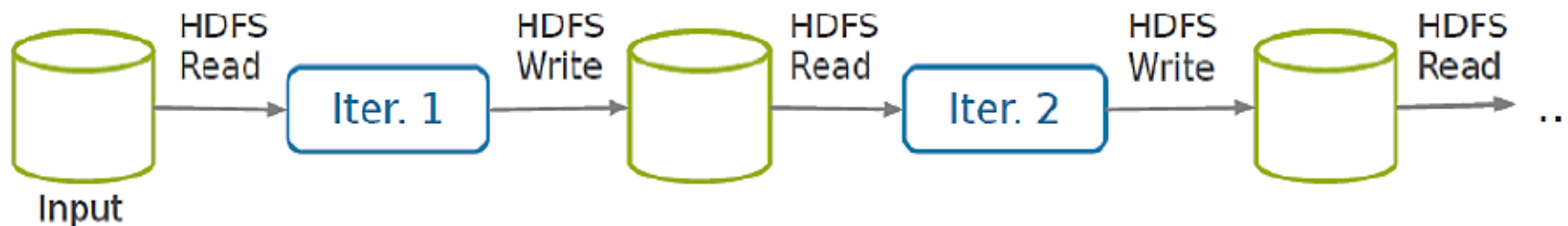
- Hadoop is not so efficient for iterative processing, as Hadoop does not support cyclic data flow(i.e. a chain of stages in which each output of the previous stage is the input to the next stage).

Programming Model

- Hard to implement everything as a mapreduce program.
- Multiple MapReduce steps are required for even simple operations.
 - E.g. Word count that sorts by their frequency.
- Lack of control, structure and data types.

No native support for iteration

- Each iteration reads/writes data from/to disk : overhead
- Need to design algorithms to reduce the number of iterations.



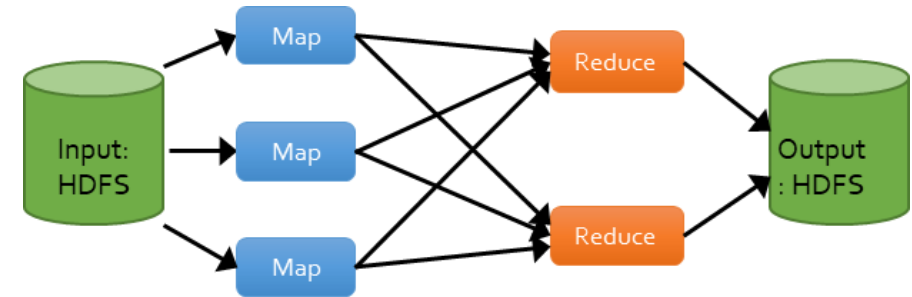
Efficiency

- Higher communication cost due to map, shuffle, combine and reduce tasks
- Frequent writing of output to disk
- Limited exploitation of Main memory

Not feasible for Real time stream processing

- MapReduce job needs to scan the entire input.

- Acyclic data flow
 - consider operating on a *data working set*
 - **Working set:** same set of data reused
 - in page rank, we keep computing importance vector and reusing in next iteration.
 - Hadoop – inefficient in such cases.
- Example of use
 - Where we need to *iterate*
 - Graph processing
 - Machine Learning
 - Where we need to do *interactive analysis*
 - Python, R
- On every iteration, storing/reloading of data from persistent storage is time consuming.



- Apache Spark was developed as an alternative **distributed processing framework** to address limitations of the MapReduce processing model.
- Apache Hadoop provides an ecosystem for Big Data storage, resource management, and processing, while Apache Spark primarily provides a distributed processing framework.
- Hadoop uses the MapReduce to process data, while Spark uses **resilient distributed datasets (RDDs)**.
- Hadoop has a distributed file system (HDFS), meaning that data files can be stored across multiple machines.
- The file system is scalable, since servers and machines can be added to accommodate increasing volumes of data. Spark does not provide a distributed file storage system, so it is **mainly used for computation**, on top of Hadoop.
- Spark does not need Hadoop to run, but can be used with Hadoop since it can create distributed datasets from files stored in the HDFS.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu